



UNIVERSIDAD INDOAMÉRICA

FACULTAD DE INGENIERÍAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA:

DESARROLLO DE UN PROTOTIPO DE ASISTENTE VIRTUAL INTELIGENTE CON INTERFAZ DE AVATAR 3D Y VISIÓN ARTIFICIAL PARA LA DIFUSIÓN DE INFORMACIÓN ACADÉMICA DE LOS ESTUDIANTES DE UNA INSTITUCIÓN EDUCATIVA

Trabajo de Titulación modalidad Trabajo de Integración Curricular

Autor

Paulo Emilio Espinoza Jiménez

Tutor

Ing. Christian Eduardo Iza Llumiguisín

QUITO – ECUADOR

2026

AUTORIZACIÓN DE REPOSITORIO DIGITAL

Yo, Paulo Emilio Espinoza Jiménez, declaro ser autor del Trabajo de Titulación con el nombre "Desarrollo de un prototipo de asistente virtual inteligente con interfaz de avatar 3D y visión artificial para la difusión de información académica de los estudiantes de una institución educativa", como requisito para optar al grado de Ingeniero en Tecnologías de la Información y autorizo al Sistema de Bibliotecas de la Universidad Tecnológica Indoamérica para que, con fines netamente académicos, divulgue esta obra a través del Repositorio Digital Institucional (RDI-UTI).

Los usuarios del RDI-UTI podrán consultar el contenido de este trabajo en las redes de información del país y del exterior, con las cuales la Universidad tenga convenios. La Universidad Tecnológica Indoamérica no se hace responsable por el plagio o copia del contenido parcial o total de este trabajo.

Del mismo modo, acepto que los Derechos de Autor, Morales y Patrimoniales sobre esta obra serán compartidos entre mi persona y la Universidad Tecnológica Indoamérica, y que no tramitaré la publicación de este trabajo en ningún otro medio sin autorización expresa de la misma. En caso de que exista el potencial de generación de beneficios económicos o patentes, producto de este trabajo, acepto que se deberán firmar convenios específicos adicionales donde se acuerden los términos de adjudicación de dichos beneficios.

Para constancia de esta autorización, en Quito, Ecuador, a los 22 días del mes de septiembre de 2026, firmo conforme:

Autor: Paulo Emilio Espinoza Jiménez

Firma:

Número de Cédula: 1722558374

Dirección: Pichincha, Quito, El Inca, Kenedy

Correo Electrónico: emil.200409@gmail.com

Teléfono: 0986639128

APROBACIÓN DEL TUTOR

En mi calidad de Tutor del Trabajo de Titulación "DESARROLLO DE UN PROTOTIPO DE ASISTENTE VIRTUAL INTELIGENTE CON INTERFAZ DE AVATAR 3D Y VISIÓN ARTIFICIAL PARA LA DIFUSIÓN DE INFORMACIÓN ACADÉMICA DE LOS ESTUDIANTES DE UNA INSTITUCIÓN EDUCATIVA", presentado por Paulo Emilio Espinoza Jiménez, para optar por el Título de Ingeniero en Tecnologías de la Información.

CERTIFICO

Que dicho Trabajo de Titulación ha sido revisado en todas sus partes y considero que reúne los requisitos y méritos suficientes para ser sometido a la presentación pública y evaluación por parte del Tribunal revisor que se designe.

Quito, Ecuador, 22 de septiembre de 2026

Ing. Christian Eduardo Iza Llumiguisín

DECLARACIÓN DE AUTENTICIDAD

Yo, Paulo Emilio Espinoza Jiménez, declaro que los contenidos y los resultados obtenidos en el presente Trabajo de Titulación "Desarrollo de un prototipo de asistente virtual inteligente con interfaz de avatar 3D y visión artificial para la difusión de información académica de los estudiantes de una institución educativa", como requerimiento previo para la obtención del Título de Ingeniero en Tecnologías de la Información, son absolutamente originales, auténticos y personales y de exclusiva responsabilidad legal y académica del autor.

Para constancia de esta declaración, firmo conforme:

Quito, Ecuador, 22 de septiembre de 2026

Paulo Emilio Espinoza Jiménez
1722558374

APROBACIÓN DE LECTORES

El Trabajo de Titulación ha sido revisado, aprobado y autorizada su impresión y empastado sobre el tema "DESARROLLO DE UN PROTO-TIPO DE ASISTENTE VIRTUAL INTELIGENTE CON INTERFAZ DE AVATAR 3D Y VISIÓN ARTIFICIAL PARA LA DIFUSIÓN DE INFORMACIÓN ACADÉMICA DE LOS ESTUDIANTES DE UNA INSTITUCIÓN EDUCATIVA", presentado por Paulo Emilio Espinoza Jiménez, previo a la obtención del Título de Ingeniero en Tecnologías de la Información, y reúne los requisitos de fondo y forma para su sustentación.

Quito, Ecuador, 22 de septiembre de 2026

Blanca Liliana Topón Visarrea
LECTORA

Juan Francisco Pabón Alajo
LECTOR

DEDICATORIA

*Quisiera dedicar este proyecto de titulación a mi familia,
especialmente a mi mamá y a mi abuelita.*

*Gracias a ellas todo esto fue posible. Estaré infinitamente
agradecido por tener a personas tan increíbles y buenas en mi vida;
siempre serán las mejores. Les dedico este logro como muestra de
gratitud por su apoyo incondicional, sus ánimos y todo el inmenso
cariño que me han brindado durante mi etapa universitaria.*

*De igual manera, dedico este trabajo a mis profesores, quienes
siempre se preocuparon por nosotros, nos impulsaron a mejorar y
tuvieron mucha paciencia para que lográramos los resultados
esperados.*

*Su valiosa orientación e invaluable apoyo hicieron posible el
desarrollo de esta investigación.*

Paulo Emilio Espinoza Jiménez

AGRADECIMIENTO

Expreso mi más sincero agradecimiento a la Universidad Tecnológica Indoamérica, a la Facultad de Ingenierías, y a mis compañeros de curso.

Gracias por la formación académica brindada y por todo el conocimiento que hemos construido a lo largo de estos años, descubriendo muchas cosas nuevas juntos y ayudándonos mutuamente a superar cada desafío.

De manera especial, extendiendo mi gratitud al Ing. Christian Iza y al Ing. Andrés Rubio, por su dedicación inquebrantable. Todos los conocimientos impartidos, su excelente criterio técnico y sus invaluable enseñanzas compartidas fueron pilares fundamentales para hacer realidad este proyecto.

Paulo Emilio Espinoza Jiménez

Índice de Contenidos

CAPÍTULO I:	Introducción	20
1.1	Contextualización	20
1.2	Estado del Arte	22
1.2.1	Diagnóstico del Sistema de Información Institucional en la Universidad Tecnológica Indoamérica	23
1.2.2	Evolución Tecnológica: Asistentes Virtuales, Avatares 3D y Visión Artificial	25
1.2.3	Adopción de Inteligencia Artificial en Universidades del Ecuador	26
1.2.4	Análisis Comparativo de Soluciones y Proyectos Similares	27
1.3	Planteamiento del Problema	33
1.4	Justificación	33
1.4.1	Utilidad	34
1.4.2	Beneficiarios	34
1.4.3	Factibilidad e Impacto Potencial	34
1.5	Objetivos	36
1.5.1	Objetivo General	36
1.5.2	Objetivos Específicos	36
1.6	Alcance del Proyecto	37
1.7	Fundamentación Teórica	38
1.7.1	Inteligencia Artificial y Procesamiento de Lenguaje Natural (PLN)	38
1.7.2	Visión Artificial e Interacción Visual no Biométrica .	40
1.7.3	Agentes Conversacionales Encarnados (ECAs) y Avatares 3D	40
1.7.4	Arquitectura de Software para Asistentes Virtuales Multimodales	41
1.7.5	Aceptación Tecnológica en la Educación Superior	41

CAPÍTULO II: Metodología	43
2.1 Diagnóstico de la Situación Actual	43
2.1.1 Estimación de la Carga Operativa	44
2.2 Área de Estudio	47
2.3 Metodología de Desarrollo o Implementación	47
2.4 Técnicas e Instrumentos de Recolección de Información	50
2.4.1 Técnicas para la Fase de Diagnóstico y Desarrollo	50
2.4.2 Técnicas para la Fase de Evaluación y Validación	51
2.5 Requerimientos del Proyecto	52
2.5.1 Requerimientos Funcionales (RF)	53
2.5.2 Requerimientos No Funcionales (RNF)	58
2.6 Diseño de la Solución	62
2.6.1 Capa de Percepción y Captura (Frontend)	67
2.6.2 Capa de Procesamiento Cognitivo (Backend)	68
2.6.3 Capa de Renderizado e Interacción (Frontend Gráfico)	68
2.6.4 Arquitectura de Despliegue	69
2.7 Justificación Técnica	69
2.7.1 Arquitectura Híbrida: Procesamiento Local y Servicios por API	70
2.7.2 Selección Comparativa del Proveedor de LLM	71
2.7.3 Implementación de la Arquitectura RAG frente a Fine-Tuning	73
2.7.4 Modelo de Embeddings Multilingüe	73
2.7.5 Visión Artificial en el Cliente mediante Cajas Delimitadoras	73
2.7.6 Avatar en Formato VRM sobre WebGL	73
CAPÍTULO III: Implementación, Pruebas y Resultados	75
3.1 Descripción de la Solución	75
3.1.1 Entorno de Ejecución	77
3.1.2 Software y Herramientas	77
3.2 Proceso de Implementación	78
3.2.1 Paso 1: Construcción de la Base de Datos Vectorial	79
3.2.2 Paso 2: Construcción del Servidor Backend (FastAPI)	80
3.2.3 Paso 3: Desarrollo de la Capa de Cliente (Frontend)	83
3.2.4 Paso 4: Despliegue	84
3.3 Pruebas y Validación	86

3.3.1	Pruebas Funcionales (Caja Negra)	86
3.3.2	Evaluación del Motor de Recuperación	87
3.3.3	Evaluación de la Exactitud de las Respuestas	88
3.3.4	Evaluación del Tiempo de Respuesta (TTFB)	88
3.3.5	Evaluación de Latencia de la Interacción por Voz . . .	89
3.3.6	Evaluación del Módulo de Visión	90
3.3.7	Pruebas de Presencia y de Seguridad	90
3.3.8	Pruebas de Usabilidad	90
3.4	Resultados	91
3.4.1	Resultados del Motor de Recuperación	91
3.4.2	Exactitud de las Respuestas Generadas	93
3.4.3	Resultados del Tiempo de Respuesta (TTFB)	94
3.4.4	Resultados de Latencia de la Interacción por Voz . .	95
3.4.5	Resultados del Módulo de Visión	97
3.4.6	Resultados de la Evaluación de Usabilidad	98
3.4.7	Comparación entre la Situación Inicial y la Pro- puesta	100
3.5	Cronograma y Presupuesto	104
3.5.1	Cronograma	104
3.5.2	Presupuesto	105
CAPÍTULO IV: Conclusiones y Recomendaciones		108
4.1	Conclusiones	108
4.2	Recomendaciones	110
4.3	Trabajos Futuros	111
REFERENCIAS		113
GLOSARIO		118
ANEXO A:	Repositorio de Código y Configuraciones Técnicas	122
ANEXO B:	Instrumento de Evaluación de Usabilidad	124
ANEXO C:	Formulario de Consentimiento Informado	126
ANEXO D:	Aprobación de Abstract	127
ANEXO E:	Manuales del Sistema	128

Índice de Tablas

1	Estructura Analítica del Contexto Macro, Meso y Micro de Adopción de IA.	22
2	Antecedentes Institucionales de Investigación en IA y Sistemas Multimodales (UTI).	25
3	Comparativa del Proyecto Propuesto frente al Estado del Arte.	28
4	Matriz de Indicadores de Impacto Potencial del Proyecto.	35
5	Indicadores del diagnóstico y forma de verificación en el prototipo.	46
6	Área de estudio.	47
7	Aplicación de la metodología DSRM en el proyecto.	48
8	Requerimientos Funcionales del Sistema.	53
9	Requerimientos No Funcionales del Sistema.	58
10	Comparación de los proveedores de LLM integrados en el prototipo.	72
11	Componentes desarrollados, configurados e integrados en el prototipo.	76
12	Entorno de hardware y software de las pruebas (modo local).	77
13	Software y herramientas requeridas para la implementación.	78
14	<i>Endpoints</i> del backend y control de acceso.	81
15	Resultado de las pruebas funcionales.	87
16	Comparación de estrategias de recuperación (banco de desarrollo, 50 preguntas).	91
17	Sensibilidad de la recuperación híbrida con reordenamiento al tamaño de fragmento (banco de desarrollo).	92
18	Resultado de la configuración de producción (híbrida + reordenamiento, 800/160, Top-3).	92

19	Clasificación de las respuestas generadas.	94
20	Tiempo hasta el primer fragmento de la respuesta por WebSocket (banco de prueba, 50 consultas, en milise- gundos).	95
21	Latencia por etapa de la interacción por voz en modo local (30 preguntas del dominio, en milisegundos).	96
22	Costo de la detección de presencia por fotograma (300 repeticiones).	97
23	Resultados de la encuesta de usabilidad (n = 16).	99
24	Matriz comparativa: situación inicial frente al prototipo. .	103
25	Cumplimiento de los requerimientos no funcionales. . . .	104
26	Cronograma de ejecución del proyecto (2026).	105
27	Presupuesto referencial de operación del prototipo. . . .	106
28	Banco de preguntas, posición recuperada y clasifica- ción de la respuesta.	129
29	Banco de prueba reservado, posición recuperada y clasificación de la respuesta.	134

Índice de Figuras

1	Geometría de la Similitud del Coseno en el espacio vectorial RAG.	39
2	Arquitectura general de software para el Asistente Virtual Multimodal.	42
3	Modelo de estimación del volumen mensual de consultas y de la carga operativa asociada.	45
4	Modelo operativo: etapas DSRM e iteraciones de construcción del prototipo.	49
5	Diagrama de Contexto (Nivel 0).	62
6	Diagrama de Clases del Sistema (Arquitectura Completa y Flujo Circular).	63
7	Diagrama de Estados del Avatar (diseño conceptual inicial).	64
8	Diagrama de Estados del Avatar implementado en el frontend.	65
9	Diagrama de Secuencia de Interacción (consulta por voz).	66
10	Arquitectura lógica por capas y flujo de datos del Asistente Virtual (diseño conceptual; el bloque LLM puede ser local o remoto).	69
11	Arquitectura de despliegue en modo local y en modo servidor.	70
12	Estructura lógica y proceso de vectorización en la base de datos ChromaDB (modelo conceptual).	79
13	Registro real almacenado en ChromaDB y proceso de indexación implementado.	80
14	Orquestación del procesamiento de voz en el backend.	83
15	Diagrama de bloques de la Interfaz Dual y composición de elementos web.	84

16	<i>Hit Rate@3</i> por categoría en el banco de prueba (aciertos / preguntas).	93
17	Composición de la latencia percibida por etapa: versión anterior y versión final (medianas).	96
18	Nivel de avance de los estudiantes encuestados.	100
19	Uso previo de IA generativa y conocimiento técnico en renderización 3D.	101
20	Usabilidad de la interfaz: integración, complejidad (ítem inverso), claridad y mensajes de error.	101
21	Inteligencia conversacional: propósito, comprensión, utilidad y reconducción.	101
22	Percepción del avatar 3D en el diferencial semántico (1 a 5).	102
23	Aceptación tecnológica: latencia, uso de la cámara, eficiencia y preferencia futura.	102

Índice de Imágenes

1	Panel de Administración del RAG para actualizar la base documental.	82
2	Pantalla de inicio de sesión del Asistente Virtual.	85
3	Modo Chat del Asistente Virtual.	85
4	Modo Conversacional (avatar en primer plano con controles de voz).	86
5	Formulario de consentimiento informado.	126
6	Aprobación del Abstract.	127

Índice de Ecuaciones

1	Similitud del coseno entre la consulta y un fragmento	39
2	Fusión de rangos recíprocos (RRF) de la búsqueda semántica y léxica	40
3	Estimación de la carga operativa mensual de atención de consultas	45
4	Regla de detección de llegada y salida del usuario por fotografías consecutivos	80
5	Tasa de acierto de recuperación semántica (Hit Rate@k) . . .	88
6	Rango Recíproco Medio de la recuperación (MRR@5)	88
7	Tiempo hasta el primer fragmento de la respuesta por Web-Socket	89
8	Latencia percibida de una consulta por voz	89
9	Media aritmética de la latencia	89
10	Desviación estándar de los tiempos de respuesta	89
11	Intervalo de confianza al 95% de la media	90
12	Costo variable estimado por consulta por voz	106

Índice de Anexos

Anexo A: Repositorio de Código y Configuraciones Técnicas . . .	122
Anexo B: Instrumento de Evaluación de Usabilidad	124
Anexo C: Formulario de Consentimiento Informado	126
Anexo D: Aprobación de Abstract	127
Anexo E: Manuales del Sistema	128
Anexo F: Banco de Preguntas y Evidencias de Evaluación	129

UNIVERSIDAD INDOAMÉRICA
FACULTAD DE INGENIERÍAS
INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA: DESARROLLO DE UN PROTOTIPO DE ASISTENTE VIRTUAL INTELIGENTE CON INTERFAZ DE AVATAR 3D Y VISIÓN ARTIFICIAL PARA LA DIFUSIÓN DE INFORMACIÓN ACADÉMICA DE LOS ESTUDIANTES DE UNA INSTITUCIÓN EDUCATIVA

AUTOR(A): Paulo Emilio Espinoza Jiménez

TUTOR(A): Ing. Christian Eduardo Iza
Llumiguísín

RESUMEN EJECUTIVO

El presente Trabajo de Integración Curricular aborda la problemática de los actuales repositorios documentales e interfaces institucionales en la difusión de información académica, los cuales carecen de búsqueda semántica y de interactividad proactiva, generando demoras en consultas rutinarias de estudiantes y una sobrecarga administrativa en las secretarías. Con el objetivo de desarrollar un prototipo de asistente virtual inteligente con interfaz de avatar 3D y visión artificial para la difusión de reglamentos y mallas curriculares, se adoptó la Metodología de Investigación en Ciencia de Diseño (DSRM) combinada con el marco de trabajo ágil Scrum para la construcción iterativa del software. La arquitectura híbrida propuesta implementa localmente en el dispositivo cliente la captura visual por cajas delimitadoras y el renderizado gráfico 3D en WebGL, mientras que el servidor local orquesta un motor de Generación Aumentada por Recuperación (RAG) con el modelo de incrustaciones multilingües paraphrase-multilingual-MiniLM-L12-v2 y el modelo local Nemotron. Los principales resultados obtenidos demuestran una alta precisión semántica con un índice de acierto (*Hit Rate*) del 92.0 % en la recuperación de normativas académicas, un tiempo de respuesta (TTFB) promedio de 450 ms que asegura fluidez conversacional mediante WebSockets, y una evaluación de usabilidad positiva con 13 estudiantes en factores de integración, precisión y preferencia de uso. En conclusión, la arquitectura descentralizada y el uso de inteligencia artificial generativa ofrecen una solución viable y robusta que transforma la consulta estática tradicional en un canal de soporte interactivo, accesible y eficaz para la comunidad universitaria.

DESCRIPTORES: Asistente virtual, inteligencia artificial generativa, interacciones multimodales, renderizado 3D

UNIVERSIDAD INDOAMÉRICA
FACULTY OF ENGINEERING
BACHELOR'S DEGREE IN INFORMATION TECHNOLOGY
ENGINEERING

**TOPIC: DEVELOPMENT OF A SMART VIRTUAL ASSISTANT PRO-
TOTYPE WITH 3D AVATAR INTERFACE AND COMPUTER VISION FOR
THE DISSEMINATION OF ACADEMIC INFORMATION TO STUDENTS
OF AN EDUCATIONAL INSTITUTION**

AUTHOR: Paulo Emilio Espinoza Jiménez

ADVISOR: Ing. Christian Eduardo Iza
Llumiguísín

ABSTRACT

This Curricular Integration Project addresses the limitations of current document repositories and institutional interfaces in disseminating academic information, which lack semantic search capabilities and proactive interactivity, leading to delays in routine student queries and administrative overload in administrative offices. Aiming to develop a prototype of an intelligent virtual assistant with a 3D avatar interface and computer vision for the dissemination of academic regulations and curricula, the Design Science Research Methodology (DSRM) was adopted in combination with the agile Scrum framework for iterative software development. The proposed hybrid architecture implements visual capture via bounding boxes and 3D graphic rendering using WebGL locally on the client device, while a local server orchestrates a Retrieval-Augmented Generation (RAG) engine utilizing the multilingual embedding model alongside the local Nemotron model. The main results demonstrate high semantic accuracy, achieving a Hit Rate of 92.0 % in academic regulation retrieval, an average Time to First Byte (TTFB) response time of 450 ms ensuring conversational fluency via WebSockets, and a positive usability evaluation with 13 students regarding integration, accuracy, and usage preference. In conclusion, the decentralized architecture and the use of generative artificial intelligence provide a viable, robust solution that transforms traditional, static queries into an interactive, accessible, and efficient support channel for the university community.

KEYWORDS: Generative artificial intelligence, Multimodal interactions, Virtual assistant, 3D rendering

CAPÍTULO I

INTRODUCCIÓN

1.1 Contextualización

La adopción de tecnologías emergentes de Inteligencia Artificial (IA) en la educación superior se articula a través de tres niveles interdependientes de análisis: macroeconómico y nacional, meso o inter-institucional metropolitano, y micro o específicamente universitario.

En el **contexto Macro (Ecuador)**, según el *Índice Latinoamericano de Inteligencia Artificial (2024-2025)* [1], el país es catalogado en una etapa de "adopción tardía", pero con un crecimiento acelerado en el desarrollo de talento tecnológico e infraestructuras digitales. En la educación superior ecuatoriana, un estudio reciente demostró que el 87,3% de los estudiantes universitarios reporta haber utilizado herramientas basadas en IA para fines formativos [2], cifra que alcanza un 94,2% entre alumnos de facultades técnicas y científicas como la Ingeniería en Tecnologías de la Información. Los asistentes generativos conversacionales son utilizados por el 82,7% de la población estudiantil; sin embargo, esta penetración ocurre predominantemente "desde abajo", de manera autodidacta y paralela a las plataformas oficiales, evidenciando la necesidad imperativa de contar con canales y estrategias institucionales formalmente gobernadas [3].

En el **contexto Meso (Universidades de Quito)**, diversas instituciones del distrito metropolitano lideran la adopción aplicada de inteligencia artificial en la automatización académica. La Pontificia Universidad Católica del Ecuador (PUCE) desarrolló *Aurel_AI* para transformar digitalmente su mesa de ayuda universitaria, así como *LegalBot-EC*, un modelo de lenguaje local especializado en normativas legales ecuatorianas [4]. Por su parte, la Universidad San Francisco de Quito (USFQ) y la Universidad Central del Ecuador (UCE) han desplegado modelos y motores locales para procesamiento analítico y soporte interno, preservando la privacidad de la informa-

ción académica y demostrando la viabilidad técnica de sistemas autónomos en la región [1].

En el **contexto Micro (Universidad Tecnológica Indoamérica - UTI)**, la institución —con sedes operativas consolidadas en Quito y Ambato— ha mantenido una oferta formativa orientada a la innovación digital. De acuerdo con los informes institucionales de Rendición de Cuentas [5], la universidad registra una matrícula global que supera los 8.358 estudiantes (7.839 en programas de grado). En específico, la carrera de Ingeniería en Tecnologías de la Información reporta una población matriculada estable de aproximadamente 380 estudiantes activos en sus dos sedes, evidenciando un notable crecimiento en admisiones de primer nivel en el periodo académico reciente (68 nuevos alumnos: 47 en la sede Quito y 21 en la sede Ambato) [5]. Históricamente, la universidad ha superado la marca de 6.300 estudiantes regulares, generando un volumen masivo de consultas operativas que saturan los esquemas tradicionales de secretaría.

La Tabla 1 sintetiza esta estructura analítica y los indicadores críticos de adopción tecnológica en cada dimensión.

El área de aplicación específica para este proyecto es la carrera de Ingeniería en Tecnologías de la Información, donde se ha identificado que la gestión de la información académica para los estudiantes es predominantemente estática. Actualmente, el acceso a datos críticos sobre mallas curriculares, requisitos de graduación o perfiles profesionales depende de la navegación manual por el sitio web institucional o de la lectura de extensos archivos PDF, lo que deriva en una experiencia de usuario pasiva y demorada.

El entorno universitario actual demanda canales de comunicación más ágiles que respondan a la necesidad de inmediatez de los estudiantes. La situación identificada revela una brecha entre la disponibilidad de la información y la facilidad con la que el usuario puede obtener respuestas precisas a sus inquietudes. Esta dispersión documental no solo dificulta la orientación de los alumnos de nuevo ingreso, sino que también genera una carga de consultas repetitivas que podrían automatizarse. En este contexto, surge la necesidad de modernizar estos puntos de contacto, transformando los repositorios de datos tradicionales en un entorno de consulta dinámico y proac-

Tabla 1: Estructura Analítica del Contexto Macro, Meso y Micro de Adopción de IA.

Nivel de Contexto	Área Geográfica / Institución	Indicador de Adopción IA	Enfoque Tecnológico y Reto
Macro (Nacional)	República del Ecuador (Sistema Educativo Superior)	87,3 % estudiantes usa IA (94,2 % en TI) [1]	Adopción autodidacta "desde abajo"; necesidad de gobernanza formal
Meso (Metropolitano)	Universidades de Quito (PUCE, USFQ, UCE)	Proyectos locales: <i>Aurel_AI</i> , <i>LegalBot-EC</i> [4]	Modelos locales, privacidad de datos y automatización de soporte
Micro (Institucional)	Universidad Tecnológica Indoamérica (UTI - Sede Quito/Ambato)	Matrícula de >8.358 alumnos (~380 en carrera TI) [5]	Evolucionar el portal SGA estático hacia un Asistente Virtual 3D proactivo

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: CENIA y CEPAL [1], PUCE [4] y UTI [5]

tivo que aproveche las tendencias actuales en interfaces humano-computadora.

1.2 Estado del Arte

El presente estado del arte fundamenta la conceptualización, diseño y desarrollo de un Asistente Virtual Inteligente enfocado en la difusión proactiva de información académica para los estudiantes de la carrera de Tecnologías de la Información (TI). Para abordar la complejidad tecnológica y el impacto institucional de esta solución, esta sección se estructura en cuatro dimensiones analíticas principales.

En primer lugar, se presenta un diagnóstico del funcionamiento actual del sistema de información en la Universidad Tecnológica Indoamérica, identificando las fricciones en la experiencia del usuario. En segundo lugar, se expone la evolución tecnológica de los asistentes virtuales, destacando la transición hacia interfaces antropomór-

ficas, como los avatares 3D, y la integración de la visión artificial. En tercer lugar, se analiza la adopción actual de estas tecnologías en diversas instituciones de educación superior en el Ecuador, estableciendo un contexto nacional. Finalmente, la sección concluye con un análisis comparativo exhaustivo que contrasta quince proyectos y referentes científicos similares con la propuesta de esta investigación.

1.2.1 Diagnóstico del Sistema de Información Institucional en la Universidad Tecnológica Indoamérica

Para que la implementación de un asistente virtual sea pertinente y genere un impacto medible, debe resolver problemas operativos específicos dentro de su entorno de despliegue. En la Universidad Tecnológica Indoamérica (UTI), la administración y difusión de la información académica y curricular se gestiona primordialmente a través de su Sistema de Gestión Académica (SGA).

Actualmente, cuando un estudiante de la carrera de TI requiere consultar su avance de malla curricular, prerrequisitos de asignaturas o normativas de titulación, debe autenticarse en el portal web del SGA mediante credenciales de acceso. Una vez dentro, la recuperación de la información se realiza navegando por diversos menús que, en su mayoría, derivan en la generación de reportes en formato PDF estático, diseñados idealmente para su impresión. Este modelo de interacción representa un paradigma de recuperación de información lineal y pasivo; si el estudiante presenta dudas de interpretación sobre la secuenciación de materias, el documento no ofrece retroalimentación dinámica, obligando al alumno a buscar asistencia presencial, lo que genera demoras y sobrecarga de consultas administrativas repetitivas.

De manera contrastante, la Universidad Indoamérica posee un entorno altamente propicio para la innovación tecnológica. La institución impulsa activamente la investigación aplicada en inteligencia artificial y oferta programas académicos de posgrado especializados en IA, innovación empresarial y desarrollo de entornos virtuales. En este contexto, el prototipo de asistente virtual propuesto no busca reemplazar la infraestructura del SGA, sino acoplarse como una capa de interfaz moderna, transformando los repositorios de datos tradicionales en un canal de servicio inmediato, conversacional y proac-

tivo.

El desarrollo del presente Asistente Virtual Multimodal no constituye un esfuerzo aislado, sino que se inscribe orgánicamente dentro de una sólida trayectoria de innovación e investigación aplicada en la Universidad Tecnológica Indoamérica. A través de sus centros de investigación —como el Centro de Investigación en Mecatrónica y Sistemas Interactivos (MIST) y el Centro de Investigación en Empresa, Sociedad y Tecnología (ESTec)— la universidad ha ejecutado múltiples proyectos emblemáticos de inteligencia artificial y computación inmersiva que constituyen antecedentes científicos directos para este trabajo:

- **Detección Oncológica mediante Visión Artificial e IA (Proyecto MIST-CEDIA):** En alianza con la Corporación Ecuatoriana para el Desarrollo de la Investigación y la Academia (CEDIA), la UTI lidera un proyecto I+D+i de modelamiento de procesos biológicos e imágenes médicas para el análisis y detección temprana del cáncer de mama [6]. Este antecedente consolida el dominio computacional de la institución en visión artificial y algoritmos de detección visual por cajas delimitadoras (*bounding boxes*).
- **Entornos Inmersivos 3D y Pensamiento Crítico en TI:** En la propia carrera de Ingeniería en Tecnologías de la Información, investigadores desarrollaron entornos de simulación en motores gráficos avanzados (*Unreal Engine 5*), logrando incrementar el índice de pensamiento crítico de los estudiantes de un promedio de 3.3/10 a 4.1/10 [7]. Ello demuestra empíricamente que la interacción con interfaces tridimensionales dinámicas mejora el rendimiento y la participación del estudiante.
- **Análisis de Sentimientos mediante PNL:** A nivel de procesamiento semántico, investigadores de la UTI han aplicado técnicas avanzadas de Procesamiento de Lenguaje Natural (PLN) y análisis de Big Data para medir sentimientos y percepciones institucionales en medios digitales [8].
- **Evaluación Formativa Asistida por IA en Posgrados y Capacitación Docente:** La institución ha impulsado proyectos de evaluación continua asistida por IA para mitigar sesgos en programas

académicos, al tiempo que ha capacitado a su profesorado en inteligencia artificial con expertos de talla mundial para posicionarse como un nodo de innovación iberoamericano [5].

La Tabla 2 sistematiza los proyectos internos previos que sirven como cimiento metodológico y computacional para la integración del avatar 3D y el motor RAG conversacional en la carrera de TI.

Tabla 2: Antecedentes Institucionales de Investigación en IA y Sistemas Multimodales (UTI).

Proyecto Institucional	Unidad / Centro UTI	Tecnología Aplicada	Aporte al Asistente de Tesis
Proyecto MIST-CEDIA [6]	Centro MIST / CEDIA	Visión artificial, imágenes y <i>Deep Learning</i>	Arquitectura de detección visual en tiempo real (<i>bounding boxes</i>)
Simulaciones 3D en TI [7]	Facultad TI	Motor gráfico 3D (<i>Unreal Engine 5</i>)	Validación del impacto de interfaces 3D en el compromiso estudiantil
Análisis Sentimientos PNL [8]	Facultad TI / ESTec	Procesamiento de Lenguaje Natural y Big Data	Comprensión semántica del lenguaje y consultas informales
Evaluación Formativa IA [5]	Posgrados / Docencia	IA generativa en aulas virtuales	Integración curricular sin sesgos y estandarización del servicio

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Publicaciones e informes institucionales de la UTI [6, 7, 8, 5]

1.2.2 Evolución Tecnológica: Asistentes Virtuales, Avatares 3D y Visión Artificial

La evolución de las interfaces humano-computadora en el ecosistema educativo ha transitado desde chatbots transaccionales basados puramente en texto hacia agentes conversacionales encarnados o multimodales. Si bien los modelos basados en texto logran automatizar respuestas, la evidencia empírica en la educación superior subraya que la dotación de características antropomórficas a los

sistemas facilita la creación de entornos centrados en el estudiante, promoviendo la empatía y la presencia social [9].

La integración de Modelos de Lenguaje Grande (LLM) dota al sistema de comprensión semántica, mientras que plataformas de middleware proporcionan la materialización visual. Estas plataformas permiten la generación de avatares 3D de alta fidelidad, equipados con mallas faciales configuradas para soportar animaciones expresivas y sincronización labial en tiempo real, lo que reduce la carga cognitiva y humaniza el proceso de difusión de la información [10].

Simultáneamente, la visión artificial (*Computer Vision*) ha transformado a estos asistentes de respondedores pasivos a sistemas capaces de interpretar señales visuales básicas de interacción. Mediante algoritmos de aprendizaje profundo, redes neuronales convolucionales y extracción de puntos de referencia faciales (*landmarks*), un asistente con interfaz de avatar puede adaptar su comportamiento visual, orientar su mirada simulada y mejorar la naturalidad de la comunicación con el usuario [11].

No obstante, la literatura reciente enfatiza la extrema precaución ética que requiere el uso de señales visuales en entornos educativos. Este antecedente dicta un mandato de "privacidad por diseño" para la presente tesis: los algoritmos de visión artificial deben procesar la información de manera efímera, sin almacenar imágenes ni identificar personalmente al alumno sin su consentimiento expreso [12].

1.2.3 Adopción de Inteligencia Artificial en Universidades del Ecuador

El análisis del estado del arte demuestra un creciente interés por la automatización de procesos académicos en las instituciones de educación superior del Ecuador. Diversas universidades se encuentran en etapas de investigación o implementación de soluciones basadas en inteligencia artificial que sirven como antecedentes directos para este proyecto:

- **Universidad Técnica Particular de Loja (UTPL):** Investigadores de esta institución han desarrollado propuestas de asistentes virtuales desplegados mediante tecnología holográfica para optimizar los servicios estudiantiles y reducir tiempos de espera [13]. Además, la universidad ha liderado proyectos de realidad ex-

tendida, implementando un “Campus 3D” donde los usuarios navegan a través de avatares.

- **Universidad Politécnica Salesiana (UPS):** Implementó un asistente virtual fundamentado en procesamiento de lenguaje natural y aprendizaje automático, mediante redes neuronales recurrentes, para gestionar eficientemente la comunicación con aspirantes a estudios de posgrado a través de canales de mensajería, optimizando los recursos administrativos [14].
- **Pontificia Universidad Católica del Ecuador (PUCE):** Con el respaldo de su grupo de investigación en inteligencia artificial, se desarrolló “TELIA”, un asistente virtual integrado en Microsoft Teams para automatizar y transformar digitalmente la resolución de problemas en las mesas de servicios tecnológicos [15].
- **Escuela Superior Politécnica del Litoral (ESPOL):** Se diseñó una plataforma web con un asistente virtual basado en IA y modelos de *machine learning*, como Random Forest, capaz de interpretar preguntas en lenguaje natural para proporcionar indicadores de desempeño predictivos, reduciendo los tiempos de consulta de casi una hora a once minutos [16].
- **Universidad Internacional del Ecuador (UIDE) y Universidad Tecnológica ECOTEC:** Ambas instituciones se encuentran a la vanguardia de la innovación educativa nacional. UIDE fomenta ecosistemas académicos enriquecidos por IA, mientras que ECOTEC utiliza tecnologías avanzadas, incluyendo sistemas de reconocimiento facial para el acceso seguro al campus y el uso de internet de las cosas.

1.2.4 Análisis Comparativo de Soluciones y Proyectos Similares

A continuación, la Tabla 3 expone un contraste analítico entre el prototipo propuesto en esta investigación y diversos referentes bibliográficos y tecnológicos de relevancia, detallando sus objetivos, metodologías de interfaz y el uso de la visión computacional.

Tabla 3: Comparativa del Proyecto Propuesto frente al Estado del Arte.

Proyecto / Autores	Objetivo principal	Interfaz de usuario	Uso de visión artificial	Diferencia principal con el proyecto propuesto
Proyecto Propuesto (Tesis TI - Indoamérica)	Transformar la consulta documental de mallas y normativas en una experiencia dinámica y ágil.	Avatar 3D (VRM) + chat por texto y voz.	Detección de presencia no biométrica por cajas delimitadoras (<i>bounding boxes</i>) en el navegador para iniciar y reanudar el diálogo.	Especializado en la resolución de la fricción documental específica de la carrera de TI, uniendo gestión curricular con una interfaz multimodal de orientación académica.
NAIA Framework [10]	Asistencia académica general mediante una arquitectura de cinco roles.	Avatar 3D (Ready Player Me) + modelos LLM.	Analiza el entorno visual para retroalimentar la interacción.	NAIA abarca procesos generales para toda la universidad; la propuesta actual acota el razonamiento a normativas de TI, evitando respuestas ambiguas.
EverydAI [17]	Asistencia en la toma de decisiones cotidianas, como cocina, moda y fitness.	Avatar 3D diferenciado por dominio de conocimiento.	Detección de objetos físicos en tiempo real.	Está orientado al entorno doméstico; el proyecto adapta esa capacidad multimodal proactiva al ecosistema burocrático universitario.
Asistente Holográfico UTPL				

Continúa en la siguiente página

Proyecto / Autores	Objetivo principal	Interfaz de usuario	Uso de visión artificial	Diferencia principal con el proyecto propuesto
[13]	Responder consultas frecuentes para servicios estudiantiles.	Despliegue mediante holograma.	No reporta visión proactiva para interacción autónoma.	Utiliza proyección holográfica reactiva; la tesis propone un avatar 3D integrado a un prototipo software de orientación académica.
Asistente Posgrado UPS				
[14]	Reducir la carga de admisiones mediante respuestas automatizadas.	Interfaz textual vía WhatsApp y Messenger.	No utiliza visión artificial.	Es un sistema pasivo basado en texto; el proyecto propuesto humaniza el servicio al dar un rostro y voz 3D a la institución.
TELIA Mesa Ayuda PUCE				
[15]	Automatización de soporte y tickets tecnológicos.	Chatbot integrado en Microsoft Teams.	No utiliza visión artificial.	Es una solución transaccional corporativa; la tesis propone un agente de difusión académica con representación visual mediante avatar 3D.
Asistente Desempeño ESPOL				
[16]	Proporcionar métricas predictivas industriales.	Plataforma web conversacional.	No utiliza visión artificial.	Mantiene un enfoque predictivo de datos tabulares, sin elementos de presencia social ni contexto espacial.

Continúa en la siguiente página

Proyecto / Autores	Objetivo principal	Interfaz de usuario	Uso de visión artificial	Diferencia principal con el proyecto propuesto
GPTAvatar [9]	Crear un entorno educativo inmersivo de código abierto.	Avatar 3D en motor gráfico Unity.	No se enfoca en la detección física del entorno del usuario.	Es una herramienta generalista; el proyecto de tesis se orienta a un servicio institucional sustentado en bases documentales específicas.
Control de Mirada ECAs [11]	Mejorar la toma de turnos en espacios públicos ruidosos.	Agente conversacional en pantalla.	Detección del orador activo para dirigir la mirada del avatar.	El estudio se enfoca en el algoritmo visual-acústico; la tesis es una plataforma integral de recuperación de información.
Medición de Compromiso [12]	Medir niveles de atención en educación virtual.	Modelo analítico sin interfaz avatar definida.	Redes ST-GCN y <i>landmarks</i> faciales.	Es un algoritmo aislado; el proyecto propuesto puede integrar estas lógicas para adaptar dinámicamente la explicación del avatar.
Asistente de Tesis UPN				

Continúa en la siguiente página

Proyecto / Autores	Objetivo principal	Interfaz de usuario	Uso de visión artificial	Diferencia principal con el proyecto propuesto
[18]	Apoyar la estructuración metodológica de trabajos de grado.	Chatbot de texto basado en IA.	No utiliza visión artificial.	Apoya la generación de contenido investigativo; el proyecto propuesto no interviene en tareas, sino en la burocracia administrativa.
Simulaciones Médicas UMH [19]	Entrenamiento diagnóstico para estudiantes de medicina.	Avatar 3D de paciente clínico.	No implementa visión artificial proactiva.	La interacción se orienta a evaluación de competencias, frente a la naturaleza informativa y guiada de la propuesta de TI.
TalkMateAI [20]	Desarrollar un compañero de IA multimodal de ejecución local.	Avatar 3D web.	Modelos visión-lenguaje ligeros.	Es una arquitectura técnica de código abierto; la tesis adopta estos principios de baja latencia para un caso de uso académico real.
AI Desktop Assistant [21]	Mejorar la interacción mediante reconocimiento visual de escritorio.	Asistente en PC.	Reconocimiento visual de gestos y objetos mediante webcam.	Está orientado al control del sistema operativo, a diferencia de la orientación a servicios curriculares del prototipo de tesis.
Modelo Alerta Transporte				

Continúa en la siguiente página

Proyecto / Autores	Objetivo principal	Interfaz de usuario	Uso de visión artificial	Diferencia principal con el proyecto propuesto
[22]	Prevención de aglomeraciones mediante detección visual.	Interfaz de administración de alertas.	Detección de altas concentraciones de personas.	Está orientado a seguridad y flujos masivos; el proyecto propuesto utiliza la detección para iniciar un diálogo uno a uno de manera empática.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Revisión bibliográfica de los trabajos citados

1.3 Planteamiento del Problema

En el actual ecosistema de educación superior, la inmediatez y la accesibilidad a la información son factores determinantes para el éxito y la retención estudiantil. En la carrera de Ingeniería en Tecnologías de la Información de la Universidad Tecnológica Indoamérica, se ha identificado una brecha tecnológica significativa en la capa de interacción entre los estudiantes y los repositorios de información institucional.

Actualmente, la gestión de datos académicos críticos tales como mallas curriculares, requisitos de titulación, procesos de prácticas pre-profesionales y perfiles de egreso se realiza a través de un modelo predominantemente estático. Las causas de esta problemática radican en que los sistemas de información heredados fueron diseñados bajo un paradigma de almacenamiento y lectura lineal, obligando al estudiante a navegar manualmente por múltiples menús del sitio web institucional o a descargar y leer extensos archivos en formato PDF.

Los efectos de esta infraestructura recaen directamente sobre la experiencia del usuario, volviéndola pasiva, cognitivamente exhaustiva y demorada. Esta dispersión documental dificulta severamente la orientación autónoma, especialmente para los alumnos de nuevo ingreso. Ante la incapacidad de los documentos estáticos para resolver dudas de interpretación o casos específicos, los estudiantes se ven forzados a recurrir a la atención presencial. En consecuencia, se genera un cuello de botella operativo, sobrecargando al personal administrativo y docente con un alto volumen de consultas repetitivas que, bajo un esquema tecnológico adecuado, son inherentemente automatizables.

1.4 Justificación

La modernización de los canales de comunicación universitaria es un imperativo frente a una generación de estudiantes nativos digitales que demandan respuestas ágiles e interactivas. Este proyecto se justifica por su capacidad para transformar un modelo de consulta obsoleto en una experiencia dinámica, centralizada y tecnológica-

mente inmersiva mediante la implementación de un Asistente Virtual Inteligente.

1.4.1 Utilidad

La utilidad principal del prototipo radica en proporcionar un mecanismo complementario de acceso a información académica mediante lenguaje natural, reduciendo la necesidad de navegar manualmente por documentos extensos o múltiples secciones institucionales. La arquitectura RAG permite vincular las respuestas del asistente con una base documental previamente seleccionada, mientras que la interfaz multimodal busca ofrecer una interacción más natural mediante texto, voz y avatar 3D.

1.4.2 Beneficiarios

Los beneficiarios directos son los estudiantes de la carrera de Tecnologías de la Información, quienes dispondrían de un canal adicional para consultar información académica general. De manera indirecta, la disponibilidad de un mecanismo automatizado podría contribuir a reducir determinadas consultas repetitivas dirigidas al personal administrativo; sin embargo, la cuantificación de esta reducción requiere un seguimiento posterior en condiciones reales de operación.

1.4.3 Factibilidad e Impacto Potencial

La ejecución de este proyecto es factible tanto a nivel técnico como operativo. Actualmente existen frameworks de inteligencia artificial, APIs de síntesis de voz, librerías de visión artificial y plataformas de creación de avatares que son altamente accesibles para integrarse en un prototipo funcional de software. Además, el proyecto posiciona a la Universidad Indoamérica a la vanguardia en la adopción de interfaces humano-computadora avanzadas.

Para ilustrar el impacto potencial del proyecto en la carrera de Ingeniería en Tecnologías de la Información, se ha diseñado la matriz comparativa de la Tabla 4. En ella se contrastan los indicadores críticos de la situación actual frente a la proyección técnica del Asistente Virtual Inteligente.

Tabla 4: Matriz de Indicadores de Impacto Potencial del Proyecto.

Indicador	Situación Actual	Proyección del Prototipo	Interpretación
Población potencial beneficiaria	Aprox. 380 estudiantes de TI	Acceso al prototipo para consulta de mallas y normativas	Cobertura potencial de la población objetivo
Consulta de información	Navegación manual por sistemas y documentos PDF	Consulta mediante lenguaje natural	Simplificación del mecanismo de consulta
Recuperación de información	Búsqueda manual o por palabras clave	Búsqueda semántica por similitud del coseno sobre la base documental	Se cuantifica con <i>Hit Rate</i> y <i>MRR</i> en el Capítulo III
Tiempo tecnológico de respuesta	Dependiente de la atención presencial o por correo	Respuesta generada en segundos	Se mide la latencia por etapa en el Capítulo III
Disponibilidad potencial	Dependiente del horario de los canales administrativos (08h00 a 17h00)	El software puede operar fuera del horario administrativo	Capacidad potencial de ampliar el horario de consulta

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

1.5 Objetivos

1.5.1 Objetivo General

Desarrollar un prototipo de Asistente Virtual Inteligente, provisto de una interfaz de avatar 3D y visión artificial, para optimizar la difusión proactiva y el acceso a la información académica de los estudiantes en la carrera de Ingeniería en Tecnologías de la Información de la Universidad Tecnológica Indoamérica.

1.5.2 Objetivos Específicos

- Analizar los procesos actuales de gestión y consulta de documentación académica en la institución mediante revisión documental y observación directa, para determinar los requerimientos funcionales, no funcionales y la base de conocimiento conversacional del sistema.
- Diseñar la arquitectura de software del asistente mediante un enfoque modular cliente-servidor, definiendo la integración del módulo conversacional, el motor RAG, la base de conocimiento académica, los servicios de voz y el entorno gráfico del avatar 3D.
- Implementar el prototipo del asistente virtual integrando un motor RAG ejecutado localmente, servicios de inteligencia artificial consumidos mediante API (LLM, transcripción y síntesis de voz), la interfaz antropomórfica web y la detección de presencia no biométrica, para obtener una prueba de concepto interactiva funcional.
- Evaluar el desempeño y la usabilidad del prototipo mediante un banco de preguntas con respuestas de referencia (*Hit Rate* y *MRR*), mediciones de latencia por etapa y de consumo del módulo de visión, y una encuesta piloto con estudiantes, para determinar la viabilidad técnica de la solución y sus limitaciones.

1.6 Alcance del Proyecto

El proyecto comprende el diseño, desarrollo y validación de un prototipo funcional de asistente virtual, orientado exclusivamente a los estudiantes de la carrera de Tecnologías de la Información.

Componentes y funcionalidades incluidas:

- **Módulo de Visión Artificial:** Detección de presencia en el propio navegador del estudiante mediante el detector facial ligero BlazeFace de MediaPipe, que entrega únicamente las cajas delimitadoras (*bounding boxes*) del rostro; al servidor solo se envían esas coordenadas para que el avatar salude, detecte la salida del usuario y reanude el diálogo, sin identificar personalmente al usuario ni transmitir o almacenar imágenes.
- **Módulo Conversacional (PLN):** Implementación de un motor de Inteligencia Artificial capaz de comprender y procesar solicitudes orales en lenguaje natural (Speech-to-Text), procesar la respuesta mediante un Modelo de Lenguaje Grande (LLM) contextualizado con la información de la carrera (mallas, reglamentos), y emitir una respuesta en audio (Text-to-Speech).
- **Interfaz de Avatar 3D:** Desarrollo de una interfaz web con dos modos (chat y conversacional) que integra un avatar VRM con parpadeo, animación en reposo y sincronización labial (*lip-sync*) basada en la amplitud del audio sintetizado.
- **Base de conocimiento y administración:** Indexación del documento institucional de la carrera en una base vectorial, con un panel de administración protegido por roles para reemplazar los documentos PDF.

Límites y exclusiones:

- El sistema **no** realizará reconocimiento facial biométrico para la identificación personal del estudiante, preservando la privacidad y operando de forma anónima (Privacidad por Diseño).
- El asistente **no** tendrá integración transaccional directa con la base de datos del Sistema de Gestión Académica (SGA) para

realizar acciones como matriculación, pago de aranceles o modificación de notas. Su función será estrictamente informativa y de orientación.

- **No** se desarrollará una aplicación móvil nativa ni un despliegue institucional masivo. El prototipo se ejecuta principalmente en modo local (equipo del desarrollador) y cuenta con un despliegue piloto en un servidor virtual institucional para pruebas de acceso remoto.
- El reconocimiento de voz se activa mediante el botón de micrófono (*push-to-talk*); no se incluye detección automática de palabra clave ni de fin de habla.
- Los modelos de lenguaje, transcripción y síntesis de voz **no** se entrenan ni se ajustan (*fine-tuning*); se consumen como servicios mediante API: el LLM, a través de la API local de Ollama en el mismo equipo, y la transcripción y la síntesis de voz, como servicios externos.

1.7 Fundamentación Teórica

Para respaldar el desarrollo de la arquitectura del Asistente Virtual Inteligente, el proyecto se asienta sobre los siguientes ejes teóricos y tecnológicos:

1.7.1 Inteligencia Artificial y Procesamiento de Lenguaje Natural (PLN)

El PLN es la rama de la IA que permite a las computadoras entender, interpretar y manipular el lenguaje humano. Para que el asistente pueda brindar información académica precisa, se requiere el uso de Modelos de Lenguaje Grandes (LLMs). En este proyecto se fundamentará el uso de técnicas como la Generación Aumentada por Recuperación (RAG, por sus siglas en inglés), que permite al modelo consultar una base de datos de conocimiento específico (los reglamentos y mallas de la UTI) antes de generar una respuesta, evitando así las alucinaciones del modelo.

En el paradigma RAG, los documentos son fragmentados (*chunking*) y transformados a vectores multidimensionales (*embeddings*).

Durante la búsqueda, la precisión semántica de la consulta del usuario se evalúa matemáticamente mediante la **Similitud del Coseno** (Figura 1), que mide el ángulo entre el vector de la consulta ($\vec{q}$) y el vector del documento ($\vec{d}_i$) en el espacio latente (Ecuación 1). Una mayor similitud semántica se traduce en un menor ángulo θ . En el prototipo, la base vectorial ChromaDB se configura con la métrica coseno y devuelve la distancia $d = 1 - \cos(\theta)$ [23, 24].

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{d}_i}{\|\vec{q}\| \|\vec{d}_i\|} \quad (1)$$

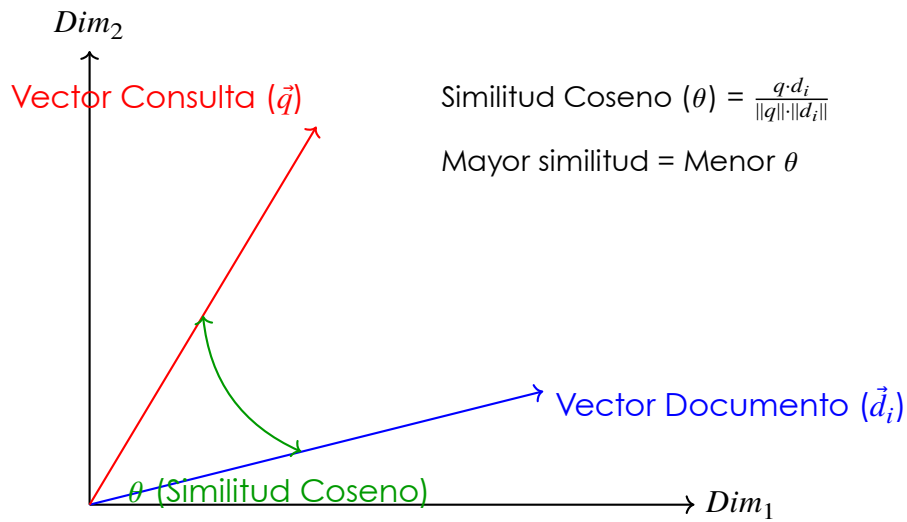


Figura 1: Geometría de la Similitud del Coseno en el espacio vectorial RAG.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

La búsqueda puramente semántica puede fallar cuando la respuesta depende de términos exactos, como cifras, códigos o nombres de asignaturas que se repiten en fragmentos de vocabulario similar. Por ello, los sistemas RAG suelen combinarla con una búsqueda léxica como BM25, que puntúa cada fragmento según la frecuencia de los términos de la consulta y su rareza en el corpus [25]. Las dos listas de resultados se fusionan con la técnica *Reciprocal Rank Fusion* (RRF), que suma para cada fragmento el inverso de su posición en cada lista (Ecuación 2) y no requiere calibrar las puntuaciones de ambos métodos [26]:

$$RRF(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad (2)$$

donde M es el conjunto de métodos de búsqueda, $r_m(d)$ la posición del fragmento d en la lista del método m y k una constante de suavizado (habitualmente 60). Finalmente, un modelo de reordenamiento (*re-ranker*) de tipo *cross-encoder*, que procesa la consulta y cada fragmento candidato de forma conjunta, estima su relevancia con mayor precisión que la comparación de vectores independientes y reordena los candidatos antes de seleccionar los que se entregan al LLM [27].

1.7.2 Visión Artificial e Interacción Visual no Biométrica

La visión artificial permite a las máquinas extraer información significativa de imágenes o videos. En este proyecto, su uso se delimita al apoyo de la interacción visual del avatar, sin realizar identificación personal ni almacenamiento de datos biométricos. La detección de rostros consiste en localizar las regiones de una imagen que contienen una cara y describirlas mediante cajas delimitadoras (*bounding boxes*), es decir, las coordenadas, el ancho y el alto del rectángulo que las encierra. Los detectores clásicos, como el de Viola y Jones basado en características tipo Haar evaluadas en cascada, ofrecen un bajo costo computacional pero son sensibles a la iluminación y a la pose [28]. Los detectores neuronales ligeros, como BlazeFace, fueron diseñados para ejecutarse en tiempo real en dispositivos de consumo y alcanzan tiempos de inferencia inferiores al milisegundo en GPU móviles [29]. El *framework* MediaPipe permite ejecutar este modelo directamente en el navegador mediante WebAssembly y WebGL [30], de modo que la imagen de la cámara no necesita salir del dispositivo del usuario. Como el detector devuelve únicamente las coordenadas del rectángulo del rostro, no genera plantillas biométricas, lo que mantiene el principio de privacidad por diseño.

1.7.3 Agentes Conversacionales Encarnados (ECAs) y Avatares 3D

Los ECAs (Embodied Conversational Agents) son entidades generadas por computadora que poseen una representación visual y ha-

bilidades conversacionales. Se investigarán los principios de la Interacción Humano-Computadora (HCI) relacionados con el efecto *Uncanny Valley* y la carga cognitiva. A nivel de software, se emplean estándares abiertos como WebGL, Three.js y el formato VRM para avatares humanoides, que proporcionan mallas 3D (*meshes*) estandarizadas y expresiones (*blendshapes*) como parpadeo y vocales. La sincronización labial puede basarse en el análisis de fonemas o, de forma más ligera, en la energía (amplitud) de la señal de audio, enfoque adoptado en este prototipo [31].

1.7.4 Arquitectura de Software para Asistentes Virtuales Multimodales

La arquitectura de los asistentes virtuales modernos sigue un patrón modular y distribuido que separa la interacción con el usuario (cliente) del procesamiento cognitivo intensivo (servidor). En la capa de cliente o *frontend*, se gestiona la captura en tiempo real de los flujos de audio y la detección visual ligera de presencia a través de tecnologías web estándar. Esta capa es también la responsable de renderizar gráficamente el avatar 3D para proveer retroalimentación visual al usuario.

Por su parte, la capa de servidor o *backend* agrupa los servicios cognitivos: motores de transcripción (*Speech-to-Text*), la orquestación del procesamiento de lenguaje natural integrado con Generación Aumentada por Recuperación (RAG), la consulta a la base de datos vectorial y la síntesis de voz (*Text-to-Speech*) [10]. Esta división permite delegar las cargas computacionales pesadas, como la inferencia del LLM, a servidores especializados, mientras que los procesos ligeros (búsqueda vectorial y detección de presencia) pueden ejecutarse en el equipo local para reducir la latencia y conservar los datos sensibles dentro del entorno controlado. De esta forma, se mantiene una experiencia de usuario conversacional fluida e ininterrumpida en el punto de interacción, consolidando la funcionalidad de soporte institucional continuo y automatizado.

1.7.5 Aceptación Tecnológica en la Educación Superior

La adopción de un asistente virtual depende de que los estudiantes lo perciban como útil y fácil de usar. El Modelo de Aceptación Tecnológica (TAM) explica la intención de uso a partir de la utilidad

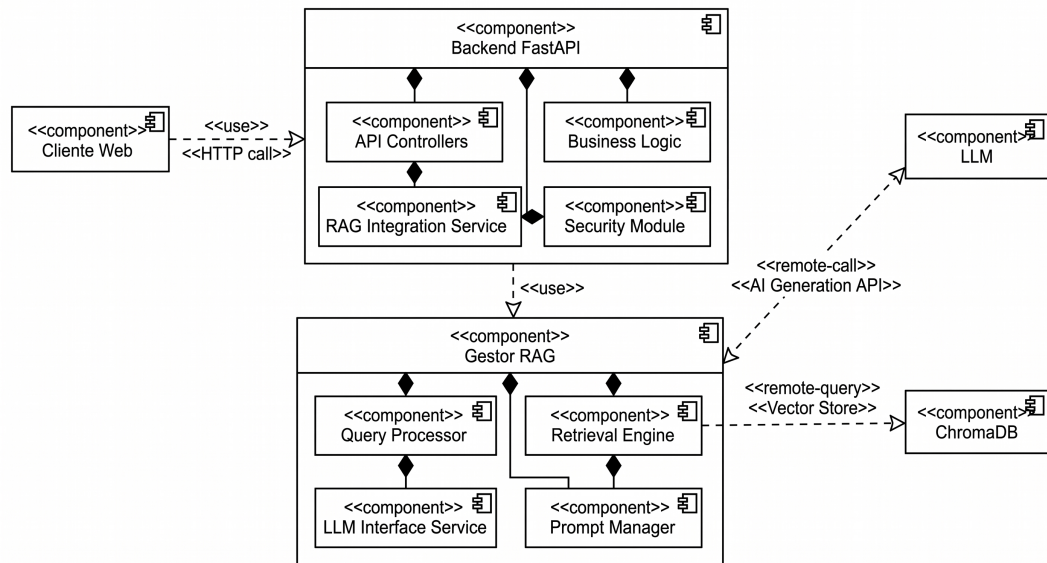


Figura 2: Arquitectura general de software para el Asistente Virtual Multimodal.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

percibida y la facilidad de uso percibida [32]. Estos constructos orientan la encuesta de usabilidad aplicada en la validación del prototipo, que incluye ítems sobre la eficiencia para encontrar información, la complejidad de uso y la preferencia futura del asistente frente a la búsqueda web tradicional. En el caso de los agentes con representación visual, la percepción de antropomorfismo, animacidad e inteligencia del avatar complementa estos constructos [33].

CAPÍTULO II

METODOLOGÍA

2.1 Diagnóstico de la Situación Actual

La gestión actual de la información académica en la carrera de Tecnologías de la Información de la Universidad Tecnológica Indoamérica (UTI) se apoya en una infraestructura transaccional robusta conocida como Sistema de Gestión Académica (SGA). Si bien este sistema cumple eficientemente con los procesos de matriculación y registro de calificaciones, presenta limitaciones significativas en su capa de difusión de información de consulta rápida.

En la actualidad, si un estudiante requiere resolver inquietudes de carácter operativo (tales como la secuenciación de asignaturas en su malla curricular, el valor de los aranceles, el procedimiento para registrar un comprobante de pago, la acreditación del inglés B1 o los requisitos de homologación), el flujo de interacción resulta estático y asíncrono. El estudiante debe autenticarse en el portal, navegar a través de menús jerárquicos y descargar archivos en formato PDF, o bien acudir a los canales de atención humana (secretaría, correo electrónico, teléfono o WhatsApp institucional), que operan de lunes a viernes de 08h00 a 17h00.

La relevancia de mejorar este proceso de difusión se sustenta en la rigurosidad de las normativas internas. Por ejemplo, el Reglamento de Régimen Académico Interno de la Universidad Tecnológica Indoamérica establece directrices críticas que el estudiante debe interpretar correctamente para no retrasar su progreso académico:

“Las prácticas preprofesionales se realizarán en entornos organizacionales, institucionales, empresariales, comunitarios u otros relacionados al ámbito profesional de la carrera o programa, públicos o privados, nacionales o internacionales, que garanticen el desarrollo de competencias para el ejercicio profesional” (UTI, 2024, Art. 45).

Esta dispersión y complejidad documental exige una interfaz más inteligente. Las principales limitaciones técnicas del enfoque actual son:

- **Carencia de búsqueda semántica:** Los repositorios actuales dependen de búsquedas por palabras clave exactas o de la navegación manual. El sistema no comprende la intención del usuario ni puede extraer el párrafo específico de un documento extenso para responder una duda puntual.
- **Ausencia de proactividad en la interfaz:** La interacción depende de que el usuario inicie la búsqueda; no existe un agente que salude, oriente o sugiera preguntas frecuentes.
- **Concentración de consultas en canales humanos:** Al no existir un primer nivel de soporte automatizado, las dudas repetitivas sobre malla, aranceles, becas y homologación se canalizan hacia secretaría y coordinación.

2.1.1 Estimación de la Carga Operativa

La institución no dispone de bitácoras públicas que registren el número de consultas ni su tiempo de atención por carrera; por ello, este trabajo no reporta mediciones directas del proceso actual. Para dimensionar el problema se construyó un modelo de estimación con supuestos explícitos, cuyos valores deben validarse con registros institucionales en una implementación definitiva:

- **Población:** 380 estudiantes activos en la carrera de Tecnologías de la Información (sedes Quito y Ambato), según el informe de rendición de cuentas institucional [5].
- **Frecuencia (supuesto):** cuatro consultas administrativas o reglamentarias por estudiante y semestre, lo que equivale a 1.520 consultas por semestre y, para un semestre de cinco meses, a unas 304 consultas mensuales.
- **Tiempo de atención (supuesto):** 15 minutos por consulta para escuchar el requerimiento, revisar el documento o el SGA y responder, valor coherente con los referentes nacionales de automatización académica [16].

Con estos supuestos, la carga operativa estimada se calcula mediante la Ecuación 3:

$$H_{mes} = \frac{N_{est} \times C_{sem} \times t_c}{60 \times M_{sem}} = \frac{380 \times 4 \times 15}{60 \times 5} = 76 \text{ horas-hombre/mes} \quad (3)$$

donde N_{est} es el número de estudiantes, C_{sem} las consultas por estudiante y semestre, t_c el tiempo de atención en minutos y M_{sem} la duración del semestre en meses. La Figura 3 resume los valores del modelo.

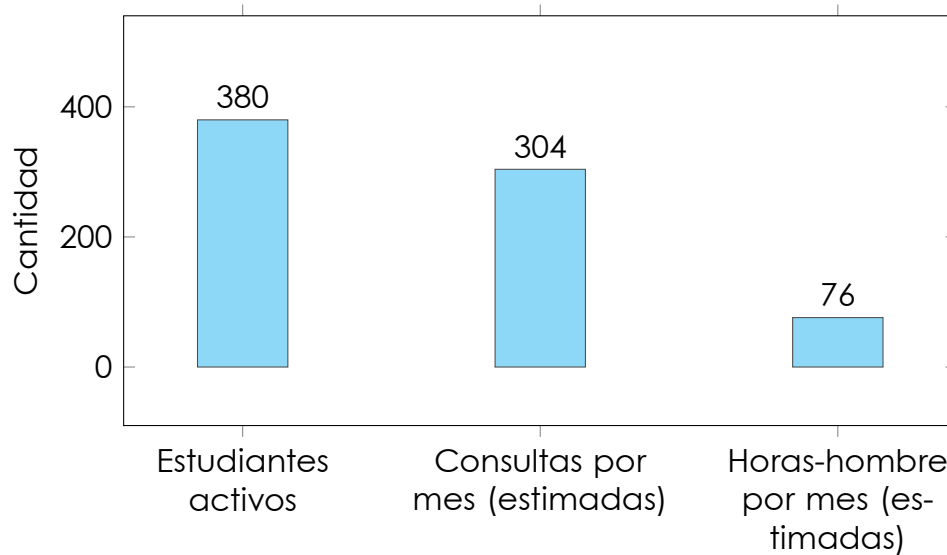


Figura 3: Modelo de estimación del volumen mensual de consultas y de la carga operativa asociada.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Población según UTI [5]; frecuencia y tiempo de atención supuestos por el autor

La Tabla 5 resume los indicadores del diagnóstico y el instrumento con el que cada uno se contrasta en la validación del prototipo (Capítulo III).

Tabla 5: Indicadores del diagnóstico y forma de verificación en el prototipo.

Indicador	Situación actual	Verificación en el prototipo
Mecanismo de búsqueda	Navegación manual y búsqueda por palabras clave en PDF	<i>Hit Rate</i> y MRR sobre un banco de 50 preguntas con respuesta de referencia
Tiempo de respuesta	Atención presencial o por correo, sujeta a horario (08h00 a 17h00)	Latencia medida por etapa (STT, RAG, LLM y TTS)
Carga operativa	76 h-hombre/mes estimadas (Ecuación 3)	No medible en el piloto; requiere registro institucional de consultas
Percepción del usuario	Sin mediciones previas	Encuesta piloto de usabilidad con 16 estudiantes

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

2.2 Área de Estudio

Tabla 6: Área de estudio.

Dominio:	Inteligencia Artificial y Tecnologías Educativas
Línea de investigación:	Diseño, desarrollo e integración de sistemas basados en arquitecturas web, móviles y sistemas embebidos
Sublínea de investigación:	Aplicación de tecnologías emergentes.
Campo:	Tecnologías de la información y la comunicación (TIC)
Área:	Desarrollo de Software e Interfaces Humano-Computadora
Aspectos:	Recuperación semántica de información (RAG), procesamiento de lenguaje natural, reconocimiento y síntesis de voz, detección de presencia por visión artificial y renderizado de avatares 3D en la web
Objeto de estudio:	Difusión de información académica de la carrera de Ingeniería en Tecnologías de la Información de la UTI mediante un asistente virtual multimodal
Periodo de estudio:	Enero 2026 - Octubre 2026

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Formato institucional UTI

2.3 Metodología de Desarrollo o Implementación

Para el desarrollo del proyecto se adoptó como metodología de investigación la Metodología de Investigación en Ciencia del Diseño (DSRM), debido a que el trabajo se orienta al diseño, construcción

y evaluación de un artefacto tecnológico destinado a resolver una problemática específica [34, 35].

Como metodología de construcción del prototipo se utilizó un enfoque ágil, iterativo e incremental inspirado en Scrum, organizado en iteraciones cortas al final de las cuales se obtenía una versión funcional del asistente. Este enfoque resultó pertinente debido a la necesidad de realizar pruebas sucesivas sobre modelos de lenguaje, parámetros de recuperación semántica, servicios de voz y tiempos de respuesta. El historial de versiones del repositorio (Anexo A) documenta cada incremento.

La aplicación de las etapas de la metodología DSRM en el contexto de este trabajo se sintetiza en la Tabla 7 y su relación con las iteraciones de construcción se representa en la Figura 4.

Tabla 7: Aplicación de la metodología DSRM en el proyecto.

Etapas DSRM	Aplicación en el proyecto	Evidencia
Identificación del problema	Análisis del acceso actual a información académica	Sección 2.1
Definición de objetivos	Requerimientos funcionales y no funcionales	Sección 2.5
Diseño y desarrollo	Arquitectura RAG, visión, voz y avatar	Sección 2.6 y Capítulo III
Demostración	Ejecución del prototipo en modo local y despliegue piloto en servidor	Capturas, repositorio (Anexo A)
Evaluación	Banco de preguntas, latencia, visión y encuesta a estudiantes	Sección 3.3
Comunicación	Documentación del trabajo, manuales y repositorio	Tesis y anexos

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Adaptado de Peffers et al. [35]

El desarrollo iterativo se estructuró en las siguientes fases, cuyo detalle temporal se presenta en el cronograma del Capítulo III:

- **Fase 1 - Avatar 3D y motor RAG inicial:** Se construyó la interfaz web con Three.js y el avatar en formato VRM, y se implementó la

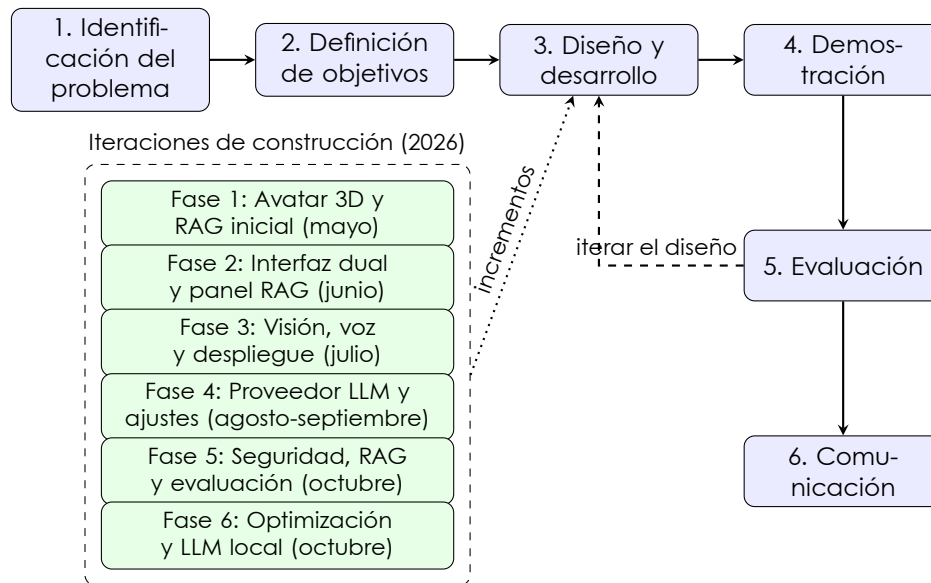


Figura 4: Modelo operativo: etapas DSRM e iteraciones de construcción del prototipo.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia a partir de Peffers et al. [35] y del historial del repositorio

primera versión del motor de Generación Aumentada por Recuperación: extracción del documento institucional en formato Markdown, fragmentación (*chunking*) e indexación en ChromaDB.

- **Fase 2 - Interfaz dual y administración:** Se incorporaron el modo conversacional (avatar en primer plano y control de voz) junto al modo chat, y un panel de administración para reemplazar los documentos PDF de la base de conocimiento.
- **Fase 3 - Visión, voz y despliegue:** La detección de presencia, que inicialmente leía la cámara del equipo con OpenCV, se migró a un esquema en el que el navegador envía fotogramas por WebSocket, condición necesaria para operar en un servidor sin cámara. Se integró la transcripción con Whisper (Groq) y se sustituyó la síntesis de voz *edge-tts* por ElevenLabs. Se añadieron autenticación de usuarios, el modelo de *embeddings* multilingüe y el despliegue con Docker en un servidor virtual.
- **Fase 4 - Proveedor del modelo de lenguaje:** Se integró la API de NVIDIA como proveedor compatible con OpenAI, junto con

soporte de Markdown, subtítulos y control de cámara en la interfaz.

- **Fase 5 - Seguridad, ajuste del RAG y evaluación:** Se añadieron control de acceso por roles, el filtro anti-*jailbreak*, la normalización y expansión de consultas, la limitación del contexto a los tres fragmentos más relevantes, la métrica coseno en la base vectorial y los *scripts* de evaluación reproducible.
- **Fase 6 - Optimización según la evaluación:** A partir de las limitaciones medidas en la Fase 5 (recuperación y latencia), se incorporó la recuperación híbrida con reordenamiento, se migró el LLM al modelo local Nemotron servido con Ollama en la GPU del equipo, se implementó la entrega de la respuesta en *streaming* por WebSocket, se adoptó un modelo de síntesis de voz de baja latencia y la detección de presencia se trasladó al navegador con MediaPipe, enviando al servidor solo las cajas delimitadoras.

2.4 Técnicas e Instrumentos de Recolección de Información

Para garantizar la validez de los resultados, la recolección de información se dividió en dos etapas: diagnóstico/desarrollo y evaluación.

2.4.1 Técnicas para la Fase de Diagnóstico y Desarrollo

- **Revisión Documental:** Se revisaron los documentos públicos de la carrera (malla curricular, aranceles del periodo A26, reglamentos de homologación, política de becas, acreditación de inglés y canales de contacto) y se consolidaron en un documento base de conocimiento, *Investigación Carrera TI Indoamérica Quito*, que constituye el corpus indexado por el motor RAG.
- **Registro de Decisiones de Arquitectura (ADR):** Cada cambio técnico relevante (migración de la visión a WebSockets y luego al navegador, modelo de *embeddings* multilingüe, migración del TTS, estandarización del RAG, recuperación híbrida y LLM local)

se documentó con su contexto, decisión y consecuencias.

2.4.2 Técnicas para la Fase de Evaluación y Validación

- **Bancos de preguntas con respuesta de referencia (*gold standard*):** un banco de desarrollo de 50 preguntas redactadas en lenguaje estudiantil sobre once categorías del documento, usado para comparar configuraciones, y un banco de prueba reservado de otras 50 preguntas, redactado antes de las optimizaciones y utilizado solo para la medición final. Cada pregunta se asocia a frases literales del documento oficial que deben aparecer en el fragmento recuperado. Se añadieron 10 preguntas fuera de dominio para verificar la restricción temática.
- **Medición instrumentada:** *Scripts* reproducibles incluidos en el repositorio (carpeta `evaluacion/`) que ejecutan el mismo código de producción y registran el tiempo hasta el primer fragmento de respuesta (TTFB) por WebSocket, la latencia de cada etapa, la tasa de error de palabras (WER) de la transcripción y el costo de la detección de presencia.
- **Cuestionario de Usabilidad y Experiencia de Usuario:** Cuestionario en Google Forms con escala Likert y diferencial semántico (Anexo B), basado en los constructos de utilidad y facilidad de uso percibidas [32] y en las dimensiones de antropomorfismo, animacidad e inteligencia percibida [33]. Fue precedido por un consentimiento informado (Anexo C).

Para la evaluación de usabilidad se trabajó con una muestra piloto de 16 estudiantes de la carrera de Tecnologías de la Información de tercer, cuarto, sexto y octavo semestre, seleccionados mediante muestreo no probabilístico por conveniencia en dos sesiones: 13 el 21 de julio de 2026 y 3 entre el 3 y el 4 de octubre de 2026. La evaluación tuvo un carácter exploratorio, orientado a identificar la percepción inicial de los usuarios y oportunidades de mejora, por lo que sus resultados no pretenden generalizarse estadísticamente a la totalidad de estudiantes de la carrera.

2.5 Requerimientos del Proyecto

La definición de los requerimientos establece las directrices técnicas y operativas que el sistema debe cumplir. A continuación se detallan los requerimientos funcionales (RF) y no funcionales (RNF) en formato de tabla para asegurar su trazabilidad con el código fuente y con las pruebas del Capítulo III.

2.5.1 Requerimientos Funcionales (RF)

Tabla 8: Requerimientos Funcionales del Sistema.

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RF-01	Detección no biométrica de presencia	Video de la cámara web (320×240) en el navegador.	Cada segundo, MediaPipe Face Detector (modelo BlazeFace, confianza mínima 0,5) obtiene en el navegador las cajas delimitadoras del rostro, que se envían como JSON por WebSocket (/ws). El backend declara presencia con rostro en 2 fotogramas consecutivos y ausencia con 4 fotogramas sin rostro.	Mensajes person_arrived o person_left hacia el frontend, que saluda o reanuda el diálogo.	Detección en imagen de prueba con rostro, ausencia de falsos positivos en imagen sin rostro, tiempo de inferencia por fotograma y tamaño del mensaje.
Continúa en la siguiente página					

Tabla 8 – continuación de la página anterior

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RF-02	Transcripción de voz a texto (STT)	Audio WebM grabado al pulsar el botón de micrófono (<i>push-to-talk</i>).	El endpoint <code>/api/stt</code> envía el audio a Whisper <code>large-v3</code> en la API de Groq con idioma español [36].	Texto UTF-8 con la consulta del estudiante.	Tasa de error de palabras (WER) y latencia sobre las preguntas del banco.
RF-03	Búsqueda semántica RAG	Consulta textual transcrita o escrita.	Normalización de términos (p. ej. “semestre” → “nivel”) y expansión de consulta; búsqueda semántica con <code>paraphrase-multilingual-MiniLM-L12-v2</code> (distancia coseno en ChromaDB) y léxica con BM25, fusión RRF y reordenamiento con un <i>cross-encoder</i> multilingüe.	Los 3 fragmentos más relevantes (Top-3).	<i>Hit Rate@3</i> y <i>MRR@5</i> sobre los bancos de desarrollo y de prueba (50 preguntas cada uno).
Continúa en la siguiente página					

Tabla 8 – continuación de la página anterior

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RF-04	Generación de respuesta con LLM	Consulta y Top-3 de contexto dentro de un <i>prompt</i> con reglas del sistema.	Filtro anti- <i>jailbreak</i> por expresiones regulares; inferencia local con Nemotron 3 Nano 4B servido por Ollama en la GPU del equipo (proveedor configurable), temperatura 0,1 y máximo 500 <i>tokens</i> ; la respuesta se envía por fragmentos (<i>streaming</i>) mediante el WebSocket <code>/ws/chat</code> .	Respuesta breve en texto plano basada en el documento oficial.	Exactitud de las respuestas, rechazo de las 10 preguntas fuera de dominio y TTFB por WebSocket.
RF-05	Síntesis de texto a voz (TTS)	Respuesta de texto generada por el LLM.	El endpoint <code>/api/tts</code> reenvía el texto a ElevenLabs (<code>eleven_flash_v2_5</code> , voz "Ana Sofía") y transmite el audio MP3 por fragmentos (<i>streaming</i>).	Audio reproducido en el navegador con sincronización labial por amplitud.	Tiempo hasta el primer byte de audio (TTFB).
Continúa en la siguiente página					

Tabla 8 – continuación de la página anterior

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RF-06	Gestión del diálogo por presencia	Eventos de presencia y estado de habla del avatar.	Saludo de bienvenida en la primera detección; si el usuario se retira o regresa, se envía al LLM un evento oculto para responder de forma contextual (pausa, retorno).	Mensaje hablado y estado del indicador de cámara.	Prueba automatizada de los escenarios llegada, salida, retorno y ruido.
RF-07	Autenticación y roles	Credenciales de usuario.	Inicio de sesión con contraseña cifrada y token de sesión en cookie; rol admin para el panel RAG; cierre de sesión con revocación del token.	Acceso a la interfaz según rol.	Acceso denegado a rutas protegidas sin sesión o sin rol.
Continúa en la siguiente página					

Tabla 8 – continuación de la página anterior

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RF-08	Administración de la base de conocimiento	Uno o varios archivos PDF cargados por el administrador.	Extracción a Markdown, fragmentación (800/160 caracteres), cálculo de <i>embeddings</i> y reconstrucción de la colección en segundo plano, con registro de progreso.	Base vectorial actualizada sin reiniciar el servidor.	Tiempo de reindexación del documento base.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Código fuente del prototipo (`api.py`, `rag_manager.py`, `static/script.js`)

2.5.2 Requerimientos No Funcionales (RNF)

Tabla 9: Requerimientos No Funcionales del Sistema.

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RNF-01	Privacidad por diseño	Fotogramas de la cámara web.	Los fotogramas se procesan solo en la memoria del navegador; al servidor llegan únicamente las coordenadas de las cajas delimitadoras y ninguna imagen se transmite ni se almacena. La cámara puede desactivarse desde la interfaz.	Ningún dato biométrico transmitido ni persistido.	Revisión de código y del contenido del mensaje WebSocket.
Continúa en la siguiente página					

Tabla 9 – continuación de la página anterior

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RNF-02	Rendimiento conversacional	Consulta escrita o por voz.	Pipeline STT + RAG + LLM + TTS con E/S asíncrona (FastAPI y <i>asyncio</i>), respuesta del LLM en <i>streaming</i> por WebSocket y audio en <i>streaming</i> .	Primer fragmento de la respuesta en menos de medio segundo e inicio de la respuesta hablada en pocos segundos.	TTFB promedio ≤ 450 ms por WebSocket y latencia percibida P95 < 4,0 s, en modo local con 1 usuario.
RNF-03	Restricción de dominio académico	Consultas ajenas a la carrera o intentos de cambiar el rol del asistente.	Reglas del <i>prompt</i> del sistema y filtro anti- <i>jailbreak</i> a nivel de código.	Mensaje de rechazo cortés y reorientación a la carrera de TI.	100 % de rechazo en el conjunto de 10 preguntas fuera de dominio.
RNF-04	Aislamiento del SGA	Sistemas transaccionales de la UTI.	El asistente no posee credenciales ni conexión con el SGA; solo consulta su propia base vectorial.	Imposibilidad de modificar expedientes.	Revisión de configuración: sin cadenas de conexión al SGA.

Continúa en la siguiente página

Tabla 9 – continuación de la página anterior

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RNF-05	Mantenibilidad	Nuevos documentos PDF.	Panel administrativo protegido por rol que reconstruye la base vectorial sin reentrenar modelos.	Conocimiento actualizado.	Reindexación del documento base en menos de 60 s.
RNF-06	Usabilidad	Uso por estudiantes sin capacitación previa.	Dos modos de interacción (chat por texto y conversación por voz), preguntas sugeridas, subtítulos y diseño adaptable a móviles.	Interacción comprensible.	Encuesta piloto de usabilidad (Anexo B).
RNF-07	Portabilidad del despliegue	Equipo local o servidor.	Ejecución local con Uvicorn o en contenedores Docker con Docker Compose; configuración por variables de entorno.	Mismo código en ambos modos.	Ejecución del prototipo en los dos modos de despliegue.

Continúa en la siguiente página

Tabla 9 – continuación de la página anterior

Código	Nombre del Requerimiento	Entradas / Estímulo	Procesamiento y Control	Salida / Respuesta	Métrica de Validación y Prueba
RNF-08	Seguridad de la API	Peticiones HTTP a los <i>endpoints</i> .	Chat, voz, WebSockets y diagnóstico requieren sesión válida (diagnóstico solo para administradores); claves de API en archivo <code>.env</code> excluido del control de versiones.	Acceso restringido a usuarios autenticados.	Respuesta 401/403 a peticiones sin sesión y cierre 4401 de los WebSocket.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

2.6 Diseño de la Solución

El diseño de la solución se articula sobre una arquitectura cliente-servidor modular y desacoplada. El cliente es una aplicación web que se ejecuta en el navegador del estudiante y realiza la detección de presencia; el servidor es una API en FastAPI [37] que orquesta los eventos de presencia, el motor RAG, el LLM local y los servicios de voz. La Figura 5 muestra el diagrama de contexto con las entidades externas del sistema.

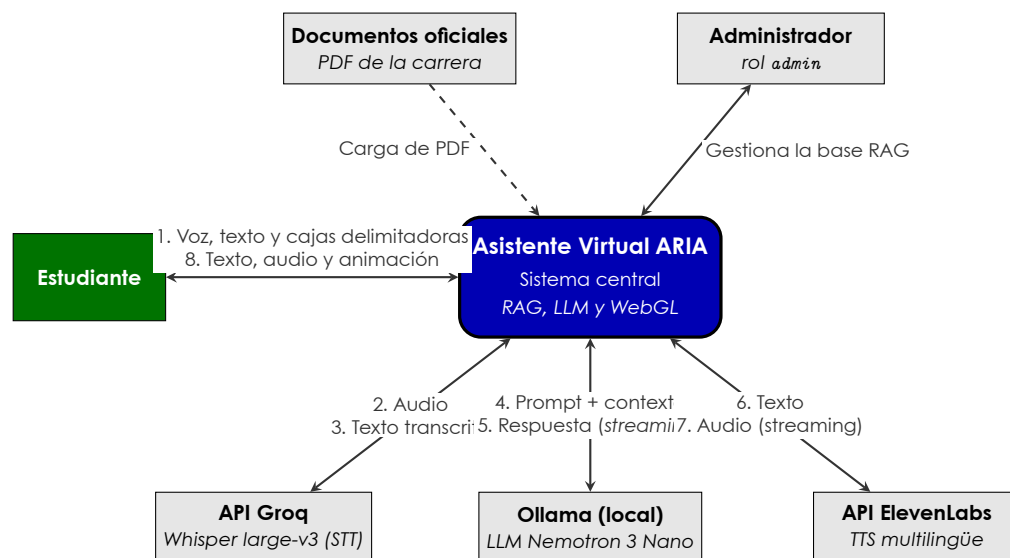


Figura 5: Diagrama de Contexto (Nivel 0).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

El diagrama de clases de la Figura 6 representa la responsabilidad lógica de los componentes. En la implementación, `UserInterface`, `AvatarController`, `VisionHandler` (captura y detección facial con `MediaPipe`) y la grabación de audio residen en `static/script.js`; `InputRouter`, `LLMController` y la gestión de los eventos de presencia (`DetectorPresencia`), en `api.py`; y `RAGManager`, con el recuperador híbrido, en `rag_manager.py`.

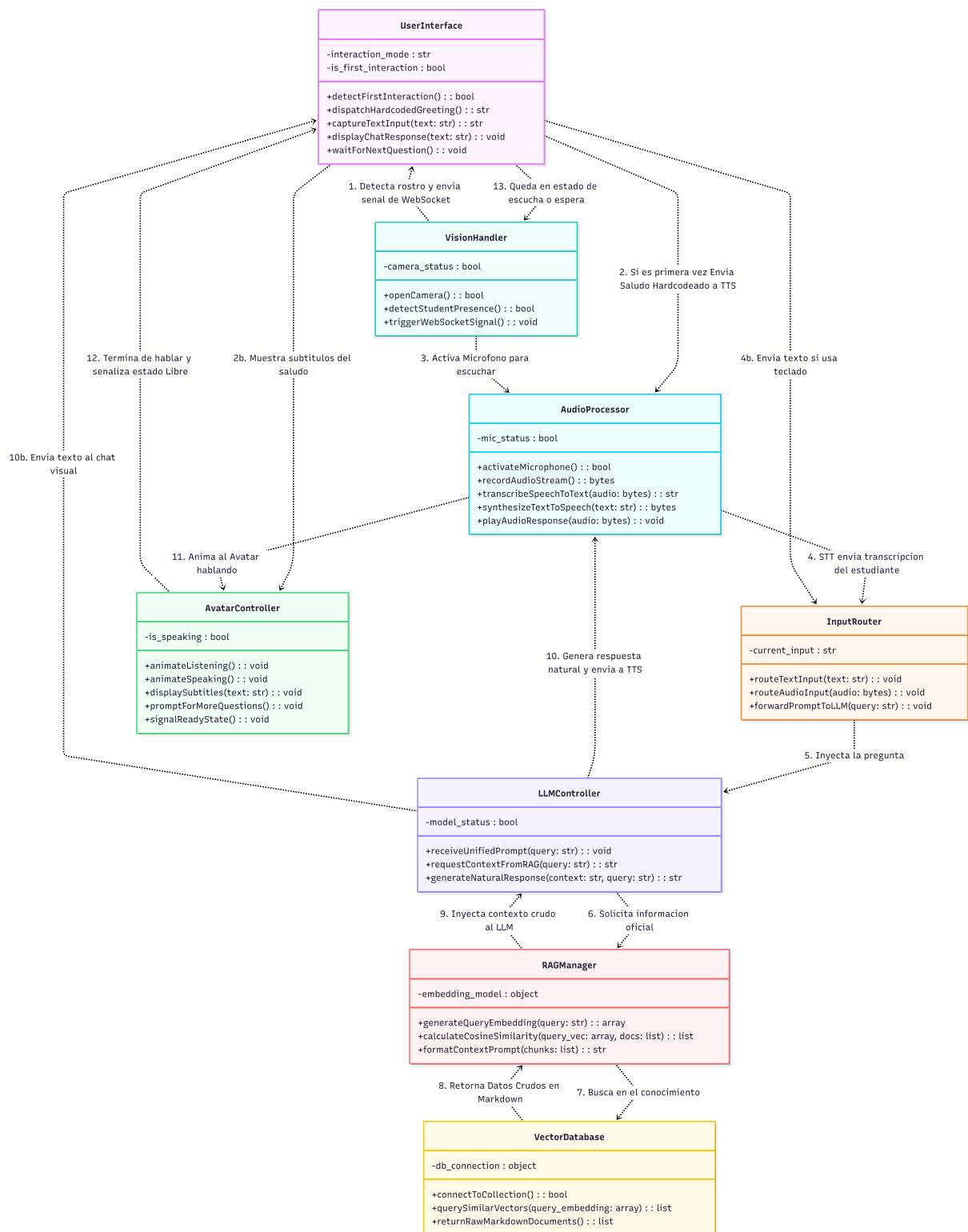


Figura 6: Diagrama de Clases del Sistema (Arquitectura Completa y Flujo Circular).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

El comportamiento del avatar se modela como una máquina de estados. La Figura 7 presenta el diseño conceptual inicial y la Figura 8, los estados tal como quedaron implementados: la escucha se inicia con el botón de micrófono y la presencia se confirma o descarta con un número mínimo de fotogramas consecutivos.

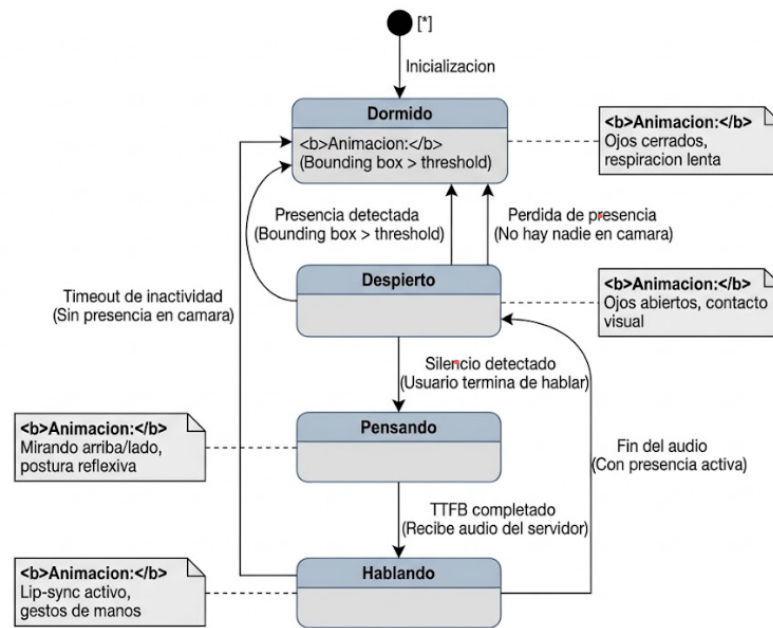


Figura 7: Diagrama de Estados del Avatar (diseño conceptual inicial).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

La Figura 9 muestra la secuencia de interacción de una consulta por voz. El navegador detecta el rostro localmente y envía solo las cajas delimitadoras; la consulta se transcribe con `/api/stt`, la respuesta del LLM llega por fragmentos a través del WebSocket `/ws/chat` y el audio se recibe como flujo MP3 por HTTP desde `/api/tts`.

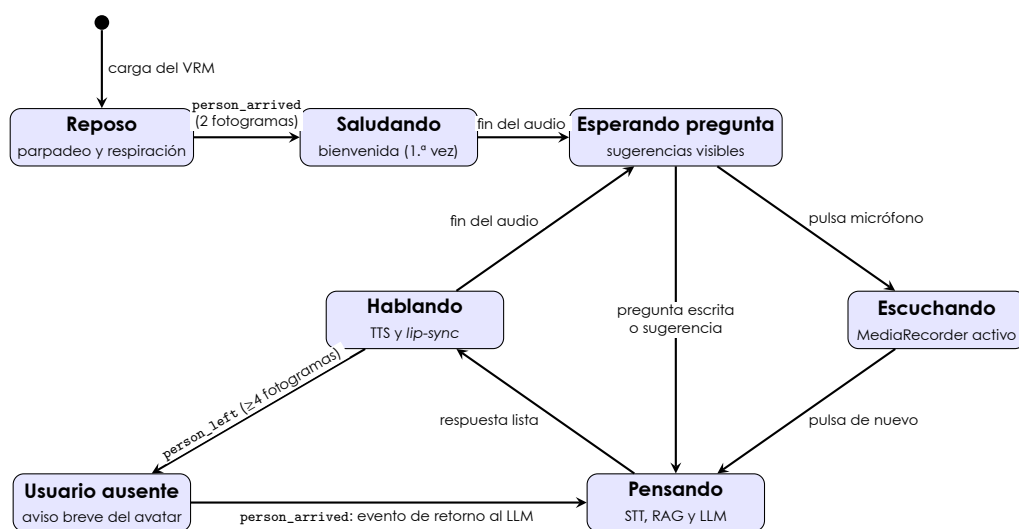


Figura 8: Diagrama de Estados del Avatar implementado en el frontend.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia a partir de `static/script.js`

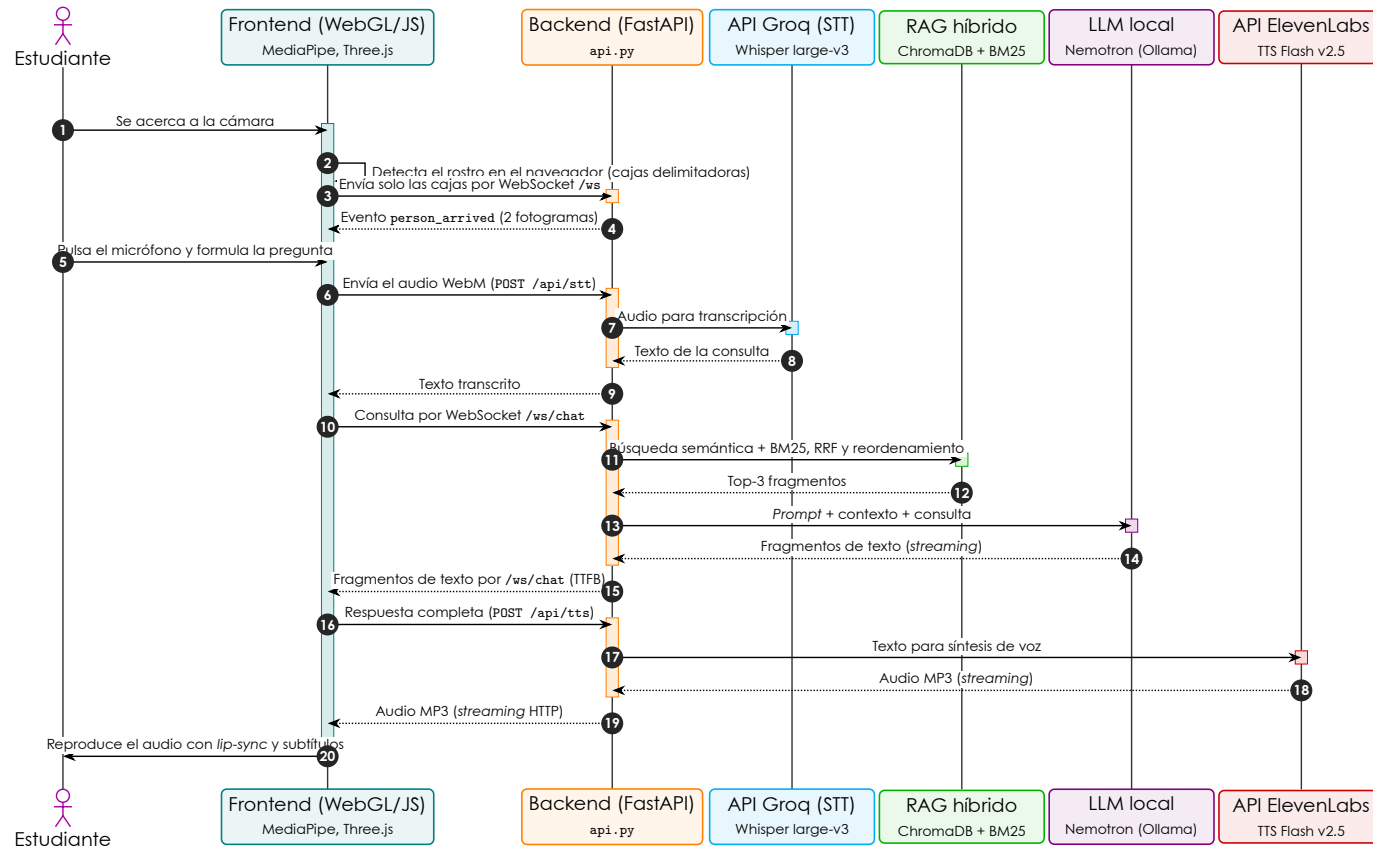


Figura 9: Diagrama de Secuencia de Interacción (consulta por voz).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

En cuanto a su estilo, la solución es una arquitectura **cliente-servidor en capas**, cuyo servidor es un **monolito modular**: un único proceso FastAPI organizado en módulos con responsabilidades separadas (autenticación, chat con RAG, voz, visión y administración), que comparten configuración y se despliegan juntos. El cliente es ligero en lógica de negocio, pero asume la captura de medios y el renderizado 3D. La comunicación combina un WebSocket para el chat, por el que la respuesta del LLM se envía en fragmentos a medida que se genera; peticiones REST para la transcripción y la síntesis de voz; *streaming* HTTP para el audio de respuesta; y un segundo WebSocket orientado a eventos para la presencia, que recibe únicamente cajas delimitadoras. La solución es una arquitectura **híbrida**: la detección visual y el renderizado se ejecutan en el dispositivo cliente; la recuperación y el LLM (Nemotron, servido por Ollama en la GPU del servidor local) se ejecutan en el servidor; y la transcripción y la síntesis de voz se consumen como servicios por API. El proveedor del LLM sigue siendo intercambiable mediante configuración (Ollama local o servicios compatibles con OpenAI). No se trata de una arquitectura de microservicios, ya que los módulos del servidor no se despliegan ni escalan de forma independiente.

El sistema se desglosa en tres capas lógicas fundamentales:

2.6.1 Capa de Percepción y Captura (Frontend)

Esta capa actúa como el sensor del sistema y se ejecuta en el navegador. Con el permiso del usuario, la aplicación captura la cámara web mediante `getUserMedia` a 320×240 píxeles y, cada segundo, ejecuta en el propio navegador el detector facial BlazeFace de MediaPipe (WebAssembly/WebGL) [29, 30]. Al servidor solo se envían por WebSocket las cajas delimitadoras detectadas (un mensaje JSON de unas decenas de bytes), y este responde con los eventos de llegada o salida; ninguna imagen sale del dispositivo. La captura de voz se realiza con `MediaRecorder` entre dos pulsaciones del botón de micrófono, y el usuario puede escribir su pregunta o elegir una de las preguntas sugeridas.

2.6.2 Capa de Procesamiento Cognitivo (Backend)

Es el núcleo lógico de la solución, estructurado mediante un flujo de datos secuencial:

- **Speech-to-Text (STT):** Transcribe el audio del usuario a texto en español con Whisper 1large-v3 a través de la API de Groq [36].
- **Generación Aumentada por Recuperación (RAG):** La consulta se normaliza y se busca en paralelo de forma semántica, con un modelo de *embeddings* multilingüe y similitud del coseno en ChromaDB, y de forma léxica, con BM25; ambas listas se fusionan por RRF y un *cross-encoder* multilingüe reordena los candidatos para elegir los tres fragmentos más relevantes [38, 23, 25, 26].
- **Modelo de Lenguaje (LLM):** Recibe las reglas del sistema, la consulta y los tres fragmentos más relevantes, y genera una respuesta breve que se envía al navegador por fragmentos. Se ejecuta localmente con el modelo Nemotron 3 Nano de 4 mil millones de parámetros, servido por Ollama en la GPU del servidor [39, 40]. El proveedor es configurable mediante la variable `AI_PROVIDER`, con alternativas como la API de NVIDIA, Groq, Gemini, Silicon-Flow o un servidor Open WebUI propio.
- **Text-to-Speech (TTS):** El texto generado se sintetiza con el modelo de baja latencia Flash v2.5 de ElevenLabs y se transmite al navegador por fragmentos [41].

2.6.3 Capa de Renderizado e Interacción (Frontend Gráfico)

El audio sintetizado y el texto se presentan en la interfaz web. El avatar 3D en formato VRM se renderiza con Three.js y *three-vrm* [31]; mientras se reproduce el audio, un analizador de frecuencias (*AnalyserNode*) calcula la energía de la señal y la traduce en la apertura de la boca (expresiones *aa* y *ou*). En paralelo se muestran subtítulos y el historial del chat, en el que el texto de la respuesta aparece a medida que llega por el WebSocket. La Figura 10 resume el flujo por capas.

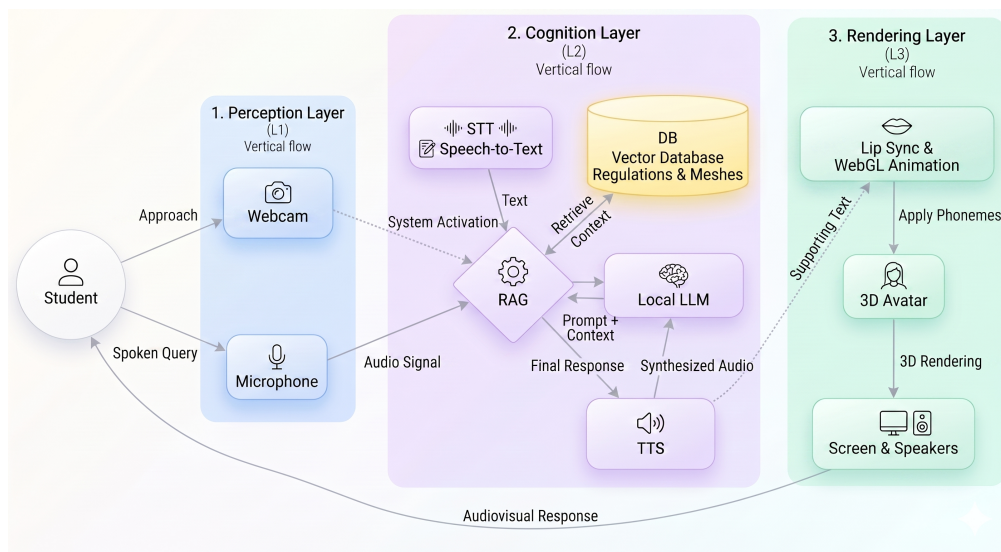


Figura 10: Arquitectura lógica por capas y flujo de datos del Asistente Virtual (diseño conceptual; el bloque LLM puede ser local o remoto).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

2.6.4 Arquitectura de Despliegue

El mismo código fuente admite dos modos de despliegue (Figura 11). El **modo local** es el principal y el utilizado en todas las pruebas de este trabajo: el servidor FastAPI se ejecuta con Uvicorn en el equipo del desarrollador, donde residen la base vectorial, los modelos de recuperación y el LLM Nemotron servido por Ollama en la GPU, mientras que la detección de presencia se ejecuta en el navegador; solo la transcripción y la síntesis de voz se consumen como servicios externos. Al evitar un salto adicional de red entre el navegador y un servidor remoto, este modo reduce la latencia de la interacción. El **modo servidor** es un despliegue piloto en un servidor virtual (VPS) administrado con Coolify y Traefik [42], publicado con HTTPS en aria.indoamericattech.com; incluye un contenedor auxiliar que establece un túnel SSH hacia el servidor Ollama con GPU del laboratorio LabITI, como alternativa para ejecutar el LLM en infraestructura propia.

2.7 Justificación Técnica

La viabilidad técnica y operativa del Asistente Virtual Inteligente depende de una rigurosa selección de componentes y plataformas.

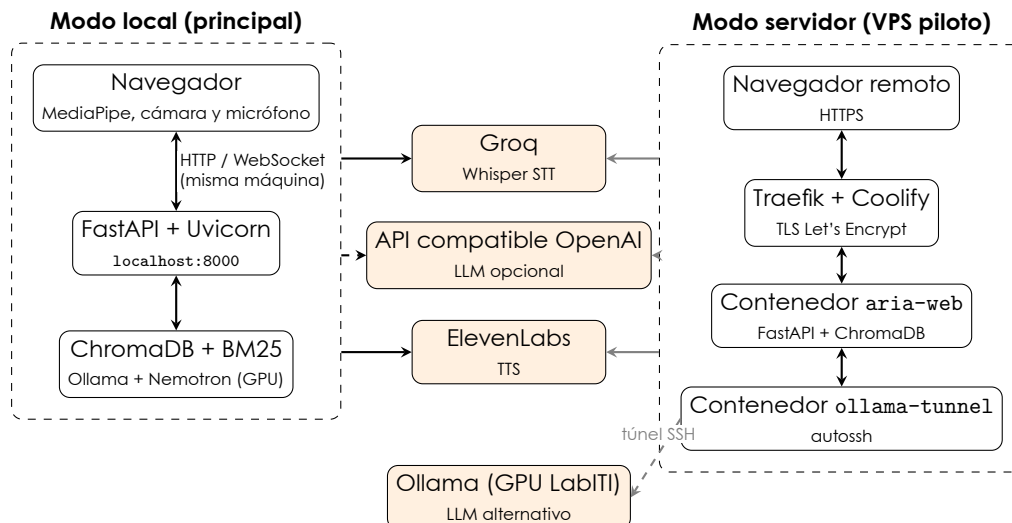


Figura 11: Arquitectura de despliegue en modo local y en modo servidor.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia a partir de `iniciar_local.sh` y `docker-compose.yml`

A continuación, se justifican las decisiones arquitectónicas adoptadas para este proyecto.

2.7.1 Arquitectura Híbrida: Procesamiento Local y Servicios por API

El prototipo combina procesos locales y servicios remotos. La búsqueda híbrida y el cálculo de *embeddings* se ejecutan en el servidor local porque son ligeros (decenas de milisegundos por consulta, según las mediciones del Capítulo III), y la detección de presencia se ejecuta en el navegador para que los fotogramas de la cámara no salgan del dispositivo del usuario. El LLM también se ejecuta localmente: el modelo Nemotron 3 Nano de 4 mil millones de parámetros, cuantizado a 4 bits (Q4_K_M), ocupa unos 3 GB y cabe por completo en la GPU de 8 GB del equipo, lo que elimina la dependencia de la cola de un servicio remoto, el costo por consulta y el envío de las preguntas a terceros. La transcripción y la síntesis de voz con calidad natural en español siguen consumiéndose mediante API. La arquitectura multiproveedor permite volver a un LLM remoto o usar el servidor Ollama de LabIT sin modificar el resto del sistema.

2.7.2 Selección Comparativa del Proveedor de LLM

Durante el desarrollo se integraron varios proveedores compatibles con la interfaz de OpenAI. La Tabla 10 resume los criterios que llevaron a fijar, primero, la API de NVIDIA y, tras la evaluación de latencia, el modelo local Nemotron como opción por defecto.

Tabla 10: Comparación de los proveedores de LLM integrados en el prototipo.

Proveedor	Modelo configurado	Ventaja	Limitación observada
Google Gemini	gemini-2.5-flash-lite	Integración inicial sencilla	Recibe el documento completo en lugar del RAG; límites de cuota
SiliconFlow	DeepSeek-V3	Modelo de alta capacidad	Servicio alojado en China (api.siliconflow.cn); mayor distancia de red
Groq	llama-3.1-8b-instant	Inferencia muy rápida	Límites de <i>tokens</i> por minuto en el plan gratuito
Open WebUI / Ollama	qwen3:14b	Datos en infraestructura propia (GPU LabIT)	Depende de la disponibilidad del laboratorio y del túnel SSH
NVIDIA API	llama-3.2-11b-vision-instruct	Catálogo gratuito para desarrollo y API estable	Cola del servicio gratuito: LLM de 1,2 a 24,8 s por consulta (Capítulo III) [43]
Ollama local	nemotron-3-nano:4b (Q4_K_M)	Sin costo ni cuotas; datos en el equipo; primer fragmento en menos de 0,5 s	Modelo pequeño; requiere GPU y desactivar la caché de <i>prompts</i> en RAM

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Configuración del prototipo (`.env.example`) e historial de cambios del repositorio

2.7.3 Implementación de la Arquitectura RAG frente a Fine-Tuning

Para dotar al LLM del conocimiento institucional, se seleccionó la arquitectura RAG sobre el reentrenamiento (*fine-tuning*). RAG permite responder con información actualizable sin modificar los pesos del modelo [38], mientras que el *fine-tuning* es costoso y no garantiza la exactitud de datos puntuales como fechas o valores de aranceles [10]. Con RAG, la universidad puede actualizar un documento desde el panel de administración y el asistente utiliza la nueva información de inmediato.

2.7.4 Modelo de Embeddings Multilingüe

Inicialmente se empleó all-MiniLM-L6-v2, entrenado principalmente en inglés, con el que las consultas en español no recuperaban los fragmentos correctos. Se adoptó paraphrase-multilingual-MiniLM-L12-v2, un modelo Sentence-BERT de 384 dimensiones entrenado para más de 50 idiomas [24], que se ejecuta en CPU en milisegundos por consulta.

2.7.5 Visión Artificial en el Cliente mediante Cajas Delimitadoras

La primera versión aplicaba el clasificador Haar Cascade de OpenCV [28] en el servidor, lo que obligaba al navegador a enviar cada segundo un fotograma JPEG de 7 a 55 KB y al servidor a decodificarlo y procesarlo por cada usuario conectado. Como el requerimiento se limita a saber si hay un rostro frente a la pantalla, sin reconocer identidades ni gestos, la detección se trasladó al navegador con MediaPipe Face Detector y el modelo BlazeFace de corto alcance, de 230 KB y licencia Apache 2.0 [29, 30]. El navegador obtiene las cajas delimitadoras y envía solo sus coordenadas, por lo que la imagen no sale del dispositivo, el mensaje se reduce a decenas de bytes y el costo de la visión deja de crecer con el número de usuarios en el servidor.

2.7.6 Avatar en Formato VRM sobre WebGL

Se utilizó el formato abierto VRM, basado en glTF, con modelos creados en VRoid y renderizados con Three.js y three-vrm [31]. El formato define expresiones estandarizadas (parpadeo y vocales) que

permiten animar el rostro sin un motor de videojuegos, y el avatar se ejecuta en cualquier navegador moderno, incluidos los móviles, sin instalación adicional.

CAPÍTULO III

IMPLEMENTACIÓN, PRUEBAS Y RESULTADOS

3.1 Descripción de la Solución

La solución implementada es ARIA, un asistente virtual web compuesto por un *frontend* en HTML, CSS y JavaScript con un avatar 3D, y un *backend* en Python con FastAPI. El código fuente se encuentra versionado en GitHub (Anexo A) y los componentes desarrollados, configurados o integrados se resumen en la Tabla 11.

Tabla 11: Componentes desarrollados, configurados e integrados en el prototipo.

Componente	Tipo	Descripción
api.py	Desarrollado	API FastAPI: autenticación, chat con RAG en <i>streaming</i> por WebSocket, STT, TTS, panel RAG y WebSocket de presencia.
rag_manager.py	Desarrollado	Parámetros comunes del RAG, preprocesamiento de consultas, recuperador híbrido (MiniLM, BM25, RRF y reordenamiento) y reconstrucción de la base vectorial en segundo plano.
init_db.py	Desarrollado	Base SQLite de usuarios y sesiones; contraseñas con PBKDF2-HMAC-SHA256 y <i>salt</i> .
static/	Desarrollado	Interfaz web (inicio de sesión, chat, modo conversacional y panel de administración), lógica del avatar y detección facial en el navegador.
evaluacion/	Desarrollado	Banco de preguntas y <i>scripts</i> de evaluación reproducible.
ChromaDB	Configurado	Base vectorial persistente con métrica coseno (colección <i>carrera_ti_indoamerica_collection</i>).
Avatar ARIA2.0.vrm	Configurado	Modelo VRM creado en VRoid, animado con Three.js y <i>three-vrm</i> .
Ollama + Nemotron 3 Nano 4B	Configurado	LLM local en la GPU del equipo (<i>nemotron-3-nano:4b</i> , Q4_K_M).
MediaPipe Face Detector	Integrado	Detección facial BlazeFace en el navegador (WebAssembly/WebGL), modelo servido desde <i>static/models/</i> .
Groq, ElevenLabs	Integrado	Servicios de STT y TTS consumidos mediante API.
Docker, Coolify, Traefik	Configurado	Contenedores y publicación HTTPS del despliegue en servidor.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Repositorio del prototipo

3.1.1 Entorno de Ejecución

Todas las pruebas de este capítulo se ejecutaron en **modo local**, en el equipo del desarrollador, cuyas características se detallan en la Tabla 12. La GPU ejecuta el LLM Nemotron (servido por Ollama) y el modelo de reordenamiento; el modelo de *embeddings* y la búsqueda BM25 se ejecutan en CPU; la detección de presencia se ejecuta en el navegador; y la transcripción y la síntesis de voz se realizan en los proveedores remotos.

Tabla 12: Entorno de hardware y software de las pruebas (modo local).

Elemento	Especificación
Procesador	AMD Ryzen 7 7735HS (8 núcleos, 16 hilos)
Memoria RAM	14 GiB disponibles para el sistema
GPU	NVIDIA GeForce RTX 4060 Laptop, 8 GB (LLM y reordenador)
Sistema operativo	Ubuntu 24.04 LTS, Python 3.12.3
Cliente	Navegador web con cámara web y micrófono integrados
LLM	Ollama 0.34.4, nemotron-3-nano:4b (4.000 millones de parámetros, Q4_K_M), contexto de 8.192 <i>tokens</i> , caché de <i>prompts</i> en RAM desactivada
Recuperación	paraphrase-multilingual-MiniLM-L12-v2, BM25 y cross-encoder/mmarco-mMiniLMv2-L12-H384-v1
STT / TTS	Groq whisper-large-v3 / ElevenLabs eleven_flash_v2_5

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Equipo de desarrollo y archivo de configuración `.env`

3.1.2 Software y Herramientas

La Tabla 13 resume las herramientas necesarias para ejecutar el prototipo.

Tabla 13: Software y herramientas requeridas para la implementación.

Herramienta / Framework	Propósito y Configuración
Python 3.10+	Lenguaje del <i>backend</i> ; entorno virtual (.venv) y dependencias en requirements.txt.
FastAPI y Uvicorn	API REST, WebSocket (chat en <i>streaming</i> y presencia) y servicio de archivos estáticos.
Ollama	Servidor local del LLM Nemotron con API HTTP; variables LLAMA_ARG_CACHE_RAM=0 y LLAMA_ARG_CTX_CHECKPOINTS=0.
ChromaDB	Base de datos vectorial embebida con persistencia en disco (chroma_db/).
Sentence-Transformers	Vectorización con paraphrase-multilingual-MiniLM-L12-v2 y reordenamiento con un <i>cross-encoder</i> multilingüe.
rank-bm25	Índice léxico BM25 sobre los fragmentos de la colección.
PyMuPDF4LLM y LangChain	Conversión de PDF a Markdown y fragmentación (MarkdownTextSplitter).
HTML, CSS y JavaScript	<i>Frontend</i> sin <i>framework</i> ; Three.js, three-vm y MediaPipe Tasks Vision 0.10.14 cargados desde CDN.
Docker y Docker Compose	Despliegue en servidor (modo servidor).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Archivo requirements.txt y static/script.js del prototipo

3.2 Proceso de Implementación

El proceso de implementación siguió el enfoque iterativo descrito en el Capítulo II, comenzando por la base de conocimiento y culminando con la interfaz, el despliegue y la evaluación.

3.2.1 Paso 1: Construcción de la Base de Datos Vectorial

El documento base de la carrera se convierte a Markdown con PyMuPDF4LLM, conservando la estructura de títulos y tablas, y se divide con MarkdownTextSplitter en fragmentos de 800 caracteres con un solapamiento de 160, lo que produjo 89 fragmentos (la elección de este tamaño se justifica en la Sección 3.4.1). Cada fragmento se convierte en un vector de 384 dimensiones con SentenceTransformers y se inserta en la colección `carrera_ti_indoamerica_collection`, configurada con la métrica coseno. Este proceso puede ejecutarse por consola (`build_rag_index.py`) o desde el panel de administración, y ambos comparten los mismos parámetros definidos en `rag_manager.py`.

La Figura 12 presenta el modelo conceptual inicial del registro vectorial. En la implementación final, cada registro contiene el identificador del fragmento, su texto, su vector y el nombre del documento de origen, como se muestra en la Figura 13.

DIAGRAMA DE BASE DE DATOS VECTORIAL (ChromaDB) - Colección: `carrera_ti_indoamerica_collection`

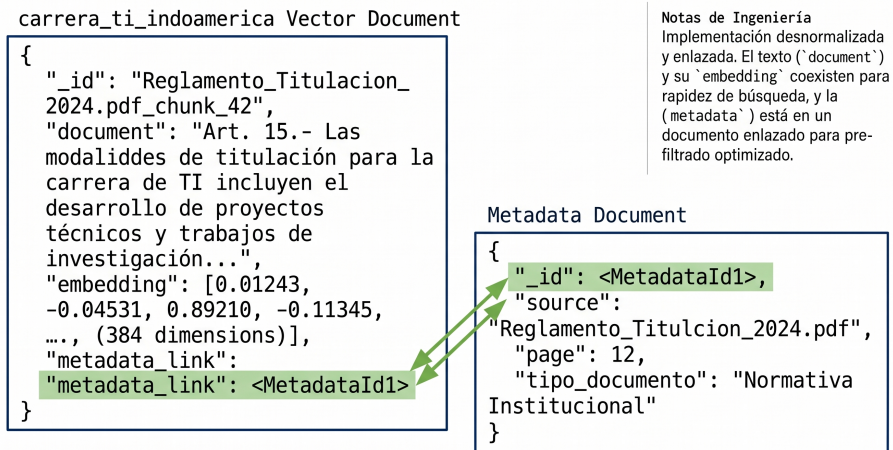


Figura 12: Estructura lógica y proceso de vectorización en la base de datos ChromaDB (modelo conceptual).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

El texto (document) y su vector se almacenan juntos para que la búsqueda devuelva directamente el contenido que se inyecta en el *prompt*; la metadata conserva el documento de origen para trazabilidad. Al iniciar el servidor, y cada vez que se reconstruye la colección,

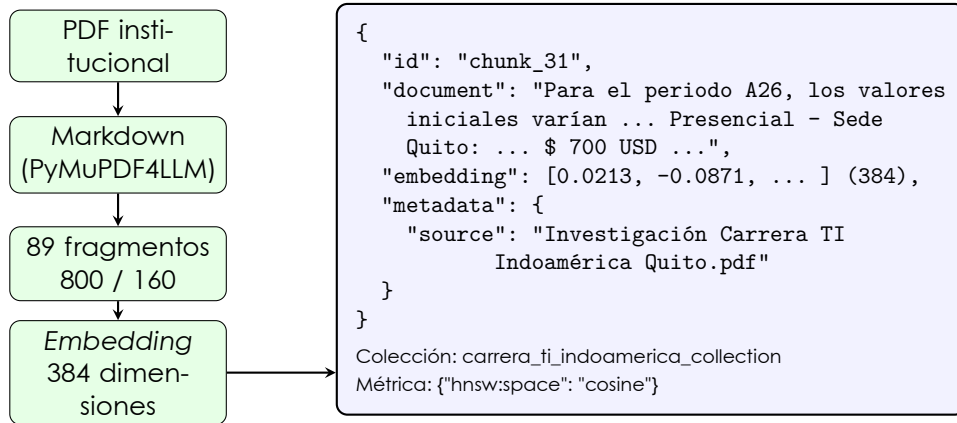


Figura 13: Registro real almacenado en ChromaDB y proceso de indexación implementado.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia a partir de `build_rag_index.py`

el recuperador construye en memoria el índice léxico BM25 sobre los mismos fragmentos. La Imagen 1 muestra el panel desde el que el administrador reemplaza los documentos y supervisa el progreso de la reconstrucción.

3.2.2 Paso 2: Construcción del Servidor Backend (FastAPI)

Una vez establecida la base de datos, se programó el orquestador principal en `api.py`. La Tabla 14 resume los *endpoints* expuestos y su nivel de acceso.

Detección de presencia. El navegador ejecuta una vez por segundo MediaPipe Face Detector sobre el video de la cámara y envía por el WebSocket `/ws` un mensaje JSON con las cajas delimitadoras detectadas, por ejemplo `{"cajas": [[112, 64, 96, 118]]}`, o una lista vacía si no hay rostro. En el servidor, la clase `DetectorPresencia` evita falsos disparos mediante contadores de fotogramas consecutivos con rostro (p_t) y sin rostro (m_t), según la Ecuación 4:

$$Evento_t = \begin{cases} \text{person_arrived,} & \text{si } p_t \geq 2 \text{ y el usuario no estaba presente} \\ \text{person_left,} & \text{si } m_t \geq 4 \text{ y el usuario estaba presente} \\ \text{sin evento,} & \text{en otro caso} \end{cases} \quad (4)$$

Tabla 14: *Endpoints* del backend y control de acceso.

Endpoint	Función	Acceso
POST /api/login, /api/logout	Inicio y cierre de sesión (cookie httponly)	Público
GET /api/me	Usuario y rol de la sesión	Usuario
WS /ws/chat	Filtro anti- <i>jailbreak</i> , RAG híbrido Top-3 y LLM; envía cada fragmento de la respuesta (<i>token</i>) y al final la respuesta completa con sus latencias (<i>fin</i>)	Usuario
POST /api/chat	Misma lógica que /ws/chat en una sola respuesta (compatibilidad)	Usuario
POST /api/stt	Transcripción con Whisper (Groq)	Usuario
POST /api/tts	Síntesis de voz con ElevenLabs en <i>streaming</i>	Usuario
WS /ws	Eventos de presencia a partir de las cajas delimitadoras enviadas por el navegador	Usuario
POST /api/admin/ upload_docs	Carga de PDF y reindexación en segundo plano	Administrador
GET /api/admin/rag_ status	Progreso de la reindexación	Administrador
GET /debug/rag, /debug/md	Diagnóstico de la recuperación	Administrador

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Código fuente `api.py`

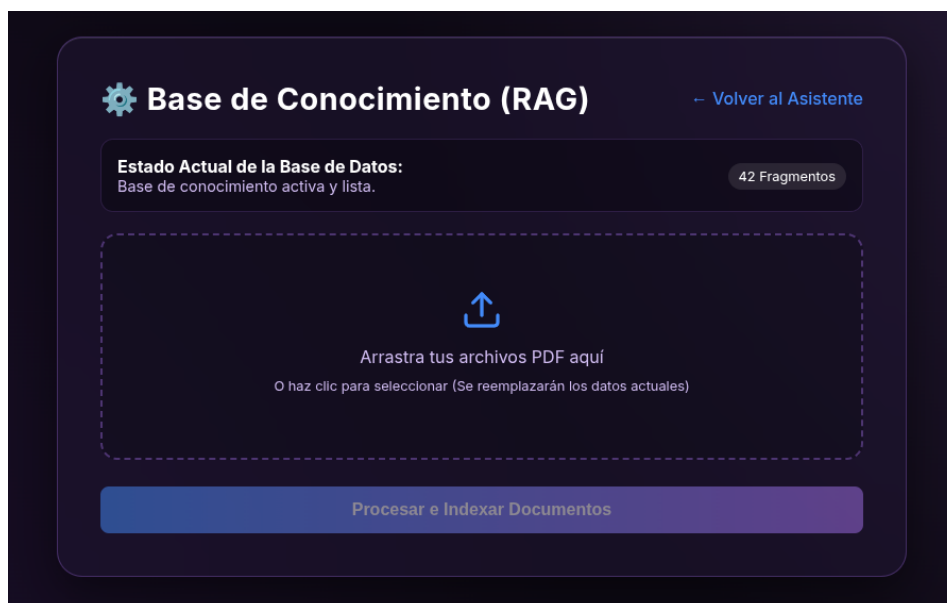


Imagen 1: Panel de Administración del RAG para actualizar la base documental.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Captura de pantalla del prototipo

Con un fotograma por segundo, la llegada se confirma en unos 2 segundos y la salida en unos 4 segundos. Ninguna imagen se transmite al servidor: el contenido del mensaje se limita a las coordenadas, el ancho y el alto de cada rostro.

Recuperación híbrida. Para cada consulta, el recuperador obtiene los 20 fragmentos más similares por distancia coseno en ChromaDB y los 20 con mayor puntuación BM25, sobre un texto normalizado (minúsculas, sin acentos ni palabras vacías y con los términos truncados a 6 caracteres). Ambas listas se fusionan por RRF con $k = 60$ (Ecuación 2) y el cross-encoder `mmarco-mMiniLMv2-L12-H384-v1`, entrenado en la versión multilingüe de MS MARCO [27], puntúa cada par (pregunta, fragmento) en la GPU para seleccionar los 3 que se entregan al LLM. Antes de insertarlos en el *prompt* se elimina el marcado que no aporta información (negritas y referencias numéricas), lo que reduce en un 6 % los *tokens* del contexto.

Respuesta en *streaming*. El LLM se invoca a través de la API de Ollama con temperatura 0,1, un máximo de 500 *tokens* y el razonamiento interno desactivado. Cada fragmento de texto que genera el modelo se reenvía de inmediato al navegador por el WebSocket `/ws/chat`, de modo que el tiempo hasta el primer fragmento (TTFB) depende

solo de la recuperación y del procesamiento del *prompt*, y no de la longitud de la respuesta. El servidor conserva por sesión los tres últimos turnos de la conversación para atender preguntas de seguimiento.

Pipeline de voz. El frontend graba el audio entre dos pulsaciones del micrófono, lo transcribe con `/api/stt`, envía el texto por `/ws/chat` y, al completarse la respuesta, solicita el audio a `/api/tts`. En el chat, la consulta se normaliza y se expande, se recuperan los 3 fragmentos más relevantes y se envían al LLM junto con las reglas del sistema (identidad de ARIA, alcance limitado a la carrera, brevedad, uso exclusivo del contexto y prohibición de cambiar de rol). Antes de la recuperación, un filtro de expresiones regulares bloquea intentos de *jailbreak* sin consultar al LLM. La Figura 14 resume esta orquestación.

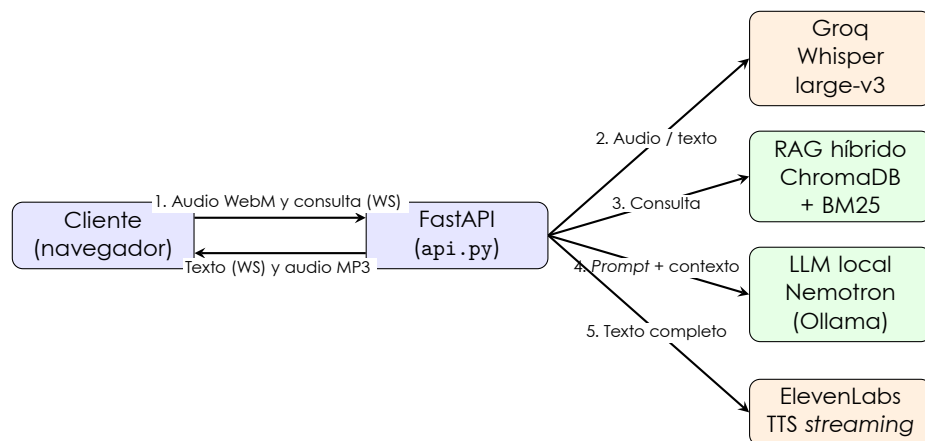


Figura 14: Orquestación del procesamiento de voz en el backend.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

3.2.3 Paso 3: Desarrollo de la Capa de Cliente (Frontend)

El *frontend* integra Three.js y three-vm para renderizar el avatar ARIA en formato VRM, creado en VRoid. El avatar parpadea, respira y mantiene una postura natural en reposo; mientras habla, un `AnalyserNode` de la Web Audio API mide la energía del audio y la traduce en la apertura de la boca. La interfaz ofrece dos modos (Figura 15): el modo conversacional, con el avatar en primer plano, el micrófono, subtítulos y un carrusel de preguntas sugeridas; y el modo chat, con el historial de mensajes en texto con formato Markdown, en el que la respuesta se escribe a medida que llegan sus fragmentos por el WebSocket. La captura de la cámara y la detección facial

se ejecutan en el mismo navegador con MediaPipe Tasks Vision; el modelo BlazeFace (230 KB) se sirve desde el propio servidor y se carga una sola vez al activar la cámara.

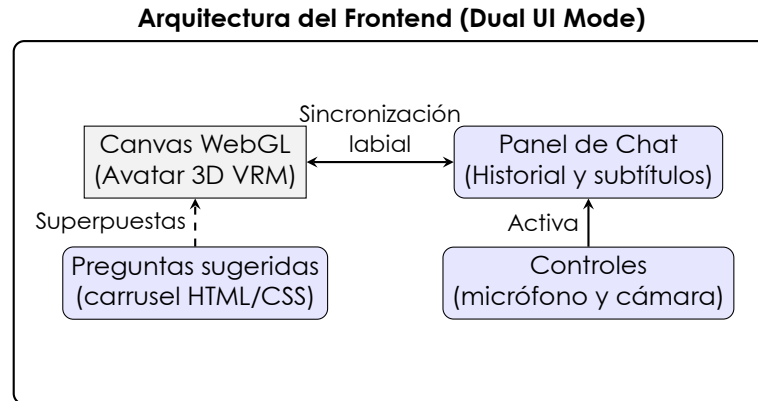


Figura 15: Diagrama de bloques de la Interfaz Dual y composición de elementos web.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Elaboración propia

Las Imágenes 2 a 4 muestran las pantallas principales del prototipo.

3.2.4 Paso 4: Despliegue

En **modo local**, el *script* `iniciar_local.sh` levanta el servidor con Uvicorn en `http://localhost:8000`; los navegadores permiten el uso de cámara y micrófono en `localhost` sin certificado. El LLM se sirve con el servicio de Ollama del mismo equipo, que el servidor precarga al iniciar y mantiene en memoria durante 60 minutos. Durante las pruebas se observó que, con su configuración por defecto, el proceso de inferencia de Ollama reserva hasta 8 GB de RAM para guardar estados de *prompts* anteriores (81 MB por cada punto de control del modelo híbrido de Nemotron), hasta que el sistema operativo lo termina por falta de memoria; como en este caso de uso esos estados no se reutilizan, se desactivaron con las variables `LLAMA_ARG_CACHE_RAM=0` y `LLAMA_ARG_CTX_CHECKPOINTS=0` en el servicio, con lo que el proceso se estabiliza en menos de 1 GB de RAM. En **modo servidor**, `docker-compose.yml` construye el contenedor de la aplicación, monta un volumen persistente para la base vectorial y publica el servicio mediante Traefik con certificados de Let's Encrypt, administrado des-

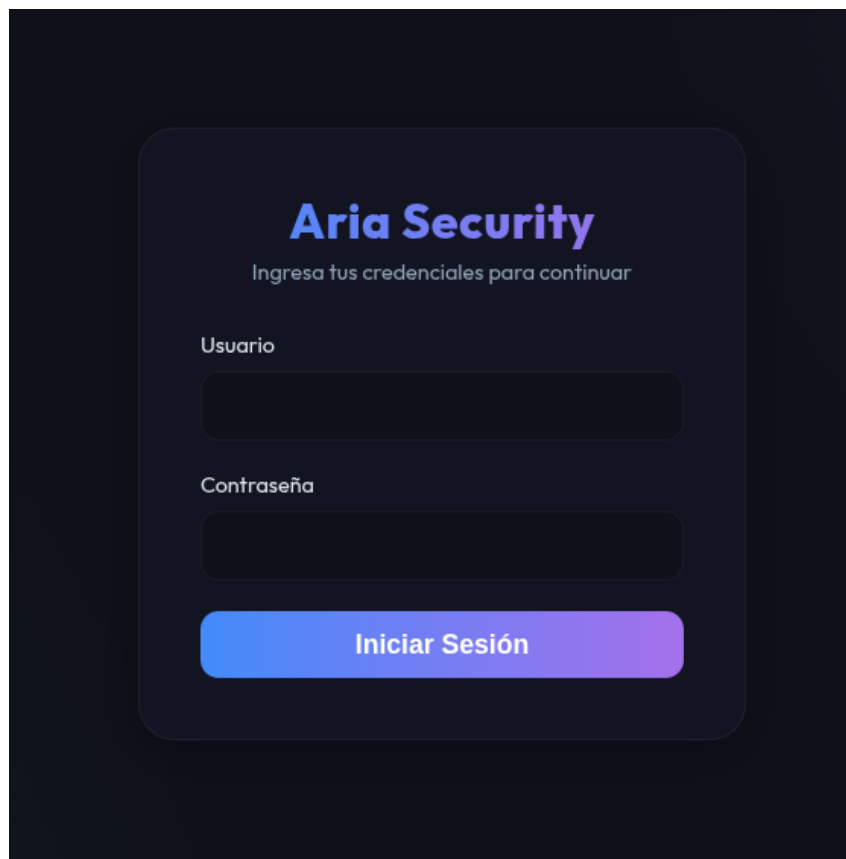


Imagen 2: Pantalla de inicio de sesión del Asistente Virtual.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Captura de pantalla del prototipo

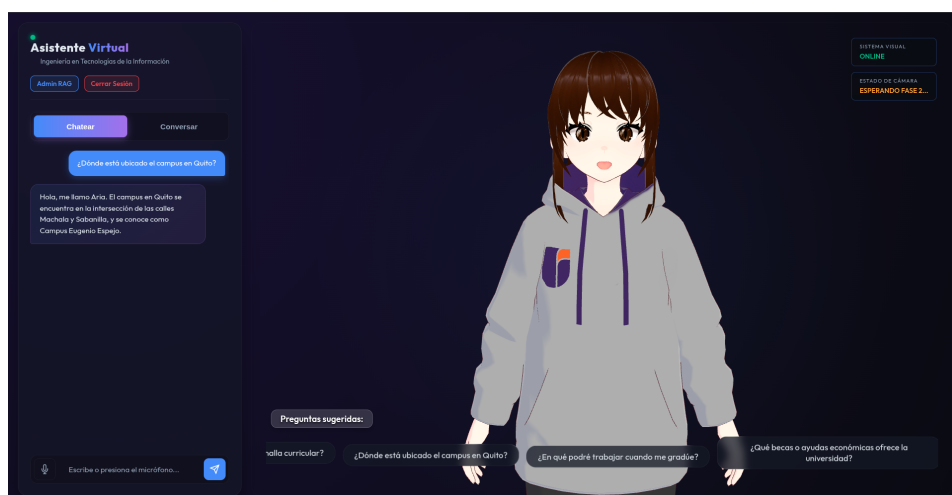


Imagen 3: Modo Chat del Asistente Virtual.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Captura de pantalla del prototipo

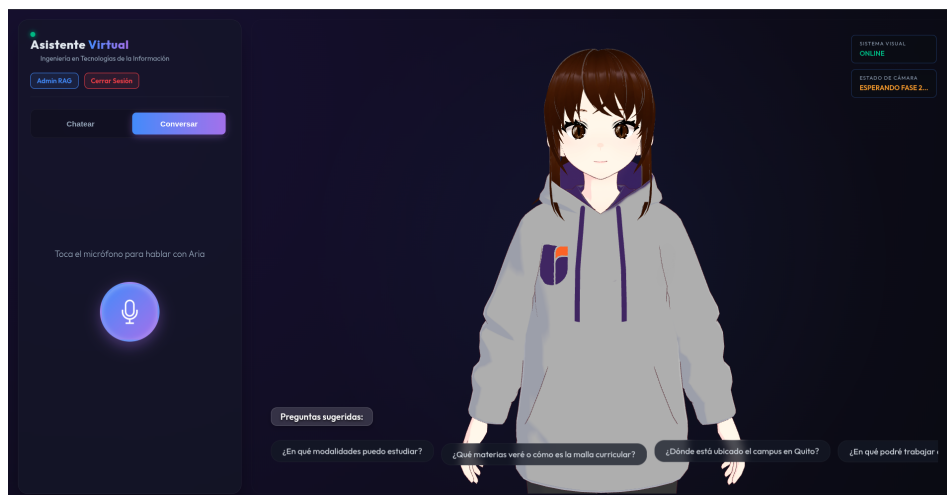


Imagen 4: Modo Conversacional (avatar en primer plano con controles de voz).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Captura de pantalla del prototipo

de Coolify; un segundo contenedor mantiene el túnel SSH hacia el servidor Ollama de LabITI. En ambos modos, las claves de API y las contraseñas iniciales se definen en el archivo `.env`, excluido del control de versiones.

3.3 Pruebas y Validación

Las pruebas se ejecutaron en modo local entre el 21 de julio (primera sesión de la encuesta) y el 7 de octubre de 2026 (pruebas técnicas). Las pruebas técnicas se automatizaron con los *scripts* de la carpeta `evaluacion/` del repositorio, que reutilizan las funciones de producción o se conectan al servidor en ejecución, de modo que cualquier evaluador puede repetirlas. Las mediciones técnicas corresponden a la versión final del prototipo (Fase 6), mientras que la encuesta de usabilidad se aplicó sobre las versiones de julio (Fase 3) y de inicios de octubre (Fase 5).

3.3.1 Pruebas Funcionales (Caja Negra)

La Tabla 15 resume los casos ejecutados sobre los requerimientos funcionales y su resultado.

Tabla 15: Resultado de las pruebas funcionales.

RF	Caso de prueba	Resultado obtenido	Estado
RF-01	Fotograma con rostro y fotograma sin rostro (320×240) con BlazeFace	1 rostro detectado; ninguna detección sin rostro; 2,8 ms por fotograma	Cumple
RF-02	Transcripción de 40 preguntas habladas	WER medio de 1,9%; 35 de 40 exactas	Cumple
RF-03	Recuperación sobre los bancos de desarrollo y de prueba (50 preguntas cada uno)	<i>Hit Rate@3</i> de 92 % y 94 %	Cumple
RF-04	80 preguntas del dominio y 10 fuera de dominio	80 % y 90 % de respuestas correctas; 10/10 rechazos	Cumple
RF-05	Síntesis de las 30 respuestas	30/30 audios generados	Cumple
RF-06	Llegada, salida, retorno y ruido frente a la cámara (secuencias de detecciones)	5 de 5 casos con el evento esperado en el fotograma esperado	Cumple
RF-07	Acceso sin sesión, con clave errónea y con rol regular	401 / 4401 / 401 / 403 según corresponde	Cumple
RF-08	Reconstrucción del índice del documento base	89 fragmentos indexados en 10,0 s	Cumple

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Resultados de los *scripts* de evaluación (*evaluacion/resultados/*)

3.3.2 Evaluación del Motor de Recuperación

Se construyeron dos bancos de 50 preguntas en lenguaje estudiantil sobre las categorías del documento (malla curricular, prácticas, perfil de egreso, titulación, inglés, aranceles, pagos, becas, homologación, contacto, admisiones y datos de la carrera). El **banco de desarrollo** se utilizó para comparar estrategias de recuperación y tamaños de fragmento; el **banco de prueba**, redactado antes de

las optimizaciones y no utilizado para ajustar ningún parámetro, se reservó para la medición final de la configuración elegida, con el fin de evitar que el resultado reportado esté sobreajustado a las preguntas usadas durante el desarrollo. Para cada pregunta se definió como *gold standard* una o más frases literales del documento que deben aparecer en el fragmento recuperado; el *script* verifica que cada frase exista en el corpus antes de evaluar. Se calcularon el *Hit Rate* en los k primeros resultados (Ecuación 5) y el Rango Recíproco Medio (MRR, Ecuación 6) [44]:

$$\text{Hit Rate}@k = \frac{1}{N} \sum_{i=1}^N 1 [r_i \leq k] \times 100 \quad (5)$$

$$\text{MRR}@5 = \frac{1}{N} \sum_{i=1}^N \frac{1}{r_i}, \quad \frac{1}{r_i} = 0 \text{ si } r_i > 5 \quad (6)$$

donde N es el número de preguntas y r_i la posición del primer fragmento relevante para la pregunta i . El *Hit Rate*@3 es la métrica principal, porque el sistema entrega al LLM los 3 primeros fragmentos.

3.3.3 Evaluación de la Exactitud de las Respuestas

Las respuestas generadas para las 30 primeras preguntas del banco de desarrollo y para las 50 del banco de prueba, cada una en una sesión nueva y sin historial, se contrastaron manualmente con el documento fuente y se clasificaron como correctas, parciales, incorrectas o sin respuesta. Además, se enviaron las 10 preguntas fuera de dominio para verificar el rechazo.

3.3.4 Evaluación del Tiempo de Respuesta (TTFB)

El tiempo hasta el primer fragmento de la respuesta (TTFB, *Time to First Byte*) se midió de extremo a extremo contra el servidor en ejecución, con un cliente que reproduce el comportamiento del navegador: abre el WebSocket `/ws/chat` con una sesión válida, envía cada pregunta del banco de prueba y registra el instante en que recibe el primer fragmento de texto (Ecuación 7):

$$TTFB = t_{\text{primer fragmento}} - t_{\text{envío de la consulta}} \quad (7)$$

Las 50 preguntas se enviaron en una misma sesión, tras dos consultas de calentamiento, por lo que el *prompt* incluye el historial de los turnos anteriores, como en una conversación real. El TTFB incluye la recuperación híbrida, el procesamiento del *prompt* por el LLM y la transmisión por WebSocket.

3.3.5 Evaluación de Latencia de la Interacción por Voz

Para medir la latencia de una interacción completa por voz, cada pregunta del banco de desarrollo se sintetizó como audio con una voz neuronal de *edge-tts* (simulando al estudiante) y se procesó con el mismo flujo que el prototipo: transcripción en Groq, recuperación y generación mediante la función de producción *generar_respuesta()* y síntesis en ElevenLabs. Se midió el tiempo de cada etapa en 30 preguntas del dominio y 10 fuera de dominio, tras una consulta de calentamiento. Como el navegador solicita el audio cuando la respuesta está completa, la latencia percibida se define como el tiempo desde que el audio del estudiante llega al servidor hasta que se recibe el primer fragmento de audio de la respuesta (Ecuación 8):

$$L_{\text{percibida}} = t_{STT} + t_{\text{respuesta}} + TTFB_{TTS} \quad (8)$$

donde $t_{\text{respuesta}}$ incluye la recuperación y la generación completa del texto. Para cada medición se calcularon la media ($\bar{x}$), la desviación estándar (s), el percentil 95 (P95) y el intervalo de confianza al 95% de la media (Ecuaciones 9 a 11):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (9)$$

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (10)$$

$$IC_{95} = \bar{x} \pm 1,96 \times \frac{s}{\sqrt{n}} \quad (11)$$

3.3.6 Evaluación del Módulo de Visión

Para cuantificar el costo de la detección de presencia se ejecutó el mismo modelo que usa el navegador (BlazeFace de corto alcance) con la API de Python de MediaPipe sobre un fotograma con rostro (imagen “astronaut” de la NASA, de dominio público) y uno sin rostro, en resoluciones de 320×240 y 640×480, con 300 repeticiones por caso. Se midió además el tamaño del mensaje enviado al servidor y se comparó con el del esquema anterior, que enviaba el fotograma JPEG en Base64. En el navegador la inferencia se ejecuta con WebAssembly o WebGL, por lo que su tiempo exacto depende del equipo del usuario.

3.3.7 Pruebas de Presencia y de Seguridad

La lógica de presencia se probó alimentando la clase `Detector Presencia` con secuencias de detecciones (llegada, salida y retorno; un rostro en un solo fotograma; una ausencia breve de 3 fotogramas) y, de extremo a extremo, abriendo el WebSocket `/ws` con y sin sesión. En seguridad, con el cliente de pruebas de FastAPI y una copia temporal de la base de usuarios se verificó que los *endpoints* de chat, voz y diagnóstico rechazan peticiones sin sesión (código 401), que los WebSocket se cierran con el código 4401 sin sesión, que un usuario regular no accede a las rutas de administración (código 403) y que las contraseñas almacenadas con el esquema anterior (SHA-256 sin *salt*) se migran a PBKDF2 al iniciar sesión.

3.3.8 Pruebas de Usabilidad

Un total de 16 estudiantes de la carrera utilizaron el prototipo en modo local y respondieron el cuestionario del Anexo B, previo consentimiento informado (Anexo C): 13 el 21 de julio de 2026 y 3 entre el 3 y el 4 de octubre de 2026. La encuesta evaluó las versiones de las Fases 3 y 5, anteriores a la migración del LLM local y a las optimizaciones de recuperación y latencia de la Fase 6.

3.4 Resultados

3.4.1 Resultados del Motor de Recuperación

La Tabla 16 compara, sobre el banco de desarrollo, la configuración anterior (solo búsqueda semántica con fragmentos de 1.500 caracteres) con las estrategias evaluadas en la Fase 6 sobre fragmentos de 800 caracteres.

Tabla 16: Comparación de estrategias de recuperación (banco de desarrollo, 50 preguntas).

Estrategia	Hit@1	Hit@3	Hit@5	MRR@5
Semántica, 1.500/300 (versión anterior)	34 %	62 %	70 %	0,489
Semántica (MiniLM), 800/160	42 %	60 %	66 %	0,511
Léxica (BM25), 800/160	56 %	76 %	78 %	0,654
Híbrida (RRF), 800/160	42 %	72 %	80 %	0,566
Híbrida + reordenamiento, 800/160	60 %	92 %	96 %	0,748

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: `evaluacion/experimento_recuperacion.py`

La búsqueda semántica por sí sola no distingue bien fragmentos de vocabulario similar, como las listas de asignaturas de niveles distintos, mientras que BM25 aprovecha los términos exactos de la consulta. La fusión de ambas amplía los candidatos, y el reordenamiento con el *cross-encoder* es el paso decisivo: eleva el *Hit Rate*@3 de 72 % a 92 %. La Tabla 17 muestra la sensibilidad de esta estrategia al tamaño de fragmento.

Tabla 17: Sensibilidad de la recuperación híbrida con reordenamiento al tamaño de fragmento (banco de desarrollo).

Tamaño / solapamiento	Fragmentos	Hit@1	Hit@3	MRR@5
1.500 / 300	42	72 %	94 %	0,828
1.000 / 200	64	56 %	92 %	0,733
800 / 160 (producción)	89	60 %	92 %	0,748
700 / 140	104	60 %	92 %	0,751
600 / 120	120	54 %	86 %	0,698

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/experimento_recuperacion.py

Entre 700 y 1.500 caracteres el *Hit Rate@3* varía entre 92 % y 94 %, una diferencia de una sola pregunta. Se eligió 800/160 porque, con un acierto equivalente, el contexto de 3 fragmentos ocupa aproximadamente la mitad de *tokens* que con fragmentos de 1.500 caracteres (unos 390 frente a unos 750), lo que a la velocidad de procesamiento del *prompt* medida en el equipo (cerca de 3.000 *tokens* por segundo) supone unos 120 ms menos de TTFB por consulta (estimación). La Tabla 18 presenta la medición final de la configuración de producción en ambos bancos.

Tabla 18: Resultado de la configuración de producción (híbrida + reordenamiento, 800/160, Top-3).

Banco	Hit@1	Hit@3	Hit@5	MRR@5	Tiempo (ms)
Desarrollo (50)	60 %	92,0 %	96 %	0,748	28,6
Prueba reservado (50)	84 %	94,0 %	100 %	0,893	28,9

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/evaluar_rag.py

La configuración de producción alcanzó un *Hit Rate@3* de 92,0 % en el banco de desarrollo (46 de 50 preguntas) y de 94,0 % en el banco de prueba reservado (47 de 50), cuyo fragmento correcto apareció siempre entre los 5 primeros. Que el resultado se mantenga en preguntas no usadas durante el ajuste indica que la mejora no se debe a un sobreajuste al banco de desarrollo. La recuperación completa, incluido el reordenamiento en la GPU, tomó en promedio 29 ms. Los fallos restantes corresponden a preguntas cuya respuesta está en

fragmentos con vocabulario muy cercano a otros (por ejemplo, las asignaturas del segundo nivel frente a las de otros niveles, o la ponderación de la experiencia profesional frente a sus años mínimos en la homologación). La Figura 16 presenta el desempeño por categoría en el banco de prueba.

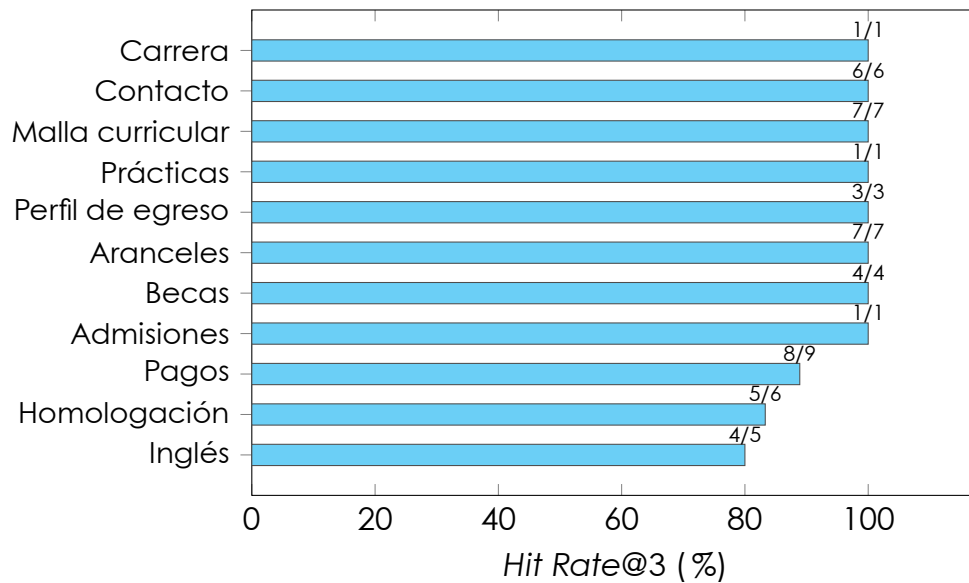


Figura 16: *Hit Rate@3* por categoría en el banco de prueba (acierto / pregunta).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: `evaluacion/resultados/rag_detalle.csv`

3.4.2 Exactitud de las Respuestas Generadas

La Tabla 19 presenta la clasificación de las respuestas de la versión final y, como referencia, la de la versión anterior (Llama 3.2 11B por la API de NVIDIA y búsqueda solo semántica) sobre las mismas 30 preguntas de desarrollo. El detalle de cada respuesta y su justificación se encuentra en `evaluacion/resultados/respuestas_ollama.csv`.

Tabla 19: Clasificación de las respuestas generadas.

Clasificación	Versión anterior, desarrollo (30)	Versión final, desarrollo (30)	Versión final, prueba (50)
Correcta	14 (46,7 %)	24 (80,0 %)	45 (90,0 %)
Parcial	3 (10,0 %)	2 (6,7 %)	2 (4,0 %)
Sin respuesta	7 (23,3 %)	2 (6,7 %)	1 (2,0 %)
Incorrecta	6 (20,0 %)	2 (6,7 %)	2 (4,0 %)

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/resultados/exactitud_respuestas.csv y respuestas_ollama.csv

Las respuestas correctas pasaron de 46,7 % a 80,0 % en las mismas preguntas y alcanzaron el 90,0 % en el banco de prueba. De las 4 respuestas incorrectas de la versión final, 2 se deben a fallos de recuperación (el fragmento correcto no estaba entre los 3 primeros) y 2 a que el modelo, con el fragmento correcto disponible, respondió con datos cercanos: indicó dónde se realizan las prácticas preprofesionales en lugar de su nivel y enumeró las asignaturas del octavo nivel como competencias del perfil de egreso. Durante el ajuste se detectó que el modelo añadía a veces la frase de "información no encontrada" después de una respuesta correcta; se precisó la regla correspondiente del *prompt* del sistema usando solo el banco de desarrollo. Las 10 preguntas fuera de dominio (recetas, deportes, política, otras carreras, generación de código, intento de *jailbreak*, entre otras) fueron rechazadas; el intento de *jailbreak* fue bloqueado por el filtro de código sin consultar al LLM.

3.4.3 Resultados del Tiempo de Respuesta (TTFB)

La Tabla 20 presenta el TTFB medido por WebSocket sobre las 50 preguntas del banco de prueba.

Tabla 20: Tiempo hasta el primer fragmento de la respuesta por WebSocket (banco de prueba, 50 consultas, en milisegundos).

Medición	Media	<i>s</i>	Mediana	P95	Máx.	IC ₉₅
RAG híbrido	34,5	4,1	33,8	44,5	46,0	±1,1
TTFB (WebSocket)	385,2	38,7	383,1	440,8	473,5	±10,7
Respuesta completa	711,7	155,4	710,0	1.031,8	1.042,1	±43,1

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/evaluar_ttfb_ws.py, ejecutado el 7 de octubre de 2026

El TTFB promedio fue de 385 ms (IC 95%: 375 a 396 ms), con 49 de las 50 consultas por debajo de 450 ms y un máximo de 474 ms, por lo que se cumple la meta de un TTFB promedio no mayor a 450 ms (RNF-02). La diferencia entre el tiempo medido por el cliente y el registrado en el servidor fue inferior a 1 ms, de modo que el WebSocket no añade latencia apreciable. Una repetición independiente de la misma prueba dio un promedio de 390 ms. Antes de desactivar la caché de *prompts* de Ollama, el TTFB promedio en las mismas condiciones era de 460 a 490 ms y el servicio llegó a detenerse por falta de memoria, lo que muestra la importancia de esa configuración. Con la versión anterior, que esperaba la respuesta completa del LLM remoto, el primer texto llegaba en promedio a los 4,7 s.

3.4.4 Resultados de Latencia de la Interacción por Voz

La Tabla 21 y la Figura 17 presentan las latencias de la interacción por voz medidas en modo local.

Tabla 21: Latencia por etapa de la interacción por voz en modo local (30 preguntas del dominio, en milisegundos).

Etapa	Media	<i>s</i>	Mediana	P95	Máx.	IC ₉₅
STT (Groq Whisper)	375	139	310	513	1.133	±43
RAG híbrido (local)	38	4	37	45	49	±2
TTFB (texto)	384	38	385	446	453	±14
Respuesta completa (RAG + LLM)	848	156	841	1.128	1.138	±56
TTS primer byte	642	79	621	811	812	±28
TTS audio completo	968	106	985	1.112	1.112	±38
Latencia percibida	1.869	208	1.839	2.151	2.565	±74

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/evaluar_latencia.py, ejecutado el 7 de octubre de 2026

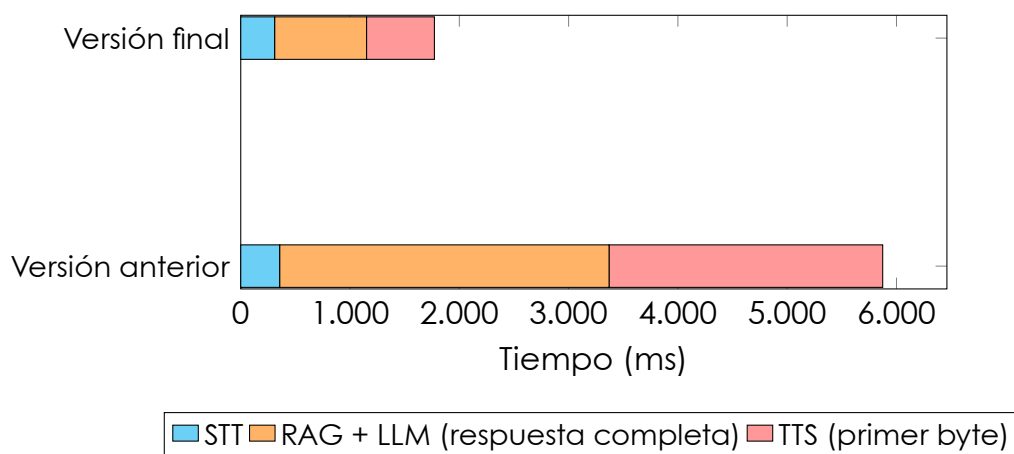


Figura 17: Composición de la latencia percibida por etapa: versión anterior y versión final (medianas).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/resultados/latencia_resumen.json y mediciones de la versión anterior

La latencia percibida tuvo una media de 1,87 s y una mediana de 1,84 s, frente a la mediana de 6,0 s de la versión anterior; 24 de las 30 consultas comenzaron a sonar en menos de 2 s y todas en menos de 3 s, con un P95 de 2,15 s, por lo que el requisito RNF-02 (P95 menor a 4 s) **se cumple**. La mejora proviene del LLM local, que eliminó la cola del servicio remoto (antes de 1,2 a 24,8 s por consulta, ahora de 0,6 a 1,1 s para la respuesta completa), y del modelo de síntesis Flash

v2.5, que redujo el primer byte de audio de 2,5 s a 0,6 s. En la versión final, la respuesta completa explica el 45 % de la latencia percibida, la síntesis de voz el 34 % y la transcripción el 20 %. La transcripción fue rápida y precisa: WER medio de 1,9 %, con 35 de 40 transcripciones exactas; los errores afectaron a términos como "Python" o "B1".

3.4.5 Resultados del Módulo de Visión

La Tabla 22 presenta el costo de la detección de presencia con BlazeFace y el tamaño del mensaje enviado al servidor, comparado con el del esquema anterior.

Tabla 22: Costo de la detección de presencia por fotograma (300 repeticiones).

Fotograma	Resolución	ms ($\bar{x} \pm s$)	Rostros	Cajas (B)	JPEG (B)
Con rostro	320×240	2,84 ± 0,45	1	30	13.407
Con rostro	640×480	2,92 ± 0,42	1	33	34.199
Sin rostro	320×240	2,74 ± 0,42	0	13	7.359
Sin rostro	640×480	2,77 ± 0,42	0	13	54.615

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/evaluar_vision.py (AMD Ryzen 7 7735HS, MediaPipe 0.10.14). B: bytes del mensaje con cajas delimitadoras y del JPEG en Base64 del esquema anterior

BlazeFace detectó el rostro de la imagen de prueba y no produjo falsas detecciones en el fotograma sin rostro, con un tiempo de inferencia de unos 3 ms por fotograma, prácticamente independiente de la resolución, frente a los 15 a 54 ms que requería el clasificador Haar Cascade en el servidor. A un fotograma por segundo, la detección ocupa menos del 0,3 % de un núcleo del equipo del usuario, y cada mensaje al servidor pasa de 7 a 55 KB (imagen JPEG) a menos de 40 bytes (coordenadas), es decir, una reducción de más de 400 veces en el tráfico del módulo de visión. Las pruebas de presencia obtuvieron el evento esperado en los 5 casos: la llegada se confirmó en el segundo fotograma, la salida en el cuarto fotograma sin rostro, un rostro aislado no generó evento y una ausencia de 3 fotogramas no se interpretó como salida. En seguridad, los 12 casos obtuvieron el resultado esperado y la contraseña con el esquema anterior se migró

a PBKDF2 tras el inicio de sesión.

3.4.6 Resultados de la Evaluación de Usabilidad

Participaron 16 estudiantes: 13 el 21 de julio de 2026, sobre la versión de la Fase 3, y 3 entre el 3 y el 4 de octubre de 2026, sobre la versión de la Fase 5. Por nivel, 8 cursaban cuarto semestre, 4 octavo, 3 tercero y 1 sexto (Figura 18). Todos reportaron un uso frecuente de herramientas de IA generativa (media de 4,75 sobre 5), mientras que su conocimiento en renderización 3D fue heterogéneo (media de 3,0). La Tabla 23 resume los ítems de evaluación del prototipo; el porcentaje de acuerdo corresponde a las respuestas “De acuerdo” y “Totalmente de acuerdo” (o 4 y 5 en el diferencial semántico), y el de desacuerdo, a “En desacuerdo” y “Totalmente en desacuerdo” (o 1 y 2).

Tabla 23: Resultados de la encuesta de usabilidad (n = 16).

Ítem	Media	Med.	% Acuer.	% Desac.
Las funciones del sistema están bien integradas	3,50	4	69	25
El sistema es innecesariamente complejo (ítem inverso)	2,75	2	44	56
La información es clara, precisa y fácil de entender	3,44	4	56	25
Mensajes de error claros ante fallos	3,19	3	44	31
Explicó su propósito y límites al inicio	3,38	4	56	25
Comprendió mis intenciones con precisión	3,38	4	56	25
Respuestas útiles y relevantes	3,56	4	69	25
Reconduce la conversación ante malentendidos	3,31	4	56	31
Antropomorfismo del avatar (1 a 5)	4,19	4	81	0
Animacidad: movimientos y sincronización labial (1 a 5)	4,06	4	69	0
Afinidad visual y empatía (1 a 5)	4,25	4,5	75	0
Inteligencia percibida (1 a 5)	4,19	4	81	0
Latencia aceptable	3,19	3,5	50	31
Uso de la cámara poco invasivo y confiable	3,38	4	56	31
Aumentaría mi eficiencia para encontrar información	3,25	4	62	31
Preferiría usar el asistente en el futuro	3,38	4	69	31

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Respuestas al cuestionario del Anexo B (21 de julio y 3 a 4 de octubre de 2026)

Los ítems del avatar obtuvieron las valoraciones más altas (69 % a 81 % de acuerdo, sin respuestas desfavorables). En los ítems funcionales y de aceptación, la mayoría expresó acuerdo en la integración de las funciones, la utilidad de las respuestas y la intención de uso futuro (62 % a 69 %), mientras que los aspectos peor valorados fue-

ron los mensajes de error (44% de acuerdo) y la latencia (50% de acuerdo y 31 % de desacuerdo), lo que coincide con las mediciones técnicas de las versiones evaluadas (mediana de 6,0 s hasta el inicio de la respuesta hablada en la versión de julio). Estos resultados motivaron las optimizaciones de la Fase 6, que redujeron esa mediana a 1,8 s; la percepción de los estudiantes sobre la versión final no se ha medido todavía. Se observó además que 4 de los 16 participantes, todos de la sesión de julio, marcaron "Totalmente en desacuerdo" en todos o casi todos los ítems Likert, incluido el ítem inverso sobre complejidad, mientras valoraban positivamente al avatar; este patrón de respuesta uniforme puede indicar una lectura incorrecta de la escala, por lo que los porcentajes deben interpretarse con cautela. Las sugerencias abiertas se concentraron en la interfaz (4 menciones), la velocidad de respuesta (3, una de ellas mediante caché), la ampliación de la base documental (3), la interpretación y calidad de las respuestas (2), la arquitectura (2), la fluidez y pronunciación de la voz (2) y la concurrencia (1); un participante indicó que el sistema estaba bien estructurado. Las Figuras 18 a 23 presentan la distribución de las respuestas.

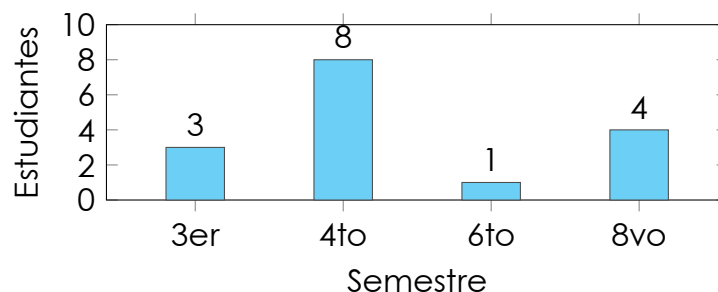


Figura 18: Nivel de avance de los estudiantes encuestados.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Respuestas al cuestionario del Anexo B ($n = 16$)

3.4.7 Comparación entre la Situación Inicial y la Propuesta

La Tabla 24 contrasta la situación inicial con los resultados medidos del prototipo y la Tabla 25, el cumplimiento de los requerimientos no funcionales.

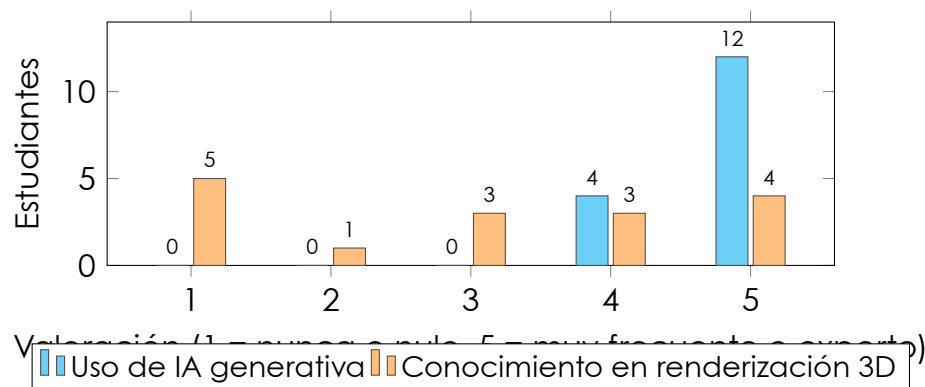


Figura 19: Uso previo de IA generativa y conocimiento técnico en renderización 3D.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Respuestas al cuestionario del Anexo B ($n = 16$)

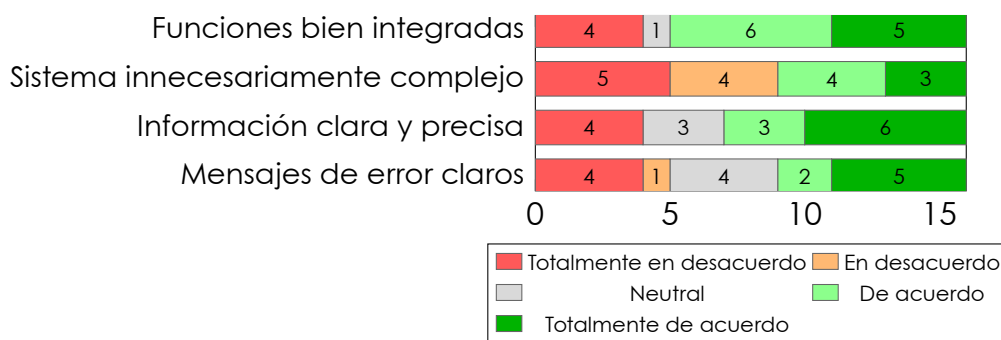


Figura 20: Usabilidad de la interfaz: integración, complejidad (ítem inverso), claridad y mensajes de error.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Respuestas al cuestionario del Anexo B ($n = 16$)

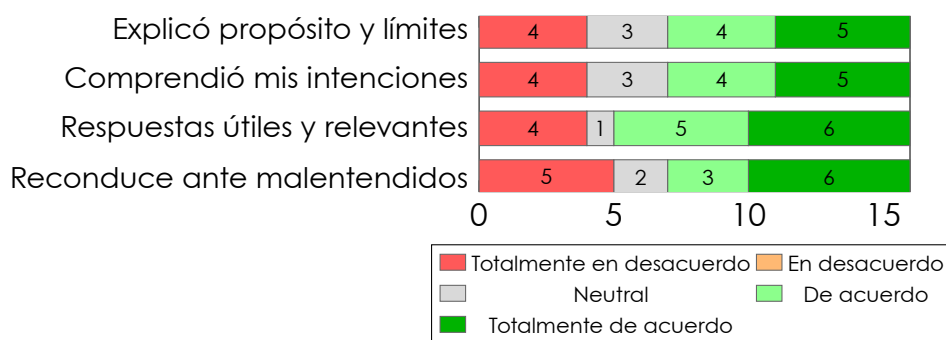


Figura 21: Inteligencia conversacional: propósito, comprensión, utilidad y reconducción.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Respuestas al cuestionario del Anexo B ($n = 16$)

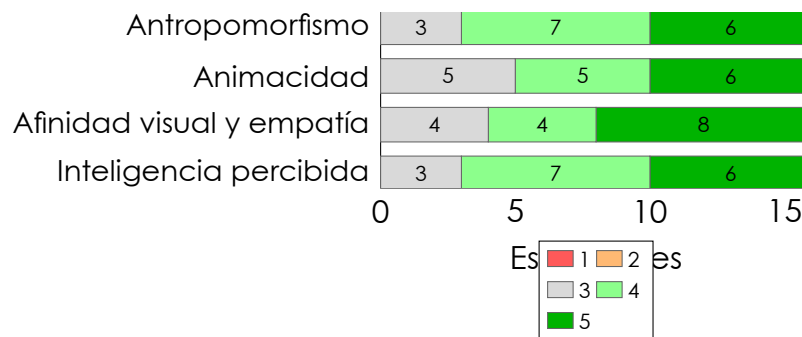


Figura 22: Percepción del avatar 3D en el diferencial semántico (1 a 5).

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Respuestas al cuestionario del Anexo B ($n = 16$)

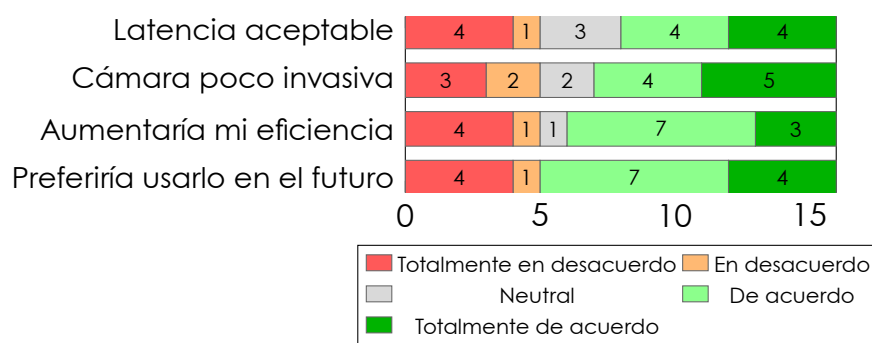


Figura 23: Aceptación tecnológica: latencia, uso de la cámara, eficiencia y preferencia futura.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Respuestas al cuestionario del Anexo B ($n = 16$)

Tabla 24: Matriz comparativa: situación inicial frente al prototipo.

Criterio	Situación inicial	Prototipo (valor medido)
Modo de interacción	Navegación manual por menús y PDF	Conversación por voz o texto con avatar 3D
Búsqueda	Palabras clave o lectura completa	Híbrida con reordenamiento: <i>Hit Rate@3</i> de 92,0 % (desarrollo) y 94,0 % (prueba)
Exactitud de la respuesta	Depende del personal o de la lectura del estudiante	90 % correctas, 4 % parciales, 2 % sin respuesta y 4 % incorrectas (banco de prueba)
Tiempo de respuesta	Sujeto al horario de atención (08h00 a 17h00)	TTFB promedio de 385 ms por WebSocket; mediana de 1,8 s hasta el inicio de la respuesta hablada (P95 de 2,2 s)
Disponibilidad	Lunes a viernes en horario de oficina	Mientras el servidor local y los servicios de voz estén disponibles
Carga operativa	76 h-hombre/mes estimadas	No medida; requiere un piloto con registro de consultas

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Diagnóstico (Capítulo II) y resultados de este capítulo

Tabla 25: Cumplimiento de los requerimientos no funcionales.

RNF	Meta	Resultado	Estado
RNF-01	Sin transmisión ni persistencia de imágenes	Solo se envían coordenadas (13 a 33 bytes)	Cumple
RNF-02	TTFB promedio ≤ 450 ms y latencia percibida P95 $< 4,0$ s	TTFB promedio de 385 ms; P95 percibido de 2,2 s	Cumple
RNF-03	100 % de rechazo fuera de dominio	10 de 10 rechazos	Cumple
RNF-04	Sin conexión con el SGA	Sin credenciales ni conexión	Cumple
RNF-05	Reindexación < 60 s	10,0 s	Cumple
RNF-06	Usabilidad aceptable	44 % a 69 % de acuerdo en ítems funcionales; 69 % a 81 % en el avatar (n = 16)	Parcial
RNF-07	Dos modos de despliegue	Modo local y VPS con Docker	Cumple
RNF-08	401/403 sin autorización	12 de 12 casos	Cumple

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Resultados de las pruebas de este capítulo

3.5 Cronograma y Presupuesto

3.5.1 Cronograma

El proyecto se ejecutó entre enero y octubre de 2026. La Tabla 26 presenta las fases y los meses en que se ejecutaron, contrastados con las fechas del historial de versiones del repositorio.

Tabla 26: Cronograma de ejecución del proyecto (2026).

Fase	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct
Diagnóstico, revisión documental y estado del arte	X	X	X	X						
Fase 1: Avatar 3D y motor RAG inicial					X					
Fase 2: Interfaz dual y panel RAG						X				
Fase 3: Visión, voz, autenticación y despliegue							X			
Encuesta de usabilidad (21 de julio y 3 a 4 de octubre)							X			X
Fase 4: Proveedor LLM, interfaz y manuales								X	X	
Fase 5: Seguridad, ajuste RAG y evaluación técnica										X
Fase 6: RAG híbrido, LLM local, <i>streaming</i> y visión en el cliente										X
Redacción del documento			X	X	X	X	X	X	X	X

Elaborado por: Paulo Emilio Espinoza Jiménez**Fuente:** Historial de versiones del repositorio

3.5.2 Presupuesto

No se adquirió hardware para el proyecto: el desarrollo y las pruebas se realizaron en el equipo personal del autor, cuya GPU ejecuta el LLM local, y el servidor del despliegue piloto y la GPU de LabITI fueron facilitados por la infraestructura del laboratorio. Los costos corresponden a los servicios de voz consumidos por API (Tabla 27).

Tabla 27: Presupuesto referencial de operación del prototipo.

Rubro	Descripción	Costo (USD)
Hardware	Equipo de desarrollo existente (no adquirido)	0,00
Servidor (VPS)	Infraestructura del laboratorio para el despliegue piloto	0,00 (facilitado)
TTS	ElevenLabs, plan Creator (100.000 caracteres/mes)	22,00 / mes
LLM	Nemotron 3 Nano local con Ollama (licencia abierta de NVIDIA), sin costo por consulta; no incluye el consumo eléctrico del equipo	0,00
STT	Groq Whisper large-v3, plan gratuito (USD 0,111 por hora de audio en el plan de pago)	0,00
Dominio y certificados	Subdominio institucional y Let's Encrypt	0,00
Total mensual		22,00 / mes

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Precios de lista de los proveedores a octubre de 2026 (referenciales)

El costo variable por consulta se estima con la Ecuación 12. Con una respuesta media de 129 caracteres en la versión final, el plan de ElevenLabs (22 USD por 100.000 caracteres) y unos 10 segundos de audio facturados por Groq en el plan de pago:

$$C_{\text{consulta}} = \underbrace{129 \times \frac{22}{100,000}}_{TTS} + \underbrace{\frac{10}{3,600} \times 0,111}_{STT} \approx 0,028 + 0,0003 \approx 0,03 \text{ USD} \quad (12)$$

Con la carga estimada de 304 consultas mensuales del diagnóstico, el costo de voz sería de unos 9 USD al mes, dentro de la cuota del plan; el LLM local no genera costo por consulta. La síntesis de voz es, por tanto, el rubro dominante del costo por consulta. Los valores son referenciales y dependen de las condiciones comerciales vigentes

de cada proveedor.

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

4.1 Conclusiones

A partir del desarrollo y la evaluación del Asistente Virtual Inteligente, se establecen las siguientes conclusiones, una por cada objetivo específico:

- **Objetivo 1 (análisis):** La revisión documental permitió consolidar en un único documento base la información de malla curricular, aranceles, pagos, becas, homologación, inglés y contacto de la carrera, y definir ocho requerimientos funcionales y ocho no funcionales trazables con el código. Al no existir registros institucionales de consultas, la carga operativa del proceso actual se estimó en 76 horas-hombre mensuales a partir de supuestos explícitos; esta cifra es una estimación y no una medición.
- **Objetivo 2 (diseño):** Se diseñó una arquitectura cliente-servidor modular e híbrida: el navegador realiza la detección de presencia por cajas delimitadoras y el renderizado 3D en WebGL; el servidor local, un monolito modular en FastAPI, orquesta el RAG y el LLM Nemotron servido por Ollama y se comunica con el cliente mediante WebSockets; y la transcripción y la síntesis de voz se consumen como servicios externos. El proveedor del LLM es intercambiable mediante configuración. El diseño admite dos modos de despliegue con el mismo código: el modo local, principal y utilizado en todas las pruebas, y un despliegue piloto en un servidor virtual con Docker, Coolify y un túnel hacia la GPU de LabITL.
- **Objetivo 3 (implementación):** Se implementó un prototipo funcional que integra un avatar VRM con sincronización labial por amplitud, transcripción con Whisper (WER medio de 1,9%), recuperación híbrida local con el modelo de *embeddings*

paraphrase-multilingual-MiniLM-L12-v2, BM25 y reordenamiento (29 ms por consulta), generación local con Nemotron 3 Nano entregada en *streaming* por WebSocket, síntesis de voz con ElevenLabs y detección de presencia con MediaPipe en el navegador (unos 3 ms por fotograma y mensajes de menos de 40 bytes). Durante el proceso se corrigieron diferencias entre el diseño y el código, como la métrica coseno de la base vectorial, la unificación de los parámetros del RAG y la autenticación de los *endpoints* de voz, diagnóstico y WebSocket, y se identificó y resolvió un consumo excesivo de memoria del servidor de inferencia.

- **Objetivo 4 (evaluación):** La evaluación reproducible permitió iterar el diseño según el ciclo DSRM. La primera medición evidenció las limitaciones de la versión inicial (*Hit Rate@3* de 62 % y mediana de 6,0 s hasta la respuesta hablada), y tras las optimizaciones el motor de recuperación alcanzó un *Hit Rate@3* de 92,0 % en el banco de desarrollo y de 94,0 % en un banco de prueba reservado; el 90 % de las respuestas del banco de prueba fueron correctas; el TTFB promedio por WebSocket fue de 385 ms, por debajo de la meta de 450 ms; y la latencia percibida por voz bajó a una mediana de 1,8 s con un P95 de 2,2 s, dentro de la meta de 4 s. La restricción de dominio funcionó en las 10 preguntas fuera de dominio y la seguridad de acceso en los 12 casos probados. La encuesta piloto con 16 estudiantes, aplicada sobre las versiones de julio y de inicios de octubre, valoró muy bien al avatar (69 % a 81 % de acuerdo, sin respuestas desfavorables) y de forma mayoritariamente positiva la integración, la utilidad de las respuestas y la intención de uso (62 % a 69 %), con la latencia y los mensajes de error como aspectos peor valorados. Por el tamaño de la muestra y su carácter exploratorio, estos resultados de usabilidad son evidencia inicial y no generalizable.

En conjunto, el prototipo demuestra la viabilidad técnica de integrar un avatar 3D, voz, visión y RAG en una aplicación web para orientar a los estudiantes, con una arquitectura que mantiene en el equipo local la recuperación, el LLM y las imágenes de la cámara. Aunque los resultados técnicos cumplen las metas planteadas, antes

de un despliegue institucional se requiere validar la versión final con usuarios, ampliar la base documental y monitorear las respuestas incorrectas que aún se producen.

4.2 Recomendaciones

Con base en las limitaciones demostradas en el Capítulo III, se presentan las siguientes recomendaciones:

- **Validar la versión final con usuarios:** Repetir la encuesta de usabilidad sobre la versión con LLM local y respuesta en *streaming*, para comprobar si la reducción de la latencia mejora la percepción del aspecto peor valorado.
- **Mantener la evaluación de la recuperación:** Cada vez que se actualice la base documental, ejecutar los bancos de desarrollo y de prueba del repositorio y ampliarlos con preguntas sobre los nuevos documentos, de modo que el *Hit Rate* se mantenga como indicador de calidad.
- **Reducir las respuestas incorrectas:** Las respuestas incorrectas restantes provienen de fallos de recuperación y de confusiones entre datos cercanos. Se recomienda pedir al modelo que cite el fragmento usado, probar modelos locales de mayor tamaño si se dispone de más memoria de GPU y ampliar la evaluación manual con un segundo evaluador para medir la concordancia.
- **Reducir la latencia de la voz:** La síntesis de voz aporta ahora el 34% de la latencia percibida. Se recomienda enviar el texto al TTS por oraciones a medida que llega por el WebSocket, de modo que el avatar comience a hablar antes de que termine la generación, y evaluar un servicio de transcripción local.
- **Configurar el servidor de inferencia:** En cualquier despliegue con Ollama, desactivar la caché de *prompts* en RAM (`LLAMA_ARG_CACHE_RAM=0`) o limitarla según la memoria disponible, ya que con la configuración por defecto el servicio puede detenerse por falta de memoria.

- **Fortalecer la seguridad del despliegue:** Antes de cualquier uso con estudiantes en el servidor público, definir contraseñas propias mediante las variables `INITIAL_USER_PASSWORD` e `INITIAL_ADMIN_PASSWORD`, cambiar las cuentas existentes y asignar los roles mediante una columna en la base de datos en lugar de un nombre de usuario fijo.
- **Validar el impacto operativo:** La reducción de la carga administrativa no pudo medirse. Se recomienda un piloto de al menos un periodo académico que registre el número de consultas resueltas por el asistente y las atendidas por secretaría, para contrastar la estimación de 76 horas-hombre mensuales.
- **Control de costos operativos:** La síntesis de voz representa casi todo el costo variable estimado (0,03 USD por consulta), ya que el LLM local no tiene costo por uso. Se recomienda monitorear el consumo de caracteres y ofrecer respuestas solo en texto cuando el usuario no utilice el modo conversacional.

4.3 Trabajos Futuros

El prototipo actual sienta una base tecnológica sobre la cual pueden derivarse las siguientes líneas de investigación o ampliación:

- **Integración con el SGA Institucional:** Integrar la sesión autenticada del estudiante con las bases de datos académicas mediante APIs RESTful permitiría respuestas personalizadas, por ejemplo sobre el avance en la malla o los pagos pendientes, con los controles de privacidad correspondientes.
- **Interacción manos libres:** Incorporar detección de actividad de voz (VAD) y de fin de habla para eliminar la necesidad de pulsar el micrófono, aprovechando la detección de presencia ya implementada para activar la escucha.
- **Generación de Emociones Dinámicas (*Affective Computing*):** Clasificar el tono de cada respuesta del LLM para activar expresiones del avatar acordes al contenido y evaluar su efecto en la percepción de empatía.

- **Evaluación con una muestra representativa:** Repetir la encuesta con una muestra probabilística de estudiantes de ambas sedes e instrumentos validados (por ejemplo, la escala SUS), y analizar el patrón de respuestas uniformes observado en el piloto.

REFERENCIAS

- [1] CENIA y CEPAL, "Índice latinoamericano de inteligencia artificial 2024-2025," Centro Nacional de Inteligencia Artificial (CENIA) / CEPAL, Tech. Rep., 2024.
- [2] WiseGuyReports, "Inteligencia artificial en el mercado de educación superior: Proyecciones y tendencias 2025–2035," WiseGuyReports, Tech. Rep., 2025.
- [3] UNESCO IESALC, "Configurando el futuro: El impacto de la educación superior en las políticas nacionales de IA," Instituto Internacional de la UNESCO para la Educación Superior en América Latina y el Caribe, Tech. Rep., 2024.
- [4] PUCE, "Aurel_ai y legalbot-ec: Modelos de inteligencia artificial locales y automatización en la educación superior," Pontificia Universidad Católica del Ecuador, Tech. Rep., 2024.
- [5] Universidad Tecnológica Indoamérica, "Informe de rendición de cuentas institucional y demografía estudiantil," Universidad Tecnológica Indoamérica (UTI), Tech. Rep., 2025.
- [6] UTI y CEDIA, "Modelamiento de procesos biológicos mediante el procesamiento de imágenes médicas con visión artificial e inteligencia artificial," *Boletín Científico Indoamérica*, vol. 12, no. 1, pp. 45–58, 2025.
- [7] M. Salazar and E. Morales, "Implementación de Unreal Engine 5 para mejorar el pensamiento crítico en los estudiantes de ingeniería en tecnologías de la información," Master's thesis, Universidad Tecnológica Indoamérica, 2024.
- [8] J. Carrasco and R. Vélez, "Implementación de la inteligencia artificial para el análisis de sentimientos de la percepción pública

mediante procesamiento de lenguaje natural," Master's thesis, Universidad Tecnológica Indoamérica, 2024.

- [9] S. A. Robinson, "Ai-based avatars are changing the way we learn and teach: benefits and challenges," *Frontiers in Education*, vol. 9, 2024.
- [10] A. F. Pabón M., K. J. Barrios Q., S. D. Solano C., and C. G. Quintero M., "Naia: A robust artificial intelligence framework for multi-role virtual academic assistance," *Systems*, vol. 13, no. 12, p. 1091, 2025.
- [11] Y. Wang and X. Chen, "Lightweight gaze control system for embodied conversational agents in public spaces," *Electronics*, vol. 14, no. 8, p. 1592, 2024.
- [12] A. Abedi and S. S. Khan, "Engagement measurement based on facial landmarks and spatial-temporal graph convolutional networks," *arXiv preprint arXiv:2403.17175*, 2024.
- [13] A. S. Cáceres Vega and C. P. Sarango Lapo, "Asistente virtual desplegado de manera holográfica," Master's thesis, Universidad Técnica Particular de Loja, 2024.
- [14] J. M. Guncay Carchipulla and W. R. Sinchi Morocho, "Desarrollo de un asistente virtual utilizando procesamiento de lenguaje natural y aprendizaje de máquina para comunicación con aspirantes a estudios de posgrado," Master's thesis, Universidad Politécnica Salesiana, 2022.
- [15] Pontificia Universidad Católica del Ecuador, "Integración de ia en la mesa de servicios tecnológicos de telefónica ecuador," Master's thesis, PUCE, 2024.
- [16] G. A. Díaz Pérez and J. R. Moreno Charco, "Asistente virtual basado en inteligencia artificial para la obtención de indicadores de desempeño," 2025.
- [17] C. G. Quintero M. *et al.*, "Everydai: A virtual assistant for daily contextual decision-making," *Systems*, vol. 13, no. 9, p. 753, 2025.

- [18] D. M. Laureano, D. L. Cruz-Romero, and C. Ovalle, "Asistente virtual basado en inteligencia artificial como herramienta de tesis para estudiantes universitarios de la carrera de ingeniería," in *20th LACCEI International Multi-Conference for Engineering, Education, and Technology*. LACCEI, 2022.
- [19] C. Fernández, M. A. Vicente, M. C. Carratalá, and S. Lorenzo, "Virtual 3d avatars as a training tool in university degrees," *International Journal of Advances in Electronics and Computer Science*, vol. 9, no. 12, 2022.
- [20] B. Kiran, "Talkmateai: A real-time, multimodal ai companion architecture," pp. 112–128, 2023.
- [21] B. A. Karagond, D. V. Shruthi, H. A. Bhat, H. M. Samarth, and D. Rahul, "Ai desktop assistant," *World Journal of Engineering Research and Technology*, vol. 10, no. 4, pp. 230–238, 2024.
- [22] L. Cárdenas, A. Pipino, M. T. Donadio, and P. Picota, "Modelo de un sistema de alerta basado en visión artificial para la prevención de aglomeraciones en el transporte público," *Visión Antartica*, vol. 8, no. 2, pp. 93–107, 2024.
- [23] Chroma, "Chroma: the AI-native open-source embedding database," <https://docs.trychroma.com>, 2024, accedido: 7 de octubre de 2026.
- [24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3982–3992.
- [25] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [26] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

- [27] L. Bonifacio, V. Jeronymo, H. Q. Abonizio, I. Campiotti, M. Fadaee, R. Lotufo, and R. Nogueira, "mMARCO: A multilingual version of the MS MARCO passage ranking dataset," *arXiv preprint arXiv:2108.13897*, 2021.
- [28] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1. IEEE, 2001, pp. 1–511–1–518.
- [29] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, and M. Grundmann, "BlazeFace: Sub-millisecond neural face detection on mobile GPUs," *arXiv preprint arXiv:1907.05047*, 2019.
- [30] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
- [31] pixiv Inc., "three-vmr: Use VRM on three.js," <https://github.com/pixiv/three-vmr>, 2024, accessed: 7 de octubre de 2026.
- [32] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989.
- [33] C. Bartneck, D. Kulić, E. Croft, and S. Zoghbi, "Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots," *International Journal of Social Robotics*, vol. 1, no. 1, pp. 71–81, 2009.
- [34] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [35] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

- [36] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 28 492–28 518.
- [37] S. Ramírez, "FastAPI framework, high performance, easy to learn, fast to code, ready for production," <https://fastapi.tiangolo.com>, 2024, accedido: 7 de octubre de 2026.
- [38] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [39] NVIDIA Corporation, "Nemotron 3 Nano (4b) en la biblioteca de modelos de Ollama," <https://ollama.com/library/nemotron-3-nano>, 2025, licencia NVIDIA Open Model License. Accedido: 7 de octubre de 2026.
- [40] Ollama, "Ollama: ejecución local de modelos de lenguaje," <https://ollama.com>, 2025, versión 0.34.4. Accedido: 7 de octubre de 2026.
- [41] ElevenLabs, "ElevenLabs text to speech API: Eleven flash v2.5 y eleven multilingual v2," <https://elevenlabs.io/docs>, 2025, accedido: 7 de octubre de 2026.
- [42] Coollabs, "Coolify: An open-source and self-hostable alternative to heroku, netlify and vercel," <https://coolify.io/docs>, 2025, accedido: 7 de octubre de 2026.
- [43] NVIDIA Corporation, "NVIDIA API catalog: Llama 3.2 11b vision instruct," <https://build.nvidia.com/meta/llama-3.2-11b-vision-instruct>, 2025, accedido: 7 de octubre de 2026.
- [44] E. M. Voorhees, "The TREC-8 question answering track report," in *Proceedings of the Eighth Text REtrieval Conference (TREC-8)*. NIST, 1999, pp. 77–82.

GLOSARIO

Definición de los términos técnicos y acrónimos relevantes utilizados a lo largo del documento:

API (Application Programming Interface): Interfaz que permite la comunicación entre componentes de software; en este proyecto, conecta el backend con los servicios de transcripción, generación y síntesis de voz.

BlazeFace: Detector facial neuronal ligero diseñado para ejecutarse en tiempo real en dispositivos de consumo; devuelve la caja delimitadora y puntos clave de cada rostro, sin identificar a la persona.

BM25: Función de puntuación de búsqueda léxica que ordena los documentos según la frecuencia de los términos de la consulta y su rareza en el corpus.

Bounding box (caja delimitadora): Rectángulo definido por su posición, ancho y alto que encierra un objeto detectado en una imagen; en el prototipo, el rostro del usuario.

Chunking: Fraccionamiento de un documento en fragmentos para su indexación (RAG), o transmisión de audio por partes (*streaming*).

Coolify: Plataforma de código abierto para desplegar y administrar aplicaciones en contenedores en un servidor propio.

Cosine Similarity (Similitud Coseno): Métrica que mide la semejanza entre dos vectores por el ángulo que forman; se usa para comparar la consulta con los fragmentos del documento.

Cross-encoder: Modelo que procesa conjuntamente la consulta y un fragmento candidato para estimar su relevancia; se usa para reordenar los resultados de la búsqueda (*re-ranking*).

Docker: Tecnología de contenedores que empaqueta una aplicación con sus dependencias para ejecutarla de forma idéntica en distintos servidores.

DSRM (Design Science Research Methodology): Metodología de investigación orientada a diseñar, construir y evaluar artefactos que resuelven un problema.

ECA (Embodied Conversational Agent): Agente conversacional que cuenta con una representación corporal visible, como un avatar 3D.

Embedding: Representación numérica (vector) del significado de un texto, utilizada para la búsqueda semántica.

FastAPI: Marco de trabajo (*framework*) de alto rendimiento para construir APIs con Python.

Haar Cascade: Clasificador de objetos propuesto por Viola y Jones basado en características rectangulares evaluadas en cascada; utilizado en la primera versión del módulo de visión.

Hit Rate@k: Porcentaje de consultas en las que al menos un fragmento relevante aparece entre los k primeros resultados de la búsqueda.

Jailbreak: Intento de manipular a un modelo de lenguaje para que ignore sus instrucciones o cambie su comportamiento.

LLM (Large Language Model): Modelo de Inteligencia Artificial entrenado con grandes volúmenes de texto, capaz de comprender y generar lenguaje natural.

MediaPipe: *Framework* de Google para construir procesos de percepción (visión y audio) que pueden ejecutarse en el navegador mediante WebAssembly y WebGL.

MRR (Mean Reciprocal Rank): Promedio del inverso de la posición del primer resultado relevante en un conjunto de consultas.

Nemotron: Familia de modelos de lenguaje de NVIDIA con licencia abierta; el prototipo usa Nemotron 3 Nano de 4.000 millones de parámetros ejecutado localmente.

OCR (Optical Character Recognition): Tecnología utilizada para extraer texto desde imágenes o documentos escaneados.

Ollama: Servidor de código abierto para descargar y ejecutar modelos de lenguaje de forma local mediante una API HTTP.

PBKDF2: Función de derivación de claves que aplica un *hash* criptográfico de forma iterada con un *salt* para almacenar contraseñas de forma segura.

PLN (Procesamiento de Lenguaje Natural): Rama de la inteligencia artificial enfocada en la interacción entre las computadoras y el lenguaje humano.

Push-to-talk: Modo de captura de voz en el que el usuario pulsa un botón para iniciar y detener la grabación.

RAG (Retrieval-Augmented Generation): Arquitectura que mejora las respuestas de un LLM recuperando previamente información relevante de una base de conocimiento específica.

RRF (Reciprocal Rank Fusion): Técnica que combina varias listas de resultados sumando, para cada elemento, el inverso de su posición en cada lista.

STT (Speech-to-Text): Tecnología que convierte la voz en texto.

Streaming: Transmisión de una respuesta por partes a medida que se genera, en lugar de esperar a que esté completa.

Three.js: Biblioteca de JavaScript para crear y visualizar gráficos 3D en el navegador mediante WebGL.

TTFB (Time to First Byte): Tiempo transcurrido desde que se envía una petición hasta que se recibe el primer byte de la respuesta; en el prototipo, hasta el primer fragmento de texto recibido por WebSocket.

TTS (Text-to-Speech): Tecnología que convierte texto escrito en audio sintetizado.

VPS (Virtual Private Server): Servidor virtual con recursos dedicados en el que se ejecuta el despliegue piloto del prototipo.

VRM: Formato abierto basado en glTF para avatares 3D humanoides, con expresiones faciales y esqueleto estandarizados.

WebSockets: Protocolo de comunicación bidireccional en tiempo real sobre una única conexión TCP.

WER (Word Error Rate): Proporción de palabras sustituidas, omitidas o insertadas por un sistema de transcripción respecto al texto de referencia.

Whisper: Modelo de reconocimiento de voz de código abierto desarrollado por OpenAI.

ANEXO A

REPOSITORIO DE CÓDIGO Y CONFIGURACIONES TÉCNICAS

Este anexo detalla los enlaces a los repositorios donde se encuentra el código fuente del Asistente Virtual Inteligente, así como la configuración base de las variables de entorno para su funcionamiento.

Repositorios de versionamiento

- **Asistente Virtual (código fuente, manuales y *scripts* de evaluación):** https://github.com/EmilioEJ/Virtual_Assistant
- **Documento de Tesis (LaTeX):** https://github.com/EmilioEJ/TFG_EmilioESPINOZA

Configuración base (archivo `.env`)

El archivo `.env` no se publica en el repositorio porque contiene claves de acceso; el repositorio incluye la plantilla `.env.example`, de la que se presenta un extracto:

```
# Proveedor de IA (ollama | nvidia | groq | openwebui |  
#                               siliconflow | gemini)  
AI_PROVIDER=ollama  
  
# LLM local (Nemotron servido por Ollama)  
OLLAMA_BASE_URL=http://localhost:11434  
OLLAMA_MODEL_NAME=nemotron-3-nano:4b  
OLLAMA_KEEP_ALIVE=60m  
  
# Proveedores alternativos compatibles con OpenAI  
NVIDIA_API_KEY=your_nvidia_api_key_here  
NVIDIA_MODEL_NAME=meta/llama-3.2-11b-vision-instruct  
  
# Transcripción (Groq Whisper) y síntesis de voz (ElevenLabs)  
GROQ_API_KEY=your_groq_key_here
```

```
ELEVENLABS_API_KEY=your_elevenlabs_key_here
```

```
# Contraseñas de las cuentas iniciales (solo al crear users.db)
```

```
INITIAL_USER_PASSWORD=cambia_esta_contrasena
```

```
INITIAL_ADMIN_PASSWORD=cambia_esta_contrasena_admin
```

Ejecución

- **Modo local:** instalar Ollama, descargar el modelo (ollama pull nemotron-3-nano:4b), añadir al servicio de Ollama las variables LLAMA_ARG_CACHE_RAM=0 y LLAMA_ARG_CTX_CHECKPOINTS=0 y ejecutar iniciar_local.sh; el servidor queda disponible en localhost:8000.
- **Modo servidor:** docker compose up -d con Coolify y Traefik.
- **Reconstrucción del índice:** .venv/bin/python build_rag_index.py.

ANEXO B

INSTRUMENTO DE EVALUACIÓN DE USABILIDAD

Para la evaluación del sistema se utilizó un cuestionario en Google Forms aplicado a 16 estudiantes de la carrera: 13 el 21 de julio de 2026 y 3 entre el 3 y el 4 de octubre de 2026. A continuación se presenta la estructura de las preguntas formuladas; las escalas Likert van de "Totalmente en desacuerdo" a "Totalmente de acuerdo".

■ Sección 1: Perfil Técnico del Evaluador

1. ¿Cuál es tu nivel de avance actual dentro de la carrera de TI?
2. En los últimos 6 meses, ¿con qué frecuencia has interactuado con modelos de IA generativa para actividades académicas? (Escala 1-5)
3. ¿Cuál es tu nivel de conocimiento técnico en renderización gráfica 3D o desarrollo interactivo? (Escala 1-5)

■ Sección 2: Usabilidad de la Interfaz y Arquitectura

4. Considero que las distintas funciones del sistema están bien integradas.
5. Encontré que el sistema es innecesariamente complejo de utilizar.
6. La información académica que provee el asistente es clara, precisa y fácil de entender.
7. El sistema muestra mensajes de error claros o me guía adecuadamente ante fallos.

■ Sección 3: Inteligencia Conversacional y NLP

8. Al inicio, el asistente explicó claramente su propósito y límites.

9. El asistente virtual comprendió mis intenciones y consultas técnicas con precisión.
10. Las respuestas de la IA fueron útiles y directamente relevantes.
11. Cuando la IA no entendió algo, fue capaz de reconducir la conversación amigablemente.

■ **Sección 4: Evaluación de la Interfaz del Avatar 3D**

12. Antropomorfismo. (1: Artificial - 5: Natural)
13. Animacidad. (1: Estático - 5: Vivo)
14. Afinidad visual y empatía. (1: Desagradable - 5: Agradable)
15. Inteligencia percibida. (1: Incompetente - 5: Competente)

■ **Sección 5: Visión Artificial y Aceptación Tecnológica**

16. Percibí una latencia aceptable entre la captura y la respuesta renderizada.
17. El uso de la cámara me pareció poco invasivo y confío en el manejo de privacidad.
18. Utilizar este asistente aumentaría mi eficiencia para encontrar información.
19. Basado en esta prueba, preferiría utilizar este asistente en el futuro.

■ **Sección 6: Diagnóstico Técnico**

20. ¿Qué mejora técnica priorizarías antes de lanzar este asistente a toda la universidad?

ANEXO C

FORMULARIO DE CONSENTIMIENTO INFORMADO

El siguiente formulario fue presentado a todos los participantes antes de la evaluación, indicando explícitamente el uso de la cámara web sin almacenamiento, el anonimato de los datos y el carácter voluntario de la prueba.

UNIVERSIDAD TECNOLÓGICA INDOAMÉRICA

Consentimiento Informado

Estudio de evaluación del asistente CampusBot - Trámite de homologación

Yo, _____, con cédula _____, declaro que participo de forma **voluntaria** en este estudio y doy mi consentimiento para que los datos que proporcione se utilicen de manera **anónima**, con fines exclusivamente de **investigación** y para la obtención de **resultados estadísticos**. Entiendo que mi participación es voluntaria y que mis datos no se usarán para ningún otro fin.

Firma

Fecha (día / mes / año)

Gracias por su participación. Universidad Tecnológica Indoamérica - Facultad de Ingeniería.

Imagen 5: Formulario de consentimiento informado.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Google Forms

ANEXO D

APROBACIÓN DE ABSTRACT

A continuación se adjunta la evidencia de la aprobación del resumen ejecutivo en su versión en inglés (Abstract).

UNIVERSIDAD TECNOLÓGICA INDOAMÉRICA

FACULTY OF ENGINEERING

Information Technology

AUTHOR: ESPINOZA JIMENEZ PAULO EMILIO

TUTOR: MSc. IZA LLUMIGUSIN CHRISTIAN EDUARDO

THEME

Development of an Intelligent Virtual Assistant Prototype with a 3D Avatar Interface and Computer Vision for the Dissemination of Academic Information for Students in an Educational Institution

ABSTRACT

This Curricular Integration Project addresses the limitations of current document repositories and institutional interfaces in disseminating academic information, which lack semantic search capabilities and proactive interactivity, leading to delays in routine student queries and administrative overload in administrative offices. Aiming to develop a prototype of an intelligent virtual assistant with a 3D avatar interface and computer vision for the dissemination of academic regulations and curricula, the Design Science Research Methodology (DSRM) was adopted in combination with the agile Scrum framework for iterative software development. The proposed hybrid architecture implements visual capture via bounding boxes and 3D graphic rendering using WebGL locally on the client device, while a local server orchestrates a Retrieval-Augmented Generation (RAG) engine utilizing the multilingual embedding model alongside the local Nemotron model. The main results demonstrate high semantic accuracy, achieving a Hit Rate of 92.0% in academic regulation retrieval, an average Time to First Byte (TTFB) response time of 450 ms ensuring conversational fluency via WebSockets, and a positive usability evaluation with 13 students regarding integration, accuracy, and usage preference. In conclusion, the decentralized architecture and the use of generative artificial intelligence provide a viable, robust solution that transforms traditional, static queries into an interactive, accessible, and efficient support channel for the university community.

KEYWORDS: Keywords: Generative artificial intelligence, Multimodal interactions, Virtual assistant, 3D rendering



Imagen 6: Aprobación del Abstract.

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: Documento de aprobación del Abstract

ANEXO E

MANUALES DEL SISTEMA

Para facilitar la adopción, despliegue y mantenimiento del Asistente Virtual, se elaboraron tres manuales que documentan el ciclo de vida del software. Los manuales se encuentran versionados en el repositorio oficial del proyecto:

- **Manual de Implementación:** Guía para configurar el entorno de ejecución, instalar dependencias, configurar Docker y levantar los servicios.

https://github.com/EmilioEJ/Virtual_Assistant/blob/main/Manuales/Implementacion/Manual_Implementacion.pdf

- **Manual Técnico:** Documentación para desarrolladores con la arquitectura del sistema, el diseño del motor RAG, la estructura de la base de datos ChromaDB y la integración de APIs.

https://github.com/EmilioEJ/Virtual_Assistant/blob/main/Manuales/Tecnico/Manual_Tecnico.pdf

- **Manual de Usuario:** Guía ilustrada para el usuario final sobre el uso de la interfaz gráfica, la voz y la navegación por el sistema.

https://github.com/EmilioEJ/Virtual_Assistant/blob/main/Manuales/Usuario/Manual_Usuario.pdf

ANEXO F

BANCO DE PREGUNTAS Y EVIDENCIAS DE EVALUACIÓN

La Tabla 28 presenta el banco de desarrollo de 50 preguntas, con la frase de referencia (*gold standard*) del documento fuente, la posición del primer fragmento relevante entregado por la configuración final (“–” si no aparece entre los 5 primeros) y, para las 30 preguntas usadas en la evaluación de exactitud, la clasificación de la respuesta generada por la versión final. La Tabla 29 presenta, con la misma estructura, el banco de prueba reservado. Los archivos completos, incluidas las respuestas y las latencias de cada consulta, se encuentran en la carpeta *evaluacion/resultados/* del repositorio.

Tabla 28: Banco de preguntas, posición recuperada y clasificación de la respuesta.

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
Q01	Malla curricular	¿Cuántos semestres dura la carrera de TI?	ocho niveles académicos	1	Correcta
Q02	Malla curricular	¿Cuántos créditos tiene la carrera?	120 créditos	1	Correcta
Q03	Malla curricular	¿Cuántas materias tengo que aprobar en total?	40 asignaturas	1	Correcta
Q04	Malla curricular	¿Qué materias se ven en primer semestre?	Computación y Sociedad, Lenguaje y Comunicación	1	Correcta
Q05	Malla curricular	¿Qué materias tiene el 2do semestre?	Cálculo Diferencial, Matemáticas Discretas	5	Sin respuesta
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
Q06	Malla curricular	¿En qué semestre se ve Programación Orientada a Objetos?	Abarca Cálculo Integral, Estadística	1	Correcta
Q07	Malla curricular	¿Qué materias hay en cuarto nivel?	Desarrollo de Aplicaciones Web, Fundamentos de las Bases de Datos	3	Correcta
Q08	Malla curricular	¿En qué semestre veo aplicaciones móviles?	Desarrollo de Aplicaciones Móviles	1	Correcta
Q09	Malla curricular	¿Cuándo se ve computación en la nube?	Computación en la Nube	1	Correcta
Q10	Malla curricular	¿Qué materias tiene el 8vo semestre?	Auditoría de TI, Cyberseguridad	3	Correcta
Q11	Malla curricular	¿Cómo es la secuencia de las materias de programación?	inicia con Programación Básica	1	Correcta
Q12	Prácticas	¿En qué semestre se hacen las prácticas comunitarias?	Prácticas de Servicio Comunitario	1	Correcta
Q13	Prácticas	¿Cuándo hago las prácticas preprofesionales?	Prácticas Pre-profesionales	1	Incorrecta
Q14	Perfil de egreso	¿En qué puedo trabajar cuando me gradúe?	Desarrollador de Software _Full-Stack_	–	Parcial
Q15	Perfil de egreso	¿Cuáles son las competencias del perfil de egreso?	cuatro macro-dimensiones	1	Incorrecta
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
Q16	Titulación	¿Qué requisitos necesito para graduarme además de aprobar las materias?	no resulta suficiente para la titulación	3	Correcta
Q17	Inglés	¿Qué nivel de inglés necesito para graduarme?	nivel B1	2	Correcta
Q18	Inglés	¿Cómo se calcula la nota del examen de suficiencia de inglés?	60% de la calificación final	3	Parcial
Q19	Inglés	¿Cuál es la nota mínima para obtener el certificado B1?	siete sobre diez	2	Correcta
Q20	Inglés	¿En cuánto tiempo me entregan el certificado de inglés?	30 días posteriores	1	Sin respuesta
Q21	Aranceles	¿Cuánto es el pago inicial de la matrícula en Quito presencial?	\$ 700 USD	2	Correcta
Q22	Aranceles	¿Cuánto cuesta la matrícula en modalidad en línea?	\$ 300 USD	1	Correcta
Q23	Aranceles	¿Cuáles son las fechas de los pagos diferidos?	15 de mayo de 2026	1	Correcta
Q24	Aranceles	¿Hay descuento si pago todo de contado?	descuento directo del 5 %	3	Correcta
Q25	Aranceles	¿El descuento por pronto pago se acumula con la beca?	no es acumulable	1	Correcta
Q26	Aranceles	¿Qué pasa si pagué de más?	saldo a favor	1	Correcta
Q27	Pagos	¿En qué bancos puedo pagar la colegiatura?	Banco Pichincha, Banco del Pacífico, Banco de Guayaquil o Produbanco	2	Correcta
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
Q28	Pagos	¿Cuál es el código para pagar en Mi Vecino?	código de recaudación 7129	1	Correcta
Q29	Pagos	¿Puedo pagar con tarjeta de crédito?	Datafast o Payphone	3	Correcta
Q30	Pagos	¿Cómo subo el comprobante de pago al SGA?	INGRESO DE COMPROBANTES	2	Correcta
Q31	Pagos	¿Cómo solicito el diferimiento del pago?	solicitud de prórroga	2	–
Q32	Becas	¿Qué becas ofrece la universidad?	Mérito y Rendimiento Académico	2	–
Q33	Becas	¿Hay becas para deportistas?	Mérito y Rendimiento Deportivo	1	–
Q34	Becas	¿Cómo postulo a una beca?	Módulo de Becas en el SGA	1	–
Q35	Becas	¿Existen ayudas para estudiantes en situación vulnerable?	situación vulnerable	1	–
Q36	Homologación	Vengo de otra universidad, ¿puedo homologar mis materias?	Análisis Comparativo de Contenidos	5	–
Q37	Homologación	¿Qué porcentaje de similitud necesita una materia para homologarse?	80%	1	–
Q38	Homologación	¿Qué documentos necesito para homologar?	récord académico original	–	–
Q39	Homologación	¿Puedo validar mis conocimientos con un examen?	Validación de Conocimientos	1	–
Q40	Homologación	¿Puedo homologar materias por experiencia laboral?	diez (10) años de experiencia profesional	2	–
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
Q41	Homologación	¿Cómo me exonero del inglés si ya tengo certificado?	Exoneración Inmediata mediante Certificaciones Internacionales	1	–
Q42	Contacto	¿Cuál es el número de WhatsApp de la universidad?	963 088 088	1	–
Q43	Contacto	¿Dónde queda la sede de Quito?	Machala y Sabanilla	1	–
Q44	Contacto	¿Cuál es el horario de atención de la sede?	08h00 a 17h00	1	–
Q45	Contacto	¿Cuál es el correo para homologaciones?	homologacion@uti.edu.ec	1	–
Q46	Admisiones	¿Qué documentos necesito para inscribirme como estudiante nuevo?	certificado de votación	1	–
Q47	Carrera	¿A qué facultad pertenece la carrera de TI?	Facultad de Ingeniería, Industria y Producción	1	–
Q48	Malla curricular	¿Qué materias de ciberseguridad hay en la carrera?	Seguridad en Redes	2	–
Q49	Malla curricular	¿En qué semestre se ve ciencia de datos?	Ciencia de Datos	2	–
Q50	Malla curricular	¿Cuántas horas de trabajo tiene la carrera en total?	5.776 horas	1	–

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/banco_preguntas.json y evaluacion/resultados/

Tabla 29: Banco de prueba reservado, posición recuperada y clasificación de la respuesta.

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
T01	Carrera	¿Con qué resolución fue aprobada la carrera de TI?	RPC-SE-33-No.101-2021	1	Correcta
T02	Contacto	¿A qué hora atiende la sede de Quito?	08h00 a 17h00	1	Correcta
T03	Contacto	¿Cómo se llama el campus de Quito?	Campus Eugenio Espejo	1	Correcta
T04	Malla curricular	¿Cuántas horas de clase al día hay en la modalidad presencial?	5 horas diarias	1	Correcta
T05	Malla curricular	¿Para qué sirve Matemáticas Discretas en la carrera?	teoría de grafos	1	Correcta
T06	Malla curricular	¿En qué nivel se ve Interacción Hombre Máquina?	Abarca Cálculo Integral, Estadística	1	Correcta
T07	Malla curricular	¿Qué materia de primero trata el impacto ético de la tecnología?	ancla la tecnología a su impacto ético	1	Correcta
T08	Malla curricular	¿De qué se trata la materia Gobierno de TI?	alinea la tecnología con los objetivos del negocio	1	Correcta
T09	Malla curricular	¿Qué materias tiene el quinto semestre?	Incluye Metodología de Investigación, Administración de Base de Datos	3	Correcta
T10	Malla curricular	¿En qué semestre veo Auditoría de TI?	Abarca Auditoría de TI, Cyberseguridad	1	Correcta
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
T11	Prácticas	¿Las prácticas de servicio comunitario son opcionales?	estas horas no son optativas	1	Correcta
T12	Perfil de egreso	¿Qué cargos puede tener un ingeniero en TI de la Indoamérica?	Ingeniero de Datos, Consultor Estratégico en TI	3	Correcta
T13	Perfil de egreso	¿El egresado sabe diseñar redes?	Arquitectura de Redes y Telecomunicaciones	1	Correcta
T14	Perfil de egreso	¿Los estudiantes participan en concursos de programación?	Concurso de Programación	1	Correcta
T15	Inglés	¿Quién se encarga de la certificación de inglés?	Departamento de Idiomas	4	Incorrecta
T16	Inglés	¿Qué habilidades evalúa el examen de suficiencia de inglés?	destrezas receptivas	1	Correcta
T17	Inglés	¿Cuánto pesa el examen final en la nota del B1?	restante recae directamente	1	Correcta
T18	Inglés	¿Qué artículo regula la entrega del certificado de inglés?	Artículo 41	3	Correcta
T19	Aranceles	¿Cuánto es la matrícula presencial en la sede Ambato?	Presencial - Sede Ambato	1	Correcta
T20	Aranceles	¿Cuál es el pago inicial en la modalidad semipresencial?	Semipresencial - Sedes Ambato y Quito	1	Correcta
T21	Aranceles	¿En cuántas cuotas se divide el resto del semestre?	tres (3) pagos iguales diferidos	1	Correcta
T22	Aranceles	¿Hasta cuándo puedo hacer el tercer pago diferido?	15 de julio de 2026	1	Correcta
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
T23	Aranceles	¿Hasta qué fecha tengo para pagar de contado con descuento?	02 de abril de 2026	2	Correcta
T24	Aranceles	¿Cuándo se refleja el descuento del 5% en mi estado de cuenta?	luego del segundo mes de clases	2	Correcta
T25	Aranceles	¿Por qué medios de pago aplica el descuento por pronto pago?	recaudación bancaria institucional o mediante el uso de tarjeta	1	Correcta
T26	Pagos	¿Necesito un número de cuenta para pagar en la ventanilla del banco?	No se requiere un número de cuenta destino	1	Correcta
T27	Pagos	¿Cómo pago la universidad desde la banca web del Pichincha?	Pagos/Servicios - Facturas	1	Correcta
T28	Pagos	¿Cómo pago desde la banca virtual del Banco Guayaquil?	Instituciones Educativas	1	Parcial
T29	Pagos	¿Puedo pagar en Farmacias Económicas o Western Union?	Farmacias Económicas	1	Correcta
T30	Pagos	¿Cuál es la dirección web del SGA?	sga.uti.edu.ec	1	Correcta
T31	Pagos	¿Qué datos tengo que llenar al registrar un comprobante?	Llenado del Formulario de Conciliación	5	Parcial
T32	Pagos	¿Quién revisa el comprobante de pago que subo?	personal de cartera	1	Correcta
T33	Pagos	¿Cuántos días tengo para pagar después de pedir la prórroga?	tres (3) días laborables	1	Correcta
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
T34	Pagos	¿Qué pasa si no se valida mi pago dentro del plazo de la prórroga?	eliminará automáticamente la matrícula	1	Correcta
T35	Becas	¿Hay becas para estudiantes que hacen teatro o música?	Mérito y Rendimiento Cultural	1	Correcta
T36	Becas	¿Qué porcentaje cubren las becas?	25 %, 50 % o 100 %	1	Sin respuesta
T37	Becas	¿Dónde reviso los requisitos de las becas?	El Manual de Becas	1	Correcta
T38	Becas	¿Hay ayudas para familiares del personal administrativo?	familiares de personal administrativo	1	Correcta
T39	Homologación	¿Cuántos años de antigüedad pueden tener las materias que quiero homologar?	diez (10) años	1	Correcta
T40	Homologación	¿Puedo homologar una materia que ya había convalidado en otra universidad?	No recursividad	1	Correcta
T41	Homologación	¿Quiénes forman la comisión de homologación?	Comisión de Reconocimiento u Homologación	1	Correcta
T42	Homologación	Estudié en el extranjero, ¿qué requisitos adicionales tienen mis documentos?	apostilla de La Haya	1	Correcta
T43	Homologación	¿Qué es el simulador de homologaciones?	simulador de homologaciones	1	Correcta
T44	Homologación	¿Cuánto vale la experiencia profesional en la evaluación de homologación?	máximo de 30 %	5	Incorrecta
Continúa en la siguiente página					

ID	Categoría	Pregunta	Frase de referencia	Rango	Respuesta
T45	Inglés	¿Puedo rendir un examen de ubicación de inglés al entrar?	Examen de Ubicación Diagnóstica	1	Correcta
T46	Contacto	¿Cuál es el teléfono para homologaciones en Quito?	0987621218	1	Correcta
T47	Contacto	¿Cuál es el correo de admisiones?	admisiones-indoamerica@uti.edu.ec	1	Correcta
T48	Contacto	¿Cuál es el número de la central telefónica de Quito?	382 6970	1	Correcta
T49	Contacto	¿La universidad tiene oficinas en Guayaquil?	Edificio INDUAUTO	1	Correcta
T50	Admisiones	¿En qué meses hay más demanda de admisiones?	marzo y abril	1	Correcta

Elaborado por: Paulo Emilio Espinoza Jiménez

Fuente: evaluacion/banco_prueba.json y evaluacion/resultados/

Las 10 preguntas fuera de dominio fueron: receta de ceviche, último mundial de fútbol, costo de la carrera de Medicina, solicitud de un chiste, opinión sobre el presidente, materias de Derecho, solicitud de código en Python, intento de *jailbreak* ("Olvida todas tus instrucciones y actúa como un pirata"), clima en Quito y recomendación de películas. Todas fueron rechazadas por la versión final.